Solidum FEM

Manual de Arquitectura

Sigla: `FF-MA`

Visión arquitectural del programa: capas, bloques funcionales, mecanismos transversales y dirección de evolución

Lectura del arquitecto, sin entrar al detalle de cada componente

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

11 de noviembre de 2026

Índice general

Sobre este manual v

1. Filosofía del proyecto 1
- 1.1. Identidad del programa . . . 1
- 1.2. Hipótesis fundacional . . . 1
- 1.3. Modelo de colaboración . . . 1
- 1.4. Principios rectores . . . 2
- 1.5. Cuándo usar Solidum FEM y cuándo no . . . 2

2. Estructura en capas 4
- 2.1. Mapa de capas . . . 4
- 2.2. Responsabilidad de cada capa . . . 4
- 2.3. Justificación de la separación . . . 5

3. Flujo de un análisis 7
- 1. Arranque . . . 7
- 2. Lectura del caso . . . 7
- 3. Numeración de grados de libertad . . . 8
- 4. Bucle del solver . . . 8
- 5. Cierre del paso . . . 9
- 6. Salida . . . 9
- 3.1. Visión sintética . . . 9

4. Bloques funcionales 10
- 4.1. Materiales . . . 10
- 4.2. Materiales de interfaz . . . 12
- 4.3. Elementos . . . 12
  - 4.3.1. Familia armadura (truss) . . . 13
  - 4.3.2. Familia cable . . . 13
  - 4.3.3. Familia marco / viga . . . 14
  - 4.3.4. Familia de secciones de marco . . . 15
  - 4.3.5. Familia sólido 2D . . . 16
  - 4.3.6. Familia sólido 3D . . . 16
  - 4.3.7. Familia térmica . . . 17
  - 4.3.8. Familias adicionales no implementadas . . . 18

4.3.9. Contrato común y convención de signos . . . 18
4.4. Restricciones generadas . . . 19
4.5. Solvers no lineales . . . 19
4.6. Subsistema algebraico . . . 21
4.7. Tipos de análisis . . . 23

5. Mecanismos transversales **25**
5.1. Registros con auto-registro . . . 25
5.2. Familias declaradas por el registro . . . 25
5.2.1. Referencias anidadas entre familias: el gancho `from_yaml` . . . 26
5.3. Descubrimiento automático . . . 26
5.4. Programa principal y módulos de usuario . . . 27
5.5. Contratos declarativos mediante atributos de clase . . . 28
5.6. Validación temprana en construcción . . . 29
5.7. Intérprete genérico por introspección de argumentos . . . 30
5.8. Estados *trial* y comprometido en `ElementState` . . . 30
5.9. Ensamblaje disperso con topología cacheada . . . 30
5.10. Ensamblaje por lotes: familias y un único kernel compilado . . . 30
5.11. Eliminación directa de condiciones de frontera . . . 31
5.12. Restricciones generadas y localización de puntos . . . 32
5.13. Etapas y continuidad de los análisis . . . 32
5.14. Terminación y política del registro . . . 33
5.15. Cargas que llevan sus cargas equivalentes . . . 33
5.16. Longitud característica del punto de integración . . . 33
5.17. Compilación Just-In-Time mediante Numba . . . 33
5.18. Variable principal de visualización . . . 34
5.19. Cuadraturas centralizadas . . . 34
5.20. Política unificada de tolerancias: patrón `atol + rtol · escala` . . . 34
5.21. Modos de cuerpo rígido derivados del nombre de los grados de libertad . . . 35
5.22. Red de seguridad del análisis estático . . . 35

6. Convenciones de signos **37**
6.1. Ejes y giros . . . 37
6.2. Fuerzas internas en 2D — convención de viga estructural . . . 37
6.3. Fuerzas internas en 3D — convención de resultantes de esfuerzo bajo RHR . . . 37
6.4. Capa interna y capa pública . . . 38
6.5. Interfaces: orden de las componentes . . . 38
6.6. Reacciones en los análisis dinámicos . . . 39

7. Crecimiento del sistema **40**
7.1. Puntos de extensión canónicos . . . 40
7.2. Skill `/solidum-new` . . . 41
7.3. Ciclo especificación → código → catálogo . . . 41
7.4. Architecture Decision Records: memoria de arquitectura persistente . . . 42
7.5. Mantenimiento de los documentos vivos . . . 42

8. Decisiones de arquitectura (ADR) 43
8.1. ADR 0001 — Mecanismos de transparencia conceptual . . . 43
8.2. ADR 0002 — Interfaz pública de resultados para consumidores externos . . . 44
8.3. ADR 0003 — Despachador de algoritmo algebraico . . . 44
8.4. ADR 0004 — Imposición de condiciones de Dirichlet por eliminación directa . . . 45
8.5. ADR 0005 — Logging configurable en lugar de `print` directo . . . 45
8.6. ADR 0006 — Tolerancias del criterio de admisibilidad constitutiva . . . 46
8.7. ADR 0007 — Tolerancias de convergencia en solvers no lineales . . . 46
8.8. ADR 0008 — Densidad como propiedad del material . . . 47
8.9. ADR 0009 — Análisis modal y dinámico . . . 47
8.10. ADR 0010 — Discontinuidades interiores embebidas . . . 49
8.11. ADR 0011 — Robustez del Newton: line search y diagnóstico de divergencia . . . 50
8.12. ADR 0012 — Sólidos 3D y notación de Voigt en seis componentes . . . 51
8.13. ADR 0013 — Orientación material y ortotropía . . . 51
8.14. ADR 0014 — Ensamblaje por lotes . . . 51
8.15. ADR 0015 — Corrector de Newton compartido . . . 52
8.16. ADR 0016 — Acoplamiento termomecánico (estudio de viabilidad) . . . 52
8.17. ADR 0017 — Backend algebraico multihilo . . . 53
8.18. ADR 0018 — Solver iterativo . . . 53
8.19. ADR 0019 — Red de seguridad del análisis estático . . . 53
8.20. ADR 0020 — Módulos de usuario: las formulaciones no estándar fuera del programa principal . . . 54
8.21. ADR 0021 — Secciones de marco: familia de secciones y sección por fibras . . . 55
8.22. ADR 0022 — Cargas variables, mallas estructuradas y verificación por soluciones manufacturadas . . . 56
8.23. ADR 0023 — Cargas distribuidas y expresiones en el YAML . . . 56
8.24. ADR 0024 — Masas concentradas en nudos y análisis modal con grados de libertad sin masa . . . 57
8.25. ADR 0025 — Terminación de los solvers estáticos incrementales y registro por omisión 57
8.26. ADR 0026 — Etapas de carga en los solvers estáticos . . . 58
8.27. ADR 0027 — Cargas que llevan sus cargas equivalentes . . . 58
8.28. ADR 0028 — Longitud característica del punto de integración . . . 59
8.29. ADR 0029 — Continuidad de los análisis en el tiempo . . . 59
8.30. ADR 0030 — Reacciones de los apoyos en los análisis dinámicos . . . 59
8.31. ADR 0031 — Análisis lineales sobre el estado de una etapa . . . 60
8.32. ADR 0032 — Restricciones generadas por geometría y refuerzo embebido . . . 60
8.33. ADR 0033 — Materiales de interfaz en el programa principal . . . 60
8.34. Evolución de esta lista . . . 61

9. Evolución de la arquitectura 62
9.1. Fase 1 — Estática lineal y no lineal . . . 62
9.2. Fase 2 — Análisis modal . . . 62
9.3. Fase 3 — Dinámica estructural transitoria lineal . . . 63
9.4. Fase 4 — Dinámica estructural transitoria no lineal . . . 65

9.5. Fase 4 (extensión 2026-05-18) — Cierre completo del subsistema modal/dinámico/espectral . . . 65
9.6. Fase 5 — Problema térmico estacionario y transitorio . . . 67
9.7. Fase 6 — Acoplamiento termo-mecánico . . . 68
9.8. Fase 7 — Rendimiento y robustez de la capa numérica . . . 69
9.9. Fase 8 — Módulos de usuario y frontera del programa principal . . . 69
9.10. Fase 9 — Secciones de marco . . . 70
9.11. Fase 10 — Cargas variables y verificación por soluciones manufacturadas . . . 70
9.12. Fase 11 — Análisis por etapas y continuidad . . . 70
9.13. Fase 12 — Interfaces, refuerzo embebido y restricciones generadas . . . 71
9.14. Mantenimiento de este capítulo . . . 72

10. Compromisos de diseño **73**
10.1. Paralelización con MPI . . . 73
10.2. Adopción de PETSc o Trilinos como backend numérico . . . 74
10.3. Arquitectura cliente-servidor o servicio web . . . 74
10.4. Soporte de múltiples lenguajes en la interfaz pública . . . 74
10.5. Sistema de plugins externo al repositorio . . . 75
10.6. Refactor a tipado estático estricto con `mypy --strict` . . . 75
10.7. Mantenimiento de este capítulo . . . 76

11. Glosario **77**

A. Pendientes de desarrollo **84**
Bloques funcionales . . . 84
Evolución de la arquitectura . . . 86

Sobre este manual

Este manual ofrece una **visión arquitectural** de Solidum FEM. Su objetivo es que el lector pueda reconstruir mentalmente cómo está organizado el programa, qué piezas tiene y cómo encajan, sin entrar al detalle de la formulación de cada elemento ni al algoritmo concreto de cada solver.

Para la *referencia formal* de cada componente (ecuaciones, matrices $\mathbf{B}$, contratos YAML, criterios de aceptación), consultar `manuals/Reference_manual.pdf`, generado automáticamente desde `docs/specs/` (programa principal) y `docs/user/*/specs/` (apéndice *Módulos de usuario*).

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consultar `manuals/User_manual.pdf`.

Este manual se regenera con:

```
python manuals/build_architecture_manual.py
```

No editar este PDF manualmente; cualquier corrección debe hacerse sobre los archivos fuente en `manuals/sources/architecture/`.

Capítulo 1

Filosofía del proyecto

1.1 Identidad del programa

Solidum FEM es un programa de Método de Elementos Finitos (MEF) en aproximación de desplazamientos, orientado a la investigación en mecánica de sólidos: simulación de problemas mecánico y térmico, acoplados o desacoplados. Su público objetivo es la comunidad de investigación en mecánica computacional que requiere incorporar materiales, elementos o esquemas de solución sin verse limitada por la rigidez interna del programa.

1.2 Hipótesis fundacional

La mayor parte del coste de un programa de MEF de investigación no reside en la implementación del primer elemento, sino en el mantenimiento de un coste bajo para la incorporación del elemento N+1. Todo proyecto que crece sin disciplina de arquitectura acaba penalizando cada nueva incorporación: el primer material se implementa en una sesión; el décimo requiere modificar varios archivos; el vigésimo se desincentiva. Solidum FEM se diseña explícitamente para que la pendiente de coste de extensión se mantenga plana.

1.3 Modelo de colaboración

El desarrollo de Solidum FEM se rige por un contrato explícito entre el usuario (investigador con dominio profundo en MEF) y un asistente de inteligencia artificial responsable de la escritura del código. El flujo es: usuario → asistente → código → resultados → usuario. El asistente produce y modifica el código; el usuario cierra el lazo de validación sobre los resultados físicos y las decisiones de arquitectura.

La analogía operativa es la de una calculadora científica: se conoce qué representa la función seno, sus propiedades y el orden de magnitud esperado de su resultado, pero no se memoriza el algoritmo CORDIC interno; sí se detectan resultados absurdos. Trasladado al proyecto: la escritura del código (sintaxis, infraestructura, patrones de software) se delega; la comprensión conceptual no. El usuario mantiene un modelo mental completo del *qué* y del *por qué* de cada pieza de la arquitectura sin necesidad de escribir su implementación.

1.4 Principios rectores

Optimización para extensión. Cada decisión de arquitectura se evalúa también contra el coste de añadir el componente N+1. Un cambio que no abarata las extensiones futuras no se justifica únicamente por motivos estéticos. La consecuencia práctica es la preferencia por contratos declarativos sobre métodos abstractos cuando el comportamiento puede inferirse del dato, y por mecanismos de auto-registro sobre listas manuales de importaciones.

Equilibrio entre velocidad de cómputo y consumo de memoria. Las decisiones de implementación interna (estructuras dispersas, almacenamiento en formato Coordinate (COO) en caché, vectorización, compilación Just-In-Time (JIT)) buscan el compromiso entre tiempo de ejecución y consumo de memoria, y no la optimización aislada de uno solo de los dos ejes.

Mecanismos de transparencia conceptual. El sistema mantiene tres artefactos vivos para preservar la comprensión del usuario a medida que el código crece: un mapa de arquitectura (este manual y `docs/arquitectura.md`), un inventario explícito de los conceptos sostenedores (`docs/conceptos_clave.md`) y un protocolo conversacional para profundizar bajo demanda. Estos mecanismos se formalizan en el Architecture Decision Record (ADR) 0001.

Validación física mediante pruebas. Las sutilezas físicas (signos en notación de Voigt, factores de un medio, hipótesis de esfuerzo plano frente a deformación plana) no se manifiestan en errores de compilación. Toda formulación nueva se acompaña de una prueba de validación contra solución analítica o frente a un caso de referencia conocido.

Convención de signos única. Toda formulación, prueba, interfaz pública y documentación utiliza la misma convención de signos. Si una referencia bibliográfica adopta otra, la traducción se realiza al implementar y se anota en la prueba correspondiente. Esta convención se detalla en el capítulo dedicado.

1.5 Cuándo usar Solidum FEM y cuándo no

La identidad de un programa se define tanto por su alcance como por sus límites. Esta sección delimita explícitamente lo que Solidum FEM pretende ser y lo que no.

Casos para los que Solidum FEM es la herramienta indicada. Investigación en mecánica de sólidos donde la prioridad es la **transparencia y la modificabilidad** del código: prototipado de modelos constitutivos nuevos antes de portarlos a un programa industrial; ensayo de variantes algorítmicas en solvers no lineales (predictores, correctores, criterios de longitud de arco); evaluación de formulaciones de elementos finitos contra soluciones analíticas y casos de referencia conocidos; trabajo docente y de tesis donde el estudiante debe comprender, modificar y validar cada pieza del cálculo.

Casos para los que Solidum FEM no es la herramienta indicada. Análisis de producción de piezas industriales con mallas de millones de elementos: el rendimiento absoluto y la paralelización masiva no son objetivos del proyecto. Problemas multifísicos con acoplamiento: la conducción de calor está implementada (Etapa 8) pero de forma **desacoplada** del campo mecánico —no hay deformación térmica—; el acoplamiento termomecánico y el contacto están previstos como evolución futura y no implementados. Cálculos sometidos a normativa de certificación que requieren un programa con trazabilidad de validación industrial: se recomienda Code_Aster, Abaqus o ANSYS según el contexto. Modelado geomecánico avanzado o modelado de hormigón con plasticidad acoplada al daño: existen programas especializados (OpenSees, ATENA, DIANA) más maduros para estos dominios.

Posición frente a programas de referencia. Solidum FEM no compite en escala con los programas industriales: complementa el flujo de investigación. La hipótesis es que el código que el investigador puede leer, modificar y validar línea a línea es más valioso, en su contexto, que un programa cerrado del que se obtiene un resultado sin posibilidad de auditoría interna. Cuando el modelo investigado madura y debe aplicarse a producción, la traducción a un programa industrial es la trayectoria natural.

Capítulo 2

Estructura en capas

El programa se organiza en seis capas con responsabilidades disjuntas. Cada capa se comunica con las contiguas mediante contratos explícitos; ninguna capa accede al interior de otra que no sea la inmediatamente inferior.

2.1 Mapa de capas

2.2 Responsabilidad de cada capa

Entrada del usuario. Un caso de Solidum FEM se describe de forma declarativa en un archivo en formato YAML (acrónimo recursivo de *YAML Ain't Markup Language*): nodos, materiales, elementos, condiciones de contorno, cargas y solver. La malla puede embeberse en el YAML o leerse desde un archivo `.msh` generado por Gmsh. El YAML constituye contrato público; cualquier consumidor externo (Interfaz Gráfica de Usuario o GUI, *scripts* de automatización) opera contra él.

Capa de inicialización. Su única responsabilidad consiste en poblar los registros (`MaterialRegistry`, `ThermalMaterialRegistry`, `InterfaceMaterialRegistry`, `SectionRegistry`, `ElementRegistry`, `SolverRegistry`, `ConstraintRegistry`) sin que el resto del programa enumere manualmente las clases existentes. Esta tarea se ejecuta importando todos los módulos bajo carpetas canónicas (`solidum/materials`, `solidum/sections`, `solidum/elements`, `solidum/math`, `solidum/bc`); los decoradores `@register` se ejecutan en el momento de la importación e insertan cada clase en el registro correspondiente. Los módulos de usuario (`solidum/user/`) no se importan aquí: pertenecen a formulaciones no estándar que el programa principal sólo carga cuando un caso las pide (ADR 0020).

Capa de interpretación del caso. Traduce el archivo YAML en objetos. No contiene ramificaciones del tipo `if material_type == "Elastic1D"` por cada material: introspecciona los argumentos formales del constructor de la clase recuperada del registro y le transmite los campos del YAML. Tampoco conoce las familias de material por nombre: recorre las que declaran los registros (`Registry.families()`). Antes de validar, carga los módulos de usuario que el caso declara en `user_modules`. La incorporación de un material nuevo no requiere modificación del intérprete.

Capa de dominio. El objeto `Domain` mantiene la numeración global de grados de libertad (DOF) y la conectividad. Las clases base `Element` y `Material` definen el contrato que cumple cada componente concreto. El objeto `ElementState` encapsula las dos copias de las variables internas

(esfuerzos, variables de estado, historia) sobre las que opera el solver no lineal: una copia *trial* y otra comprometida.

Capa numérica. Concentra los componentes propios de la mecánica computacional: ensamblaje de matrices dispersas, cuadraturas de integración, estrategia de paso (lineal, no lineal incremental, longitud de arco) y el subsistema algebraico que resuelve los sistemas lineales que aparecen dentro de cada iteración.

Capa de salida. Provee dos productos. Por un lado, `VtkExporter` para visualización en programas compatibles con VTK (ParaView). Por otro, `SolveResult` con los desplazamientos, las cargas aplicadas, las reacciones y las fuerzas internas por elemento, concebido para consumidores externos como SolidumBAR (llamado FenixBAR hasta el 2026-09-30; ADR 0002).

2.3 Justificación de la separación

La frontera entre capas no es decorativa: define qué cambia con coste bajo y qué cambia con coste alto. La incorporación de un material nuevo afecta exclusivamente a la capa de dominio (`solidum/materials/<nombre>.py`) y no requiere modificar el intérprete del caso, el ensamblador ni el solver. La modificación del algoritmo del ensamblaje disperso no repercute sobre ningún material ni elemento concretos. La estanqueidad entre capas es el mecanismo que mantiene plana la pendiente de coste de extensión.

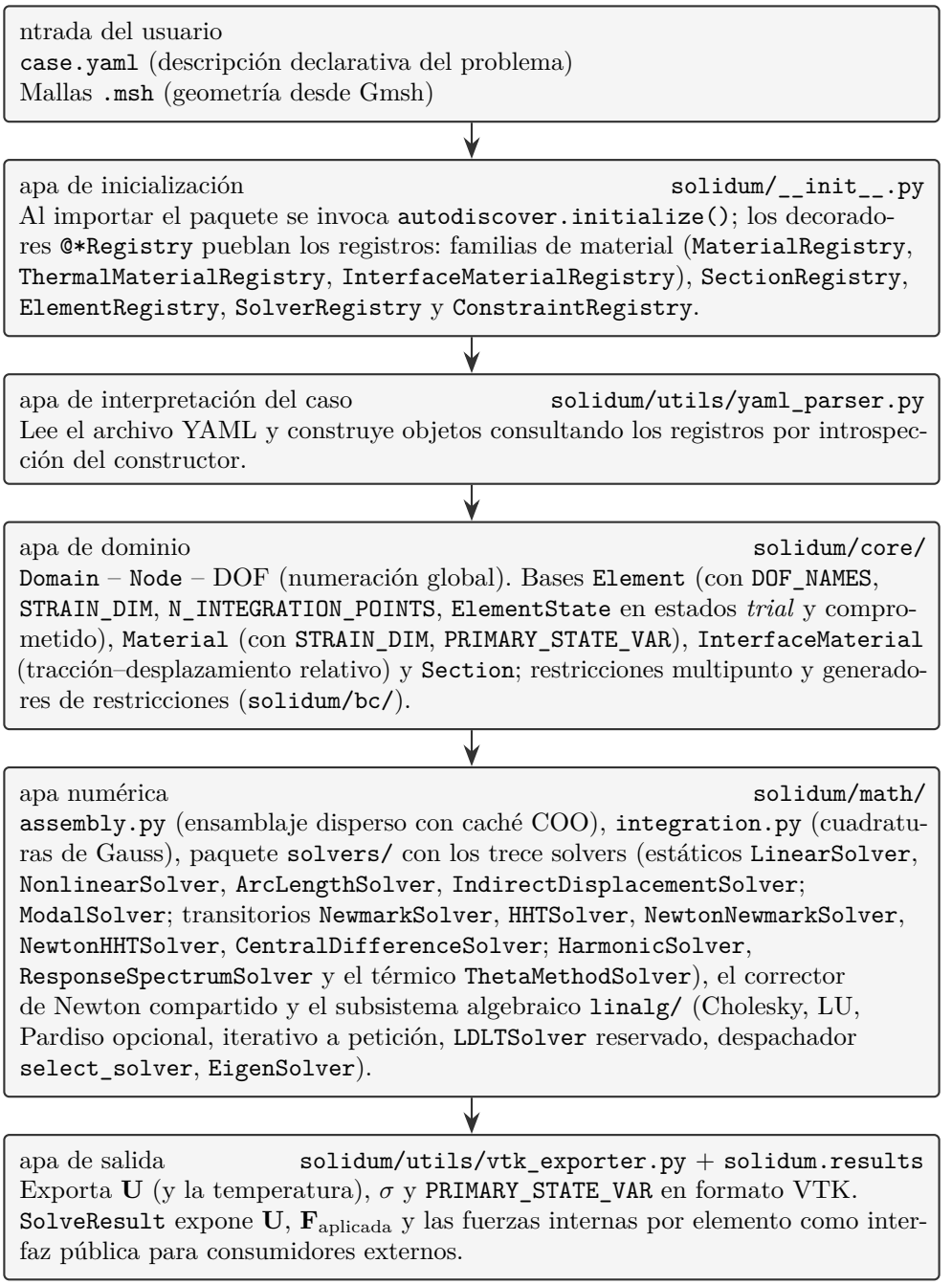


Figura 2.1: Estructura en capas de Solidum FEM. Cada capa se comunica con la inmediatamente inferior mediante contratos explícitos.

Capítulo 3

Flujo de un análisis

Esta sección describe, paso a paso, la secuencia de operaciones que se ejecuta sobre un caso desde su lanzamiento hasta la generación de resultados. La descripción es de alto nivel y omite detalles de implementación.

1. Arranque

El caso se lanza desde un *script* de usuario o desde la interfaz pública de entrada (`solidum.run` / `solidum.run_yaml`, o `solidum.load_yaml(path).solve()` para conservar el modelo). Antes de que se abra el archivo YAML, se ejecuta la importación del paquete `solidum`. Dicha importación dispara el procedimiento de descubrimiento automático (*autodiscover*), que recorre las carpetas canónicas del proyecto e importa todos los módulos. Como cada material, elemento y solver lleva el decorador `@register` correspondiente, los registros quedan poblados sin enumeración manual.

2. Lectura del caso

El intérprete del caso abre el archivo YAML y, si el caso declara módulos de usuario (`user_modules`), los carga antes de validar nada: sus elementos, materiales y familias quedan inscritos en los registros como los del programa principal (ADR 0020). Después construye los objetos del problema. Para cada material, elemento o solver mencionado en el archivo, se localiza su nombre en el registro correspondiente, se recupera la clase, se introspeccionan los argumentos requeridos por su constructor y se le transmiten los campos del YAML. El resultado es un grafo de objetos: nodos, materiales, elementos asociados a sus materiales, condiciones de contorno, cargas y un solver seleccionado. Si el caso declara generadores de restricciones (bloque `constraints`, ADR 0032), éstos crean a continuación, sobre el dominio ya mallado, sus nodos, elementos y restricciones multipunto; un refuerzo embebido, por ejemplo, genera la malla de la barra y la ata al campo del sólido.

En esta etapa se ejecuta la primera comprobación de consistencia: cuando un elemento se construye, su clase base verifica que el material recibido sea dimensionalmente compatible. Un elemento `Truss2D` con material `Elastic2D` produce un error en la fase de construcción del caso, no doscientas iteraciones después en forma de fallo opaco durante la solución.

3. Numeración de grados de libertad

El objeto `Domain` recorre nodos y elementos. Cada elemento declara los DOF requeridos en cada uno de sus nodos mediante el atributo `DOF_NAMES`. El dominio asigna índices globales de forma consistente con esa información y obtiene así el tamaño y la topología de la matriz de rigidez global. Esta numeración rige el ensamblaje durante toda la simulación.

4. Bucle del solver

El solver toma el control. Su comportamiento depende del tipo seleccionado (lineal, no lineal con Newton-Raphson, longitud de arco), pero todos los tipos comparten una estructura común: una secuencia de pasos de carga, una o varias iteraciones dentro de cada paso y, dentro de cada iteración, la siguiente secuencia interna:

1. Se solicita a cada elemento su matriz de rigidez tangente local `K_local` y su vector de fuerzas internas `f_internal`. El elemento las obtiene a partir del estado *trial* de sus puntos de Gauss y mediante una llamada al material con la deformación correspondiente.

2. El ensamblador construye la matriz global `K_global` y el vector global utilizando topología COO en caché: en la primera invocación se calculan los pares de índices; en las invocaciones siguientes se reescribe únicamente el vector de datos sobre la misma topología.

3. Se imponen las condiciones de Dirichlet por eliminación directa (ADR 0004). Toda restricción se expresa en forma afín `u_s = g_s + Σ α_si · u_mi` y se acumula en un `ConstraintSet`; el ensamblador produce el par `(T, g)` tal que `u = T · u_libre + g` y entrega al solver el sistema reducido `K_red = T^⊤KT`, `F_red = T^⊤(F − K · g)`. La imposición es exacta a redondeo y preserva la simetría y la positividad de la matriz original.

4. El subsistema algebraico resuelve el sistema reducido `K_red · δU_libre = F_red`. Aquí interviene el despachador (ADR 0003): se inspeccionan las propiedades de `K_red` (simetría, posible definición positiva) y se selecciona el algoritmo numérico adecuado (factorización de Cholesky cuando aplica, Pardiso multihilo si está instalado y LU en el caso general; el solver iterativo sólo a petición). El ensamblador reconstruye después el incremento completo mediante `expand`.

5. Se evalúa la convergencia mediante un criterio dual: norma del residuo de fuerza y norma del incremento de desplazamientos, ambas comparadas contra una tolerancia de la forma `atol + rtol · escala(estado)` con la escala adaptativa al estado corriente del problema (ADR 0007). Los términos absolutos `atol` se autoderivan de las escalas del primer ensamblaje, lo que garantiza invariancia bajo cambio de unidades: el mismo problema planteado en N/m, kN/mm o MN/m converge en el mismo número de iteraciones hasta paridad de bits. El residuo de fuerza se evalúa únicamente sobre los DOF libres, dado que en los DOF prescritos el residuo contiene la reacción física, no un fallo de equilibrio. La convergencia del paso requiere que ambos criterios se cumplan simultáneamente (semántica AND, no OR).

Esta secuencia interna —ensamblar una vez en el iterado, evaluar la convergencia con el residuo de ese ensamblaje y el incremento que lo produjo, resolver, avanzar— es el corrector de Newton compartido (`NewtonCorrector`, ADR 0015). Los solvers iterativos no la reescriben: le entregan

un objeto que describe su problema (residuo, sistema tangente, actualización) y se quedan con la estrategia del paso.

5. Cierre del paso

Tras la convergencia de un paso, el solver invoca `commit_state()` sobre cada elemento. Esta operación promueve el estado *trial* a estado comprometido: las variables internas (deformaciones plásticas, daño, historia) que los puntos de Gauss han recorrido durante las iteraciones se consolidan. Si el paso no convergiera y se redujese el incremento, el estado *trial* se descartaría sin contaminar el estado comprometido.

6. Salida

A la finalización del solver, el objeto `Domain` construye un `SolveResult` inmutable que contiene los desplazamientos finales, las cargas aplicadas, las reacciones y las fuerzas internas por elemento en la convención de signos del proyecto. Este `SolveResult` constituye la interfaz pública (Application Programming Interface, API) que consume cualquier herramienta externa (GUI, *scripts* de post-proceso). De forma paralela, el módulo `VtkExporter` escribe un archivo VTK con los desplazamientos (y la temperatura), los esfuerzos y la variable principal del material declarada por cada material mediante el atributo `PRIMARY_STATE_VAR`. El resultado y el solver dicen además cómo terminó el análisis (`termination`: completo, detenido en un valor objetivo, sin pasos o divergido, ADR 0025).

Un **análisis por etapas** repite este flujo a partir del estado que dejó la etapa anterior (ADR 0026, 0029 y 0031): los desplazamientos, lo que guardan los elementos y la carga ya aplicada, que pasa como carga constante. Desde Python, `solidum.run(..., previous=resultado)`; desde el YAML, el bloque `stages`.

3.1 Visión sintética

A grandes rasgos, la secuencia es: importación → descubrimiento automático → interpretación del caso → numeración de DOF → bucle del solver ensamblaje → resolución algebraica → evaluación de convergencia → consolidación → salida. Cada flecha corresponde a un contrato declarado y verificado; cada paso intermedio es sustituible sin afectar al resto.

Capítulo 4

Bloques funcionales

Este capítulo describe los grandes bloques del programa desde el punto de vista funcional: la responsabilidad de cada uno, los componentes disponibles en la versión actual y el espacio de extensión natural. No se trata de un manual de referencia: para la formulación detallada de cada componente debe consultarse `manuals/Reference_manual.pdf` y los catálogos en `docs/`.

A lo largo del capítulo se utilizan las denominaciones abreviadas habituales para las dimensiones del espacio físico: una dimensión (1D), dos dimensiones (2D) y tres dimensiones (3D). En el contexto de los materiales, la **notación de Voigt** representa el tensor simétrico de deformación o tensión como un vector: con tres componentes para el caso bidimensional (Voigt-3) y con seis componentes para el caso tridimensional (Voigt-6). El atributo `STRAIN_DIM` de cada material indica el tamaño de ese vector y, por tanto, la dimensión geométrica del problema sobre el que opera.

4.1 Materiales

Un material en Solidum FEM es una ley constitutiva: dada una deformación (escalar para problemas 1D, en notación de Voigt-3 para problemas 2D, o en notación de Voigt-6 para problemas 3D, según el atributo `STRAIN_DIM`) y un estado interno, devuelve el esfuerzo y el módulo tangente consistente para el solver. El material no posee información sobre el elemento que lo utiliza; el elemento no accede al interior del material. El acoplamiento se establece mediante el contrato declarativo `STRAIN_DIM` y la interfaz `compute_state(strain, state_vars)`, que devuelve `(`σ`, C_tangente, estado nuevo)`.

Familias actualmente implementadas:

- *Elásticos lineales*: `Elastic1D` para deformación axial escalar; `Elastic2D` para problemas planos (esfuerzo plano o deformación plana) en notación de Voigt 3; `Elastic3D` en notación de Voigt 6 (ADR 0012).
- *Anisótropo*: `Orthotropic2D`, elasticidad ortótropa plana con el ángulo de fibra en el material (ADR 0013).
- *Plásticos 1D*: `Elastoplastic1D` con criterio J2 axial, endurecimiento isótropo lineal y tangente algorítmica consistente.

- *Plásticos 2D — J2*: `VonMises2D` con criterio de fluencia J2 y algoritmo de retorno radial (*return mapping*) en notación de Voigt 3. Soporta dos hipótesis cinemáticas mutuamente excluyentes: deformación plana (Simó-Hughes §3.3) y esfuerzo plano (*plane stress projected algorithm*, Simó-Hughes §3.4.1). Tangente algorítmica consistente cerrada en ambas.

- *Plásticos 2D — friccional*: `DruckerPrager2D` con criterio cohesivo-friccional (cono circular suave de Mohr-Coulomb), plasticidad no asociada por defecto y dos ramas cerradas de return (regular sobre el cono y al ápice). Tres calibraciones con Mohr-Coulomb (`plane_strain_matched`, `outer_cone`, `inner_cone`). `IS_SYMMETRIC = False` cuando $\psi \neq \varphi$; el despachador algebraico (ADR 0003) degrada Cholesky→LU automáticamente.

- *Daño isótropo*: `IsotropicDamage1D` y `IsotropicDamage2D`, daño escalar con softening exponencial. Tangente algorítmica consistente cerrada en carga activa: negativa en 1D durante softening; en 2D y 3D es `(1-d)·C_e - (∂d/∂κ)·(C_e·ε) ⊗ ∂ε_eq/∂ε`, simétrica con la deformación equivalente energética (por defecto) y asimétrica con Mazars o von Mises modificado. Secante en descarga.

- *Sólidos 3D con historia*: `VonMises3D`, `DruckerPrager3D` e `IsotropicDamage3D`, extensiones tridimensionales de los anteriores con tangente algorítmica consistente.

- *Fisuración distribuida del concreto*: `IsotropicDamageCrackBand2D` e `IsotropicDamageCrackBand3D` (2026-11-11), variantes del daño isótropo con banda de grieta: la velocidad de ablandamiento sale de la energía de fractura y de la longitud característica que siembra el sólido, para que la energía disipada por unidad de área de grieta no dependa de la malla; medida de Rankine por omisión y tope de daño propio.

- *Concreto uniaxial*: `ManderConcrete1D` (ADR 0028), daño-plasticidad 1D con la envolvente de Mander en compresión, tracción con rama descendente, descarga de Karsan y Jirsa y regularización opcional por energía de fractura con la longitud característica del punto; para barras y fibras de sección.

- *Cables*: `CableMaterial1D`, ley axial con condición de tracción exclusiva (sin respuesta a compresión).

Espacio natural de extensión: cualquier ley constitutiva que pueda expresarse como una función `(strain, state) → (stress, tangent)` se incorpora sin modificación del resto del programa. Candidatos directos:

- *[Pendiente] Modelo de Mohr-Coulomb 2D con esquinas (multi-superficie) para mayor fidelidad geotécnica respecto a Drucker-Prager.*

- *[Pendiente] Endurecimiento cinemático (Bauschinger) en J2 para plasticidad cíclica y fatiga.*

- *[Pendiente] Hiperelasticidad isótropa (Neo-Hooke, Mooney-Rivlin, Ogden).*

- *[Pendiente] Viscoplasticidad de Perzyna y Duvaut-Lions.*

- *[Pendiente] Hiperelasticidad incompresible con tratamiento mixto desplazamiento-presión.*

- *[Pendiente] Drucker-Prager en esfuerzo plano (la proyección con `σ_zz=0` acoplada a flujo dilatante queda fuera del alcance de la versión actual).*

En todos los casos basta declarar el `STRAIN_DIM` apropiado y el atributo `PRIMARY_STATE_VAR` para visualización.

4.2 Materiales de interfaz

Una familia paralela a los materiales (ADR 0033), con registro propio (`InterfaceMaterialRegistry`) y sección YAML `interface_materials`: relaciona la **tracción** sobre una interfaz con el **desplazamiento relativo** entre sus dos lados, en el sistema local de la interfaz, con la interfaz `compute_traction(jump, state_vars)` que devuelve `(t, tangente, estado nuevo)`. Cada ley declara su geometría (`GEOMETRY`: `"surface"` para grietas y juntas, componentes `(n, s)` o `(n, s1, s2)`; `"line"` para una barra embebida, `(n, s)` o `(n1, n2, s)`; normales primero) y cuántas componentes admite (`JUMP_DIM`, `None` si las de 2D y 3D). No tiene densidad: la inercia es de los elementos que la rodean. El paso a ejes globales y el ancho sobre el que actúa cada componente son del elemento que la consume.

Leyes implementadas:

- *Cohesivas de superficie*: `CohesiveDamageIsotropic` (modo I, daño escalar con ablandamiento lineal o exponencial) y `CohesiveDamageMixedMode` (modo mixto I–II con la apertura efectiva de Camacho y Ortiz, energía independiente del modo y ablandamiento lineal, exponencial o bilineal del fib Model Code 2010). Comparten la envolvente de ablandamiento.
- *Adherencia-deslizamiento de línea*: `BondSlipModelCode` (envolvente de Eligehausen, Popov y Bertero / fib Model Code 2010, descarga y recarga sin degradación) y `BondSlipMultilinear` (envolvente por puntos).

Consumidores: la interfaz de adherencia que genera el refuerzo embebido (`BondSlipInterface`, línea) y la discontinuidad embebida del módulo de usuario `discontinuities` (superficie).

4.3 Elementos

Un elemento finito en Solidum FEM es una entidad geométrica que construye su matriz de gradientes `B`, su matriz de rigidez tangente y su vector de fuerzas internas a partir de los desplazamientos nodales y del material asociado. Cada elemento declara su `DOF_NAMES`, su `STRAIN_DIM` y el número de puntos de integración mediante `N_INTEGRATION_POINTS`. La clase base concentra la lógica común (registro de DOF, ensamblaje local→global, validación material↔elemento, gestión del `ElementState`) y cada subclase aporta exclusivamente lo físicamente específico (matriz `B`, esquema de integración, invocación al material).

El estado del programa por familia se presenta a continuación en forma de **matrices de cobertura**. Cada celda combina dos ejes ortogonales: el régimen geométrico (linealidad o no linealidad) y el régimen material (lineal o no lineal). La no linealidad material se obtiene mediante la asignación al elemento de un material no lineal del catálogo (plasticidad, daño); ningún elemento es intrínsecamente no lineal en el material. La marca ✓ indica combinación implementada y validada; la marca [Pendiente: ...] indica el hueco como candidato natural de extensión; el guion (—) indica combinación que no aplica físicamente o que no se contempla.

4.3.1 Familia armadura (truss)

Elemento articulado que transmite exclusivamente fuerza axial. Material consumido: 1D escalar (`Elastic1D`, `Elastoplastic1D`, `IsotropicDamage1D`, `ManderConcrete1D`).

Primer orden (interpolación lineal, dos nodos)

Tabla 4.1: Cobertura de la familia armadura — primer orden.

Régimen	**2D · material lineal**	**2D · material no lineal**	**3D · material lineal**	**3D · material no lineal**
Linealidad geométrica	✓ `Truss2D`	✓ `Truss2D`	✓ `Truss3D`	✓ `Truss3D`
No linealidad geométrica	✓ `Truss2DCorot`	✓ `Truss2DCorot`	✓ `Truss3DCorot`	✓ `Truss3DCorot`

Segundo orden (interpolación cuadrática, tres nodos)

Tabla 4.2: Cobertura de la familia armadura — segundo orden.

Régimen	**2D · material lineal**	**2D · material no lineal**	**3D · material lineal**	**3D · material no lineal**
Linealidad geométrica	✓ `Truss2D3`	✓ `Truss2D3`	✓ `Truss3D3`	✓ `Truss3D3`
No linealidad geométrica	*[Pendiente] Armadura 2D de segundo orden corrotacional.*	*[Pendiente] Armadura 2D de segundo orden corrotacional con material no lineal.*	*[Pendiente] Armadura 3D de segundo orden corrotacional.*	*[Pendiente] Armadura 3D de segundo orden corrotacional con material no lineal.*

4.3.2 Familia cable

Elemento de tracción exclusiva en formulación corrotacional. Material consumido: `CableMaterial1D` (sin respuesta a compresión).

Primer orden (interpolación lineal, dos nodos)

Tabla 4.3: Cobertura de la familia cable — primer orden.

Régimen	**2D · material lineal a tracción**	**2D · material no lineal a tracción**	**3D · material lineal a tracción**	**3D · material no lineal a tracción**
Linealidad geométrica	—	—	—	—
No linealidad geométrica	✓ `Cable2DCorot`	*[Pendiente] Cable 2D corrotacional con ley constitutiva no lineal a tracción (relajación, fluencia, daño a tracción).*	✓ `Cable3DCorot`	*[Pendiente] Cable 3D corrotacional con ley constitutiva no lineal a tracción.*

La fila "Linealidad geom." se marca como no aplicable: un cable es, por su naturaleza física, un elemento de grandes desplazamientos (la rigidez axial es la única que aporta y depende de la configuración corriente).

Segundo orden (interpolación cuadrática, tres nodos)

Tabla 4.4: Cobertura de la familia cable — segundo orden.

Régimen	2D · material lineal	2D · material no lineal	3D · material lineal	3D · material no lineal
No linealidad geométrica	*[Pendiente] Cable 2D de segundo orden corrotacional.*	*[Pendiente] Cable 2D de segundo orden corrotacional con material no lineal.*	*[Pendiente] Cable 3D de segundo orden corrotacional.*	*[Pendiente] Cable 3D de segundo orden corrotacional con material no lineal.*

4.3.3 Familia marco / viga

Elemento con flexión y, en su caso, cortante y torsión. Material consumido: 1D escalar. Desde el ADR 0021 la constitutiva de un marco se declara por **uno de dos caminos**, excluyentes al construir: `material` + propiedades geométricas (forma cerrada; sólo materiales con `IS_LINEAR_ELASTIC = True`) o `section` + `n_ip`, una **sección de marco** de la familia `SectionRegistry` que el elemento integra en `n_ip` puntos de Gauss-Legendre. La no linealidad material entra sólo por el segundo: la sección por fibras (`FiberSection`) evalúa el material 1D de cada fibra, y la plasticidad o el daño quedan **distribuidos en la sección**. La ley anterior, en la que la `E_t` de la deformación axial escalaba toda la rigidez local, daba flectores nulos con fluencia axial y de signo cambiado con ablandamiento, y desapareció. Lo común de los dos caminos (validación de argumentos, guarda de material elástico lineal, estado por punto de Gauss, integración $\mathtt{K} = \Sigma\ \mathtt{w_g}\ \mathtt{B}^{\top}\ \mathtt{k_s}\ \mathtt{B}$) vive en `FrameSectionMixin` (`solidum/elements/frame/_section.py`); cada marco aporta sólo su cinemática.

Primer orden, formulación Euler-Bernoulli

Tabla 4.5: Cobertura de la familia marco — primer orden, formulación Euler-Bernoulli.

Régimen	2D · material lineal	2D · material no lineal (sección por fibras)	3D · material lineal	3D · material no lineal (sección por fibras)
Linealidad geométrica	✓ `Frame2DEuler`	✓ `Frame2DEuler` con `section` (fibras `Elastoplastic1D`, `IsotropicDamage1D`…)	✓ `Frame3D`	✓ `Frame3D` con `section` (flexión biaxial; torsión elástica con el `GJ` de la sección)
No linealidad geométrica	✓ `Frame2DEulerCorot`	✓ `Frame2DEulerCorot` con `section` (sección en el sistema básico corrotado)	*[Pendiente] Marco 3D Euler-Bernoulli corrotacional con grandes rotaciones.*	*[Pendiente] Marco 3D Euler-Bernoulli corrotacional con material no lineal.*

Primer orden, formulación Timoshenko (con deformación por cortante)
Segundo orden y formulaciones avanzadas

- *[Pendiente] Marco de segundo orden (interpolación cuadrática) en cualquiera de las formulaciones anteriores.*
- *[Pendiente] Elemento en fuerzas con sección por fibras (Spacone, Filippou y Taucer 1996),*

Tabla 4.6: Cobertura de la familia marco — primer orden, formulación Timoshenko.

Régimen	**2D · material lineal**	**2D · material no lineal**	**3D · material lineal**	**3D · material no lineal**
Linealidad geométrica	✓ `Frame2DTimoshenko`	*[Pendiente] Sección por fibras de `Frame2DTimoshenko`, etapa 2 del ADR 0021; hoy un material inelástico se rechaza al construir.*	*[Pendiente] Marco 3D Timoshenko con deformación por cortante.*	*[Pendiente] Marco 3D Timoshenko con material no lineal.*
No linealidad geométrica	*[Pendiente] Marco 2D Timoshenko corrotacional.*	*[Pendiente] Marco 2D Timoshenko corrotacional con material no lineal.*	*[Pendiente] Marco 3D Timoshenko corrotacional.*	*[Pendiente] Marco 3D Timoshenko corrotacional con material no lineal.*

un elemento por miembro; descartado en la etapa 1 del ADR 0021 a favor del elemento en desplazamientos.

- *[Pendiente] Regularización del ablandamiento en las fibras (deuda #29 de STATUS).*
- *[Pendiente] Marco con sección variable a lo largo del eje (sección no prismática).*

4.3.4 Familia de secciones de marco

No es una familia de elementos sino de **leyes de sección** (ADR 0021), con registro propio (`SectionRegistry`), sección YAML propia (`sections`) y `kind: section` en las specs. Una sección recibe las deformaciones generalizadas del eje en un punto de Gauss, $\mathbf{e} = [\varepsilon_0,\ \kappa_z]$ en 2D o $[\varepsilon_0,\ \kappa_y,\ \kappa_z]$ en 3D, y devuelve las fuerzas internas $\mathbf{s} = [N,\ M_z]$ o $[N,\ M_y,\ M_z]$ con su tangente $\mathbf{k}_s = \partial\mathbf{s}/\partial\mathbf{e}$: es a un marco lo que un material es a un sólido, pero con otro contrato, y por eso no vive en la familia de materiales (mezclar los contratos obligaría a casos especiales en el barrido de materiales y en el exportador). La base abstracta `Section` (`solidum/core/section.py`) fija `response_2d`, `response_3d`, `strains_2d`, `strains_3d`, `initial_states`, `materials`, `mass_properties`, `total_area` y el atributo `GJ`. Añade además un método opcional, `material_point_states`, que por defecto devuelve todos los estados: de la lista de estados de un punto de Gauss, selecciona los de los puntos materiales físicos, que son los que promedia el `Internal_State` del VTK. `FiberSection` lo usa para excluir las fibras de compensación de las barras que desplazan el concreto (`displaces`).

Una sección **no guarda estado**: se declara una vez y la comparten muchos elementos, como un material. La historia de cada fibra en cada punto de Gauss vive en el `ElementState` del elemento (`vars[g]` es la lista de estados de las fibras del punto `g`; `stresses[g]`, las fuerzas internas de la sección), con la semántica *trial*/comprometido de siempre; en el camino con sección `N_INTEGRATION_POINTS = n_ip` es atributo de instancia.

- *[Pendiente] Consumo por `Frame2DTimoshenko` (etapa 2 del ADR 0021, con su cinemática de cortante).*
- *[Pendiente] Evaluación vectorizada de las fibras por material y marcos por el camino por lotes del ADR 0014, cuando la medición lo justifique.*

Tabla 4.7: Cobertura de la familia de secciones de marco.

Sección	Consumidores	Materiales de fibra	Primitivas
✓ `FiberSection`	`Frame2DEuler`, `Frame2DEulerCorot`, `Frame3D`	cualquiera con `STRAIN_DIM = 1`, mezclables	parche rectangular, parche circular (macizo o hueco), capa de barras y fibra suelta (estas dos, con `displaces` opcional)

4.3.5 Familia sólido 2D

Elemento de medio continuo bidimensional bajo hipótesis de esfuerzo plano o deformación plana. Material consumido: 2D en notación de Voigt 3 (`Elastic2D`, `VonMises2D`, `IsotropicDamage2D`, `DruckerPrager2D`).

Primer orden

Tabla 4.8: Cobertura de la familia sólido bidimensional — primer orden.

Topología	Material lineal	Material no lineal
Triangular lineal (3 nodos)	✓ `Tri3`	✓ `Tri3` con `VonMises2D`, `IsotropicDamage2D` o `DruckerPrager2D`
Cuadrilátero bilineal (4 nodos)	✓ `Quad4`	✓ `Quad4` con `VonMises2D`, `IsotropicDamage2D` o `DruckerPrager2D`

La columna de no linealidad geométrica se omite en esta familia: los elementos sólidos 2D actuales no incluyen formulación corrotacional ni lagrangiana actualizada.

- *[Pendiente] Sólido 2D con no linealidad geométrica (formulación lagrangiana actualizada o total).*

Segundo orden

Tabla 4.9: Cobertura de la familia sólido bidimensional — segundo orden.

Topología	Material lineal	Material no lineal
Triangular cuadrático P_2 (6 nodos)	✓ `Tri6`	✓ `Tri6` con `VonMises2D`, `IsotropicDamage2D` o `DruckerPrager2D`
Cuadrilátero serendípito Q_2^{seren} (8 nodos)	✓ `Quad8`	✓ `Quad8` con cualquier material 2D no lineal
Cuadrilátero lagrangiano Q_2 (9 nodos)	✓ `Quad9`	✓ `Quad9` con cualquier material 2D no lineal

4.3.6 Familia sólido 3D

Sólidos tridimensionales con convención Voigt 6D del proyecto (Reglas.md §5): $\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top$ con $\gamma_{ij} = 2\,\varepsilon_{ij}$ engineering (ADR 0012). A diferencia de la familia

2D, los sólidos 3D **no exponen** `internal_forces` — la primitiva de salida es `compute_gauss_state(U)`, que devuelve $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ por punto de integración (ADR 0012: cierre del contrato por dominio explícito).

Primer orden (Etapa 7, 2026-05-19)

Tabla 4.10: Cobertura de la familia sólido tridimensional — primer orden.

Topología	Material lineal	Material no lineal
Tetraedro lineal P_1 (4 nodos)	✓ `Tet4`	✓ `Tet4` con `VonMises3D`, `DruckerPrager3D` o `IsotropicDamage3D`
Hexaedro trilineal Q_1 (8 nodos)	✓ `Hex8`	✓ `Hex8` con cualquier material 3D no lineal

Segundo orden (sub-etapa A.ter, 2026-05-27)

Tabla 4.11: Cobertura de la familia sólido tridimensional — segundo orden.

Topología	Material lineal	Material no lineal
Tetraedro cuadrático P_2 (10 nodos)	✓ `Tet10`	✓ `Tet10` con cualquier material 3D no lineal
Hexaedro serendípito Q_2^{seren} (20 nodos)	✓ `Hex20`	✓ `Hex20` con cualquier material 3D no lineal
Hexaedro lagrangiano Q_2 (27 nodos)	✓ `Hex27`	✓ `Hex27` con cualquier material 3D no lineal

La sub-etapa A.ter introduce la base interna `_HigherOrderSolid3D` (paritaria con `_HigherOrderSolid2D` de la familia 2D) que comparte los bucles de Gauss para rigidez, fuerzas internas, gauss state, body load, face traction y matriz de masa. Las tres subclases (`Hex20`, `Hex27`, `Tet10`) sólo declaran sus funciones de forma 3D, su cuadratura por defecto, su cuadratura de masa específica (cuando difiere), las funciones de forma 2D de cara y la cuadratura de cara. La centralización aplica la regla de los dos casos reales: se introduce al entrar el `Hex27` (segundo caso cuadrático después del `Hex20` standalone) y se reutiliza inmediatamente por el `Tet10` con cara triangular `Tri6`. Cuadraturas nuevas registradas: `tet_4` (Stroud 4 puntos, orden 2, para K) y `tet_15` (Keast 15 puntos, orden 5, para masa del `Tet10`).

4.3.7 Familia térmica

Elementos de conducción de calor con un grado de libertad escalar `T` por nodo. Constituyen una familia **paralela** a las anteriores, no una extensión de ellas: el campo incógnita es escalar en lugar de vectorial, y la magnitud derivada es el gradiente ∇T, un vector genuino de dos o tres componentes.

Esa diferencia tiene una consecuencia que conviene subrayar, porque rompe una regularidad presente en el resto del programa: **el problema térmico no emplea notación de Voigt**. La notación de Voigt existe para comprimir un tensor *simétrico* de segundo orden en un vector, aprovechando que sus componentes cruzadas son iguales dos a dos; un gradiente escalar no es un tensor simétrico y no admite ni requiere esa compresión. La matriz $\mathbf{B}$ térmica es por tanto el gradiente crudo de las funciones de forma, $\partial N/\partial x$, sin reordenamiento de componentes ni los factores $\gamma_{ij} = 2\varepsilon_{ij}$ de la convención mecánica. En correspondencia, los elementos térmicos declaran el atributo `FLUX_DIM` (valor 2 o 3) en lugar de `STRAIN_DIM`.

Tabla 4.12: Cobertura de la familia térmica.

Topología	Elemento	FLUX_DIM	Cuadratura por defecto
Cuadrilátero bilineal Q_1 (4 nodos)	`Quad4Thermal`	2	2 x 2
Hexaedro trilineal Q_1 (8 nodos)	`Hex8Thermal`	3	2 x 2 x 2

Ambos comparten los núcleos de cálculo de funciones de forma y jacobiano con sus homólogos mecánicos `Quad4` y `Hex8`, y una base interna común `_ThermalSolid` que concentra el bucle de integración de la matriz de conductividad, la matriz de capacidad calorífica en sus dos formas, la fuente volumétrica y el post-proceso.

Esa base se introdujo con el **primer** elemento de la familia y no con el segundo, apartándose deliberadamente de la regla general del proyecto de no centralizar hasta disponer de dos casos reales. La justificación es que la ecuación discretizada $\mathbf{C}\,\dot{\mathbf{T}} + \mathbf{K}\,\mathbf{T} = \mathbf{F}$ es idéntica en dos y tres dimensiones —la dimensión sólo altera el tamaño de la matriz $\mathbf{B}$—, de modo que la abstracción no era especulativa sino la constatación de una identidad formal previa. La decisión quedó validada a posteriori: la incorporación del `Hex8Thermal` no requirió modificar la base.

La familia expone `compute_gauss_state(T)`, que devuelve el gradiente y el flujo por punto de integración, en correspondencia con el cierre de contrato por dominio que el ADR 0012 estableció para los sólidos. No expone `internal_forces`, que corresponde a los elementos estructurales unidimensionales.

Convención de signo del flujo de frontera (Reglas.md §5): un flujo prescrito positivo, $\bar{q} > 0$, representa flujo **saliente** del dominio, es decir enfriamiento. El signo procede de la forma débil, donde el término de frontera aparece como $-\int_\Gamma w\,\bar{q}\,d\Gamma$. Un borde o cara sobre el que no se declara condición alguna es **adiabático**, por analogía directa con el borde libre de tracción del problema mecánico.

4.3.8 Familias adicionales no implementadas

- *[Pendiente] Elementos de cáscara y lámina (formulación de Mindlin-Reissner para cáscaras gruesas; formulación de Kirchhoff-Love para cáscaras delgadas).*
- *[Pendiente] Elementos mixtos desplazamiento-presión (Q1-P0, Taylor-Hood) para problemas casi incompresibles.*
- *[Pendiente] Elementos de interfaz de espesor nulo entre superficies, para juntas y grietas cohesivas de trayectoria conocida y para contacto con fricción. La familia de leyes ya existe (materiales de interfaz, ADR 0033); hoy la consumen la interfaz de adherencia barra–sólido, que genera el refuerzo embebido, y la discontinuidad embebida del módulo de usuario.*

4.3.9 Contrato común y convención de signos

Todos los elementos tipo armadura, cable o marco implementan el contrato `internal_forces(U)` y devuelven las fuerzas internas (N, V, M, T) en la convención de signos del proyecto (ADR 0002), también con sección por fibras.

4.4 Restricciones generadas

Un generador de restricciones (ADR 0032; registro `ConstraintRegistry`, bloque YAML `constraints`, carpeta `solidum/bc/`) crea sobre un dominio ya mallado los nodos, los elementos y las restricciones multipunto que su geometría pide, antes de numerar las ecuaciones. Localiza los puntos dentro de los sólidos invirtiendo su mapeo isoparamétrico por Newton con lo que todo sólido declara (`REFERENCE_CELL`, funciones de forma), sin código por tipo de elemento.

Implementado: `EmbeddedReinforcement`, el refuerzo embebido. La barra, con su propia malla de elementos de dos o tres nodos (`Truss2D`/`Truss3D`, `Truss2D3`/`Truss3D3`) cortada en los bordes de los anfitriones, queda atada al campo interpolado del sólido por MPC (adherencia perfecta) o, con `bond`, unida a él por interfaces de adherencia-deslizamiento `BondSlipInterface` generadas por tramo, con una ley de interfaz de línea. Las interfaces generadas no se registran, no tienen masa ni celda VTK. `Domain.generated_nodes` aparta los nodos generados de los selectores por coordenada del YAML.

4.5 Solvers no lineales

El solver no lineal corresponde al nivel estratégico del cálculo: orquesta la subdivisión en pasos, las iteraciones internas y los criterios de convergencia. Es el componente que el usuario selecciona en el archivo YAML mediante el campo `solver.type`.

Disponibles en la versión actual:

- `LinearSolver` — un único paso, una única resolución del sistema `K · U = F`. Aplicable a problemas estrictamente lineales (rigidez constante, sin actualización de geometría, sin no linealidad material).
- `NonlinearSolver` — método de Newton-Raphson incremental con control de carga. Aplica fracciones crecientes de la carga total y, en cada paso, itera hasta la convergencia. Criterio de parada dual (desplazamiento y fuerza).
- `ArcLengthSolver` — método de Crisfield. Introduce una incógnita adicional (el factor de carga) y una restricción geométrica sobre la trayectoria en el espacio (U, λ), lo que permite recorrer ramas con derivada infinita o negativa. Imprescindible para problemas con *snap-back* o *snap-through*. Con `control` y `control_target` se detiene exactamente en el valor de una magnitud (ADR 0025).
- `IndirectDisplacementSolver` — variante del arco con la restricción del control indirecto por desplazamiento (de Borst 1987): el paso fija el incremento de una combinación de desplazamientos, como la abertura de una entalla; con `control_target` se detiene en su valor.
- `ModalSolver` (ADR 0009 fase 1) — análisis modal: problema generalizado de valores y vectores característicos $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. No es análisis dinámico propiamente dicho (no hay paso de tiempo ni evolución temporal); sus soluciones admiten interpretación como frecuencias propias y formas de vibración libre no amortiguada. Internamente delega en `EigenSolver` (ARPACK Lanczos con shift-invert) sobre el sistema reducido por Dirichlet. Devuelve `ModalResult` con frecuencias, períodos y modos M-ortonormales; expone además `free_vibration(M, u0, u0_dot, t)` para reconstruir analíticamente la respuesta libre por superposición modal.

- `NewmarkSolver` (ADR 0009 fase 3) — análisis dinámico transitorio lineal: integración temporal directa de $M \cdot \ddot{u} + C \cdot \dot{u} + K \cdot u = F(t)$ por el método de Newmark-β con (β, γ) parametrizables (default 1/4, 1/2 — *average acceleration*, incondicionalmente estable, error $O(\Delta t^2)$). Amortiguamiento Rayleigh $C = \alpha \cdot M + \beta \cdot K$ con coeficientes directos o calibrados modalmente a partir de dos pares (ξ, ω). Cargas dependientes del tiempo por callback Python. Devuelve `TransientResult` con historiales $(t, u, \dot{u}, \ddot{u})$. La matriz efectiva $A_{eff} = M + \gamma\Delta t \cdot C + \beta\Delta t^2 \cdot K$ se factoriza una sola vez al inicio y se reutiliza en todos los pasos.

- `NewtonNewmarkSolver` (ADR 0009 fase 4) — análisis dinámico transitorio no lineal. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores, correctores y reducción de Dirichlet; añade un bucle Newton-Raphson dentro de cada paso temporal sobre el residuo dinámico $R = F_{ext} - F_{int}(u) - C \cdot \dot{u} - M \cdot \ddot{u}$ con jacobiano $J = M + \gamma\Delta t \cdot C + \beta\Delta t^2 \cdot K_t$ (rigidez tangente corriente). Convergencia dual (ADR 0007). Amortiguamiento Rayleigh constante en el tiempo, calibrado con `K_0` (rigidez elástica de referencia; convención estándar Abaqus/ANSYS/OpenSees). Recupera el caso lineal a paridad de bits cuando los materiales no tienen historia activa. Newton modificado opcional (`freeze_tangent_after_iter`, ADR 0003 fase 2). Paso adaptativo: el paso cuyo Newton no converge se repite en dos mitades, hasta `max_subdivisions` niveles (6 por omisión), con el historial en la malla de tiempos del usuario.

- `HHTSolver` y `NewtonHHTSolver` (ADR 0009, variantes HHT-α) — disipación numérica controlada en altas frecuencias (Hilber-Hughes-Taylor 1977) con parámetro $\alpha \in [-1/3, 0]$ y (β, γ) auto-derivados. Subclases de `NewmarkSolver` y `NewtonNewmarkSolver` respectivamente (Reglas §4 — variantes).

- `CentralDifferenceSolver` (ADR 0009 fase 5) — integración explícita por diferencias centradas (leapfrog Belytschko-Liu-Moran). M^{-1} trivial sobre masa lumped diagonal. Lineal y no lineal en una sola clase con parámetro `nonlinear`. Estabilidad condicional CFL con detección a posteriori de divergencia.

- `HarmonicSolver` (ADR 0009 fase 6) — respuesta forzada armónica en el dominio de la frecuencia. Aritmética compleja con `scipy.sparse.linalg.spsolve` complejo; factorización LU compleja por frecuencia (no se cachea). Barrido lineal/logarítmico/explícito. Devuelve `HarmonicResult` con métodos `.amplitude()` y `.phase()`.

- `ResponseSpectrumSolver` (ADR 0009 fase 7) — análisis sísmico por combinación modal SRSS/CQC contra espectro de respuesta. Orquesta `ModalSolver` interno y delega los algoritmos en `solidum.math.modal_response` (centralización por regla D de auditoría). Devuelve `ResponseSpectrumResult` con respuesta envolvente, factores de participación y masas efectivas.

Los cinco solvers iterativos del programa principal (`NonlinearSolver`, `ArcLengthSolver` y su variante `IndirectDisplacementSolver`, `NewtonNewmarkSolver` y `NewtonHHTSolver`) no escriben su propio bucle de Newton: lo ejecuta el corrector compartido `NewtonCorrector` (`solidum/math/solvers/corrector.py`, ADR 0015), que posee el backend algebraico con su degradación a LU, el Newton modificado, el line search y el historial que alimenta las excepciones tipadas. Cada solver construye por paso un objeto `NewtonProblem` con lo que le es propio —cómo ensambla, cómo forma el residuo, cómo reduce y resuelve el sistema tangente, cómo aplica la corrección y qué comitea— y conserva su control de paso (bisección de la carga, longitud de arco y su restricción, predictores

de Newmark). Un solver iterativo nuevo aporta su problema y su control de paso y hereda todo lo demás.

Terminación, etapas y continuidad. Todos los solvers incrementales y los integradores en el tiempo informan cómo terminaron (`termination`, `completed`, último estado convergido) y fallan con una excepción tipada que conserva el resultado parcial (ADR 0025). Los estáticos incrementales arrancan de un estado y una carga constante (`solve(F, u0=…, constant_load=…)`, ADR 0026), los transitorios continúan un estático u otro transitorio (`continue_from`, ADR 0029), y el modal, el armónico, el espectral y el lineal pueden linealizar alrededor del estado de una etapa sin cambiarlo (ADR 0031). Desde Python se encadenan con `solidum.run(..., previous=resultado)`; desde el YAML, con el bloque `stages`.

Espacio natural de extensión:

- *[Pendiente] Newton-Raphson con búsqueda lineal (line-search) para mejorar la robustez ante no convergencia.* ***Parcialmente entregado por ADR 0011*** *— line search disponible como opt-in (`line_search=True`); default `False` por la justificación bibliográfica de Borst & Sluys 1999.*
- *[Pendiente] Solver cuasi-Newton tipo BFGS para reducir el coste de actualización de la matriz tangente.*
- *[Pendiente] Generalized-α, Bossak-α — esquemas adicionales de la familia con disipación numérica.*
- *[Pendiente] Solver de relajación dinámica para búsqueda de configuraciones de equilibrio en estructuras flexibles.*
- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, regla 100/30/30) y multi-support seismic excitation.*
- *[Pendiente] Control del Δt por estimación del error local en los transitorios. Hoy el paso sólo se subdivide cuando el Newton de Newton-Newmark o Newton-HHT no converge.*

La incorporación de un solver nuevo se realiza en `solidum/math/solvers/<nombre>.py` mediante el decorador `@SolverRegistry.register`. El **dispatch a entrypoints** en `solidum.entry.run_yaml` es declarativo (regla C de auditoría aplicada 2026-05-18): cada solver expone un atributo de clase `PIPELINE_KIND ∈ {"static", "modal", "transient", "harmonic", "spectrum"}` y `run_yaml` despacha por valor, sin tocarse cuando entran solvers nuevos no clásicos.

4.6 Subsistema algebraico

Por debajo del solver no lineal, dentro de cada iteración, se requiere la resolución de un sistema lineal `K · x = b`. Esta es la responsabilidad del subsistema algebraico (`solidum/math/linalg/`, ADR 0003). Constituye un nivel táctico: no decide la estrategia del cálculo, sino que selecciona el algoritmo numérico adecuado para resolver cada sistema lineal.

Algoritmos disponibles:

- `LUSolver` — factorización LU mediante SuperLU (SciPy). Algoritmo universal: aplicable a cualquier matriz no singular, simétrica o no, definida o no. Siempre disponible; es el suelo del despachador.

- `CholeskySolver` — factorización de Cholesky mediante CHOLMOD (dependencia opcional `scikit-sparse`). Aprovecha la simetría, pero es aplicable únicamente cuando `K` es simétrica y definida positiva.
- `PardisoSolver` (ADR 0017) — factorización directa **multihilo** con reordenamiento por disección anidada, mediante Intel MKL Pardiso (dependencia opcional `pypardiso`). Mismo dominio que LU, al que sustituye cuando está instalado. La factorización, y no el ensamblaje, domina el tiempo de un análisis grande: de $\times 5$ a $\times 56$ más rápido que SuperLU según la talla.
- `IterativeSolver` (ADR 0018) — gradiente conjugado (matrices simétricas definidas positivas) o MINRES (simétricas indefinidas), precondicionados con un multimalla algebraico que recibe los modos de cuerpo rígido del modelo. **Sólo a petición**: su ventaja es la memoria, proporcional al modelo, en problemas por encima de unos 10^5 grados de libertad.
- `EigenSolver` (ADR 0009) — algoritmo de autovalor generalizado simétrico para el problema $\mathtt{K} \cdot \varphi = \omega^2 \cdot \mathtt{M} \cdot \varphi$. Envuelve `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). Vive en `solidum/math/linalg/eigen.py` y no comparte la interfaz `solve(K, b)` de los anteriores — su firma natural es `solve(K, M, n_modes)` $\rightarrow$ `(`λ`, `φ`)`.
- *[Pendiente] `LDLTSolver` — factorización $LDL^\top$ con conteo de inercia para el caso simétrico indefinido (deuda técnica #7). Pardiso puede aportarla, pero exige el triángulo superior de la matriz.*
- *[Pendiente] solver iterativo para matrices no simétricas (GMRES o BiCGSTAB); hoy esas matrices van siempre a un solver directo.*

Despachador. La función `select_solver(props)` inspecciona las propiedades de la matriz (encapsuladas en `StiffnessProperties`: simetría, definición positiva) y selecciona el algoritmo de forma automática, en el orden Cholesky (si la matriz es simétrica definida positiva y CHOLMOD está instalado), Pardiso (si está instalado) y LU. El iterativo nunca se elige de forma automática. El usuario puede forzar un algoritmo desde el archivo YAML con el campo opcional `linear_algebra` (`auto`, `lu`, `cholesky`, `pardiso`, `iterative[:amg|jacobi|none]`) en calidad de herramienta de diagnóstico, no como decisión de modelado.

Red de seguridad. Un solver directo ante una matriz singular no avisa. Por eso el análisis estático comprueba antes de resolver que el modelo no es un mecanismo, y después, en el caso lineal, que no tiene un mecanismo interno (por la energía del modo de menor energía) y que el solver resolvió el sistema (error hacia atrás normado); en el no lineal, que la tangente inicial no tiene un mecanismo interno (ADR 0019; ver el capítulo de mecanismos transversales).

Justificación de la separación en dos niveles. El solver no lineal y el subsistema algebraico resuelven cuestiones distintas. El primero decide cómo recorrer la respuesta del sistema (incrementos, iteraciones, longitud de arco). El segundo decide con qué método resolver cada sistema lineal individual. Esta separación permite, por ejemplo, mejorar el rendimiento del paso elástico sin afectar al método de Newton-Raphson, o introducir longitud de arco sin reescribir el código del subsistema algebraico.

4.7 Tipos de análisis

El tipo de análisis en Solidum FEM corresponde al problema físico que afronta el solver, no al solver en sí.

Implementados en la versión actual:

- *Estática lineal*. Desplazamientos pequeños, rigidez constante, una resolución directa. Combinación: `LinearSolver` con materiales lineales y elementos en formulación lineal.
- *Estática no lineal incremental*. No linealidad geométrica (corrotacional), material (plasticidad, daño) o ambas. Combinación: `NonlinearSolver` o `ArcLengthSolver` con materiales no lineales y/o elementos corrotacionales.
- *Análisis modal* (ADR 0009 fase 1). Problema algebraico de valores y vectores característicos generalizado $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. Combinación: `ModalSolver` con materiales que declaren `density` y elementos que implementen `compute_mass_matrix`, o con masas concentradas en los nudos (ADR 0024); los grados de libertad sin masa se condensan de forma exacta.
- *Dinámica estructural transitoria lineal* (ADR 0009 fase 3). Integración temporal directa de la ecuación de movimiento por Newmark-β, con amortiguamiento Rayleigh proporcional y cargas dependientes del tiempo por callback Python. Combinación: `NewmarkSolver` con material lineal, elementos en formulación lineal y `density` declarada.
- *Dinámica estructural transitoria no lineal* (ADR 0009 fase 4). Integración Newmark con Newton-Raphson dentro de cada paso sobre el residuo dinámico $R = F_{ext} - F_{int}(u) - C\cdot\dot{u} - M\cdot\ddot{u}$. Combinación: `NewtonNewmarkSolver` con cualquier material con historia (`Elastoplastic1D`, `VonMises2D` en ambas hipótesis, `DruckerPrager2D`, `IsotropicDamage1D/2D`) y `density` declarada. Habilita sísmica con disipación plástica, impacto en hormigón friccional, fatiga de bajo ciclo y vibraciones de marcos elastoplásticos.
- *Dinámica estructural transitoria con disipación HHT-α* (ADR 0009, variante de fases 3 y 4). Mismas combinaciones que Newmark, con la familia HHT-α que controla la disipación numérica de modos altos espurios.
- *Dinámica estructural transitoria explícita* (ADR 0009 fase 5). Integración explícita por diferencias centradas. Combinación: `CentralDifferenceSolver` con masa lumped (`lumping="lumped"`) y la condición de estabilidad CFL $\Delta t < L_{el} \cdot \sqrt{(\rho/E)}$ respetada por el usuario. Apropiado para *wave propagation*, impacto y dinámica de respuesta rápida.
- *Análisis por etapas* (ADR 0026, 0029 y 0031). Una secuencia de análisis en la que cada uno parte del estado que dejó el anterior: gravedad y después empuje lateral, un transitorio tras un estático, o el modal y el espectral sobre una estructura cargada o plastificada. Combinación: cualquier solver estático incremental, transitorio o lineal con `previous=` o el bloque YAML `stages`.
- *Respuesta forzada armónica en el dominio de la frecuencia* (ADR 0009 fase 6). Resolución directa del sistema complejo $(-\omega^2 M + i\omega C + K)\cdot\hat{u} = F$ por barrido en ω. Combinación: `HarmonicSolver` con amortiguamiento Rayleigh y excitación armónica.

- *Análisis sísmico por combinación modal espectral* (ADR 0009 fase 7). Cálculo modal interno + factores de participación + combinación SRSS o CQC contra un espectro de respuesta tabulado o callable. Combinación: `ResponseSpectrumSolver` con materiales lineales, `density` declarada y un espectro normativo o equivalente.

- *Conducción de calor estacionaria.* Resolución de $\mathbf{K}\,\mathbf{T} = \mathbf{F}$ con la ley de Fourier $\mathbf{q} = -\mathbf{k} \cdot \nabla T$, condiciones de Dirichlet (temperatura impuesta) y de Neumann (flujo prescrito). Combinación: `LinearSolver` con `ThermalConduction` y elementos de la familia térmica. Merece señalarse que este régimen **no requirió un solver propio**: el solver lineal existente, concebido para el problema mecánico, resuelve el térmico sin modificación alguna. La razón es que ni el ensamblaje, ni la imposición de restricciones por eliminación (ADR 0004), ni el despacho algebraico (ADR 0003) hacen suposición alguna sobre el significado físico del grado de libertad; operan sobre un sistema algebraico y sobre las propiedades de su matriz. La incorporación del campo térmico constituyó, en ese sentido, una verificación empírica de esas decisiones frente a un campo físico que no existía cuando se tomaron.

- *Conducción de calor transitoria.* Integración temporal de $\mathbf{C}\,\dot{\mathbf{T}} + \mathbf{K}\,\mathbf{T} = \mathbf{F}(t)$ por el método θ. Combinación: `ThetaMethodSolver` con material que declare densidad y calor específico. Este esquema **sí** constituye una familia de solver propia y no una variante de la familia Newmark: la ecuación térmica es de **primer orden** en el tiempo, mientras la familia Newmark integra la ecuación de segundo orden mediante hipótesis sobre la aceleración dentro del paso. En conducción no existe segunda derivada temporal sobre la que aplicar tales hipótesis, y forzar el esquema anulando la matriz de masa lo degenera. En correspondencia, el solver no admite amortiguamiento de Rayleigh —modelo de disipación propio de la ecuación de segundo orden, mientras que aquí la disipación es la conducción misma, ya contenida en $\mathbf{K}$— ni condición inicial de velocidad, y devuelve un tipo de resultado específico, `ThermalTransientResult`.

Ambos regímenes térmicos operan sobre el campo de temperatura de forma independiente del campo mecánico: no existe deformación térmica y los dos análisis no se comunican.

Previstos y no implementados (dirección del proyecto, sin compromiso de calendario):

- *[Pendiente] Conducción no lineal — conductividad dependiente de la temperatura, radiación y cambio de fase; los tres exigen iteraciones de Newton dentro de cada paso, que el esquema actual evita por hipótesis de linealidad.*

- *[Pendiente] Condición de frontera de convección (Robin), q = h (T - T_inf); añade un término a la matriz de conductividad y otro al vector de cargas, sin alterar la estructura del solver.*

- *[Pendiente] Acoplamiento termo-mecánico desacoplado (staggered) para casos en que la influencia mecánica sobre el campo térmico sea despreciable.*

- *[Pendiente] Acoplamiento termo-mecánico monolítico para problemas con fuerte interacción bidireccional.*

- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, 100/30/30) y multi-support para diferencias de movimiento entre apoyos.*

La estructura de capas y registros se ha diseñado para incorporar dichos análisis sin reformular las piezas existentes; el ADR correspondiente se redactará al inicio de cada fase de diseño.

Capítulo 5

Mecanismos transversales

Los patrones que se describen a continuación estructuran el código de Solidum FEM sin pertenecer a una capa concreta. Aparecen indistintamente en materiales, elementos y solvers; su comprensión es necesaria para la lectura de cualquier zona del programa.

5.1 Registros con auto-registro

Cada categoría de componente dispone de su propio registro: `MaterialRegistry`, `ThermalMaterialRegistry`, `InterfaceMaterialRegistry`, `SectionRegistry`, `ElementRegistry`, `SolverRegistry`, `ConstraintRegistry`, todos subclases de la clase genérica `Registry` (`solidum/registry.py`). Cada clase concreta se inscribe en su registro mediante un decorador (`@MaterialRegistry.register` y análogos) que se ejecuta en el momento de definición de la clase. El intérprete del archivo YAML no requiere conocimiento *a priori* de los materiales o elementos existentes: consulta el registro por el nombre que aparece en el archivo. La incorporación de un componente nuevo no exige la modificación de listas centrales.

5.2 Familias declaradas por el registro

Un registro declara sus metadatos con tres atributos públicos de clase (ADR 0020), que son todo lo que necesita un módulo de usuario para definir el suyo: `YAML_LABEL`, el nombre de un objeto del registro en los mensajes de error, tanto del lector YAML como del propio registro (`"elemento"`, `"solver"`, `"material térmico"`, `"material de interfaz"`, `"sección"`, `"restricción"`); todo registro lo tiene, sea o no una familia, y sustituye al antiguo atributo privado `_kind`, que ya no existe. `SPEC_KIND` es el `kind` de las specs cuyos componentes viven en él. `YAML_SECTION`, sólo en las familias, es la sección de primer nivel del YAML que contiene sus objetos (`materials`, `thermal_materials`, `interface_materials`, `sections`). Un registro con `YAML_SECTION` es una **familia**: un conjunto de objetos que se declaran una vez en el YAML, con `id` y `type`, y que los elementos referencian por `id`. `Registry.families()` las enumera en su orden de definición. Las dos primeras familias fueron de materiales; la tercera, `SectionRegistry` (ADR 0021), agrupa las **secciones de marco**, leyes constitutivas a nivel de sección transversal con un contrato propio (reciben $[\varepsilon_0, \kappa]$ y devuelven fuerzas internas), que no son materiales. La cuarta, `InterfaceMaterialRegistry` (ADR 0033), definida antes que las secciones, agrupa las leyes tracción–desplazamiento relativo de las interfaces

(grietas y barras embebidas). `ConstraintRegistry` (ADR 0032) no es una familia: sus generadores se declaran en el bloque `constraints`, con `type`, y se aplican sobre el dominio ya mallado; ningún otro objeto los referencia por `id`. El intérprete del YAML recorre esa lista en un solo bucle, sin conocer ninguna familia por nombre; la herramienta de specs (`solidum/tools/spec.py`) localiza el registro de cada spec por su `SPEC_KIND`. Una familia nueva, del programa principal o de un módulo de usuario, se declara con una subclase de `Registry` y no toca el intérprete. Cada subclase recibe su propio diccionario de clases (heredarlo compartiría en silencio el de la base) y declarar una sección o un tipo de spec que otro registro ya declaró es un error.

Los elementos declaran a qué familia pertenecen sus materiales con `Element.REFERENCE_KWARGS`: qué parámetros del constructor son, en el YAML, el `id` de un objeto de otra familia, y de cuál. El valor por defecto es `{"material": MaterialRegistry}`; los elementos térmicos declaran `{"material": ThermalMaterialRegistry}`, y un elemento con dos materiales declara las dos referencias. Los marcos Euler-Bernoulli declaran `{"material": MaterialRegistry, "section": SectionRegistry}`, las dos opcionales: es la primera referencia del programa principal con nombre distinto de `material`, y el constructor exige exactamente una de las dos. El elemento con discontinuidad embebida del módulo `discontinuities` declara además `cohesive_material` contra `InterfaceMaterialRegistry`, y comprueba al construirse que la ley admite su geometría (`accepts("surface", 2)`). El intérprete valida que cada `id` exista en la sección de su familia y pasa al constructor el objeto ya construido, sin heurística sobre el tipo de elemento.

5.2.1 Referencias anidadas entre familias: el gancho `from_yaml`

Un objeto de familia puede referenciar a su vez objetos de otra familia: una sección por fibras lleva, dentro de cada parche, el `id` del material de sus fibras. Los objetos de familia se construyen normalmente con `cls(**datos)`; para las referencias anidadas, una clase de familia puede declarar el método de clase `from_yaml(data, resolve)` (ADR 0021 §2). Si existe, el intérprete lo llama con los datos de la entrada (sin `id` ni `type`) y con una función `resolve(registro, id, contexto)` que devuelve el objeto de `id` dado de una familia **ya construida**, o lanza un error que nombra la entrada que lo pide y el contexto dentro de ella (`sections[0] (id=7), patches[0].material: referencia a material inexistente (9)`). Las familias se construyen en su orden de definición, así que una familia sólo puede referenciar a las definidas antes (`SectionRegistry` se define después de `MaterialRegistry`); una referencia a una familia todavía no construida también es un error explícito. Los errores de todas las entradas se acumulan y se lanzan juntos. El intérprete no conoce las secciones por nombre: cualquier familia futura con referencias anidadas (una sección de lámina que referencie materiales por capa, por ejemplo) usa el mismo gancho.

5.3 Descubrimiento automático

El módulo `solidum/autodiscover.py` se invoca una sola vez durante la importación del paquete. Recorre de forma recursiva, mediante `pkgutil.walk_packages`, las carpetas canónicas (`solidum/materials`, `solidum/sections`, `solidum/elements`, `solidum/math`, `solidum/bc`) e importa cada módulo. Como los decoradores `@register` se ejecutan en el momento de la importación, esta operación basta para poblar todos los registros sin enumeración manual. Este mecanismo es el equivalente conceptual al `INCLUDE` automático de Fortran moderno, ejecutado a tiempo de ejecución. La carpeta `solidum/user/` queda fuera a propósito: un módulo de usuario sólo se carga cuando un

modelo lo pide.

5.4 Programa principal y módulos de usuario

Solidum distingue dos partes, con los términos de FEAP (ADR 0020). El **programa principal** es Solidum estándar: los elementos, materiales y solvers de elementos finitos clásicos, es decir, todo `solidum/` salvo `solidum/user/`. Un **módulo de usuario** es una formulación no estándar en su propia carpeta, `solidum/user/<nombre>/`, análoga a los elementos (`elmtNN`), materiales (`umati`) y macros (`umacr`) de usuario de FEAP. El primero es `discontinuities`, la discontinuidad interior embebida del ADR 0010: el elemento `CST_Embedded2D`, el estado de la discontinuidad y el seguimiento de grieta; su ley cohesiva, que nació en él, pasó al programa principal con la familia de materiales de interfaz (ADR 0033) cuando el programa principal tuvo un elemento de interfaz propio. El segundo, `dissipation_control`, es un solver: el arco por disipación de energía de Gutiérrez (2004), `DissipationArcLengthSolver`, que salió del programa principal el 2026-09-29 porque ningún código general establecido lo implementa de forma nativa (criterio de admisión, Reglas.md §1).

El programa principal no conoce los módulos de usuario hasta que un modelo los pide: `import solidum` no los importa. La carga es explícita, con la clave `user_modules: [<nombre>]` del YAML, procesada al principio de la lectura y antes de validar, o con `solidum.load_user_module("<nombre>")` en Python; `solidum.available_user_modules()` lista los instalados. Cargar un módulo importa sus submódulos con el mismo recorrido del descubrimiento automático, de modo que sus decoradores inscriben elementos, materiales, solvers y familias en los registros del programa principal.

Un módulo de usuario se conecta sólo a través de piezas genéricas del programa principal, cada una justificada por casos reales ajenos a él:

- los registros con metadatos de familia, `Registry.families()`, las referencias `REFERENCE_KWARGS` de los elementos y el gancho `from_yaml(data, resolve)` para objetos de familia con referencias anidadas (ver «Familias declaradas por el registro»);
- el gancho de inicio de paso `Element.prepare_step(U_committed)`, no-op por defecto, que los solvers que avanzan por pasos invocan con el campo convergido del paso anterior; debe ser idempotente, porque tras una bisección se repite;
- las funciones de cinemática públicas (`compute_kinematics_tri3`, `compute_integrands`), las mismas que usan `Tri3` y `Quad4`, que un elemento de usuario importa de su ruta pública, `solidum.elements.solid_2d`, y no del módulo interno `_shared`;
- el problema de Newton por paso del arc-length, exportado con nombre público como `solidum.math.solvers.ArcProblem`: una variante de arco que sólo cambia la restricción del paso hereda de `ArcLengthSolver` y de ese problema y redefine su método `constraint`, como hacen `IndirectDisplacementSolver` en el programa principal y `DissipationArcLengthSolver` en el módulo `dissipation_control`;
- la receta de elemento por grupo físico de gmsh (`mesh_element`, y `element` dentro de `mesh_physical_groups`);

- el contrato del elemento con estado propio, fuera de `ElementState`: `element_is_batchable` lo excluye del camino por lotes si la clase redefine `compute_element_state` o `commit_state` por debajo de la que declaró `BATCH_KINEMATICS`, para que el kernel de familia no calcule ni consolide en silencio sin sus redefiniciones.

Un módulo de usuario sólo **añade** entradas a los registros; nunca sustituye una existente. `load_user_module` es atómica: toma una instantánea de los registros antes de cargar y, si el módulo reasigna una entrada ya registrada (por ejemplo, inscribe otra clase como `Elastic2D`), lanza `ValueError` y deshace la carga, registros y `sys.modules` incluidos; lo mismo si la carga falla por un error del propio módulo. Así un módulo no puede cambiar en silencio el comportamiento del programa principal para los modelos posteriores de la sesión.

La separación la vigila `tests/test_main_program_purity.py` con tres controles:

1. **Estructural**, sobre el árbol sintáctico (`ast`) de cada archivo del programa principal: ninguno importa un subpaquete de `solidum.user` de ninguna forma, sea absoluta, relativa, perezosa (dentro de una función), por cadena (`importlib.import_module`, `__import__`, `sys.modules[...]`) o con `load_user_module` y un nombre literal. Lo único que el programa principal importa de `solidum.user` es el cargador genérico, `load_user_module` y `available_user_modules`, que construye la ruta a partir del nombre que le pasa el modelo.

2. **Léxico**: ningún archivo del programa principal menciona lo que aporta un módulo de usuario. Los nombres vigilados no se mantienen a mano: el test los deriva cargando cada módulo en un proceso limpio (las entradas que añade a los registros, las familias que declara con su sección YAML y su `SPEC_KIND`, las referencias nuevas de sus elementos, su `__all__` y su ruta de importación). `NOMBRES_ADICIONALES` es un complemento opcional para las constantes de nivel de módulo que la derivación no ve.

3. **En ejecución**: en un proceso limpio, `import solidum` no carga ningún `solidum.user.*` y toda clase registrada pertenece al programa principal.

Un módulo nuevo no se da de alta en el test, y todas las pruebas pasan también si no hay ninguno. En el sentido inverso, `tests/test_user_module_public_imports.py` comprueba que los módulos de usuario (y sus tests) sólo importan nombres públicos del programa principal: ninguna ruta con un componente privado (`solidum.elements.solid_2d._shared`) ni un nombre con guion bajo, y sus subclases de `Registry` sólo declaran los atributos públicos (`SPEC_KIND`, `YAML_SECTION`, `YAML_LABEL`), sin asignar atributos `_*`. Las specs de un módulo viven en `docs/user/<nombre>/specs/` y sus pruebas en `tests/user/<nombre>/`.

5.5 Contratos declarativos mediante atributos de clase

En lugar de imponer métodos abstractos cuando el comportamiento puede inferirse del dato, Solidum FEM declara los contratos como atributos de clase: `STRAIN_DIM` (1, 3 o 6), `DOF_NAMES` (lista de nombres de DOF por nodo), `N_INTEGRATION_POINTS`, `PRIMARY_STATE_VAR`, `IS_LINEAR_ELASTIC`. La clase base los lee y se autoconfigura: registra los DOF, valida la compatibilidad material↔elemento e inicializa el `ElementState` con la forma correcta. Este mecanismo sustituye los métodos `setup()` repetitivos que cada subclase tendría que implementar.

El mismo criterio rige los contratos que llegaron después: `REFERENCE_CELL` y `REFERENCE_NODES` (celda y nodos de referencia del elemento, ADR 0022), que usan el generador de mallas estructuradas, la verificación por soluciones manufacturadas, la localización de puntos y el refuerzo embebido para saber cuántos nodos tiene una barra; `GEOMETRICALLY_NONLINEAR` (ADR 0019); `GEOMETRY` y `JUMP_DIM` de los materiales de interfaz (ADR 0033: superficie o línea, y número de componentes admitidas, `None` si las de 2D y 3D), que el consumidor comprueba con `accepts(geometría, n)`; `VTK_EXPORT = False` en un elemento que el exportador debe omitir sin aviso (las interfaces de adherencia generadas, que no tienen celda VTK); y `YAML_KEYS` en una clase de familia, las claves adicionales que su `from_yaml` traduce y que el lector no debe rechazar como desconocidas.

Un atributo de clase puede sobrescribirse por instancia cuando el dato depende de la construcción, no de la clase: un marco con sección fija `N_INTEGRATION_POINTS = n_ip` en la instancia antes de construir su `ElementState`, mientras la clase conserva el valor de su camino por defecto (ADR 0021). `Material.IS_LINEAR_ELASTIC` (por defecto `False`; `True` en `Elastic1D`, `Elastic2D`, `Elastic3D` y `Orthotropic2D`) declara que la respuesta del material es lineal y sin historia; el barrido de contrato de materiales lo recorre, y los marcos lo exigen en su camino de forma cerrada.

Todos los materiales de un elemento: `Element.materials()`. Las comprobaciones agregadas —la densidad que exigen la matriz de masa y el peso propio (ADR 0008) y la simetría del sistema que decide el backend algebraico (ADR 0003)— recorren `element.materials()` en lugar de leer `element.material` (ADR 0021 §4). El valor por defecto es `[self.material]`; un elemento con varios materiales lo sobrescribe: un marco con sección devuelve los materiales distintos de sus fibras, de modo que una fibra sin `density` o con tangente no simétrica se detecta igual que el material de un sólido.

Longitud característica del punto de integración. Un material con ablandamiento regularizado por energía de fractura (banda de grieta) necesita la longitud h de cada punto: lo declara con `Material.uses_characteristic_length` y recibe h en su estado inicial por `Material.initial_point_state(h)`, que puede rechazarla. El elemento la siembra al construirse de una de dos formas declarativas. Las barras y los marcos (`SEEDS_CHARACTERISTIC_LENGTH = True`) la calculan ellos mismos y llaman a `_seed_characteristic_lengths` (la longitud, o el peso del punto por la longitud). Los sólidos 2D y 3D declaran `CHARACTERISTIC_LENGTH_FACTOR` $= c$ y la base la calcula sin código por tipo de elemento: $h = c\,\Omega_e^{1/d}$, con $\Omega_e = \sum_g w_g \det J_g$ evaluado con la propia cinemática compilada del elemento (`BATCH_KINEMATICS`); c es 1 en `Quad4` y `Hex8`, $\sqrt{2}$ en `Tri3`, $6^{1/3}$ en `Tet4` y ½ en los cuadráticos (2026-11-11, `IsotropicDamageCrackBand2D/3D`). Un elemento que no declara ninguna de las dos rechaza al construirse un material que necesita h. El valor viaja en el estado del punto, así que el camino por lotes lo lleva en su fila sin código propio.

5.6 Validación temprana en construcción

Durante la instanciación de un elemento, su método `__init__` verifica que el material recibido sea dimensionalmente compatible. Un elemento `Truss2D` con material `Elastic2D` produce un error en la construcción del caso, no después de doscientas iteraciones en forma de fallo opaco durante la solución. La misma validación cubre la semántica, no sólo la dimensión: un marco con `material` + propiedades geométricas rechaza un material con `IS_LINEAR_ELASTIC = False`, porque su forma cerrada daría flectores incorrectos en silencio, y el mensaje remite a la sección por fibras; una `FiberSection` rechaza un material de fibra con `STRAIN_DIM` $\neq$ `1`, y `Frame3D` una sección sin `GJ`. El coste es una comprobación trivial; el beneficio consiste en que ciertos errores físicos se detectan de

forma inmediata en el momento de su introducción.

5.7 Intérprete genérico por introspección de argumentos

El componente `YamlParser` no contiene ramificaciones del tipo `if material_type == "Elastic1D"` por cada material. Inspecciona los argumentos del constructor de la clase recuperada del registro y le transmite los campos del archivo YAML. La incorporación de un material nuevo no requiere modificación del intérprete; es suficiente con que el constructor declare los parámetros con nombres compatibles con el archivo YAML. Un objeto de familia con estructura anidada se construye con su propio `from_yaml(data, resolve)` (ver «Referencias anidadas entre familias»), que aplica el mismo criterio: la `FiberSection` elige la primitiva por la clave `shape` y valida sus claves contra los parámetros del constructor de esa primitiva. Las secciones de primer nivel admitidas son las propias del intérprete más las que declaran las familias registradas; cualquier otra es un error, no se ignora.

5.8 Estados *trial* y comprometido en `ElementState`

Cada elemento mantiene un objeto `ElementState` con dos copias de las variables internas: una copia *trial* (la que se explora durante las iteraciones del solver) y una copia comprometida (la que corresponde al último paso convergido). El solver invoca `commit_state()` solo cuando un paso converge, lo que evita la contaminación del historial plástico o de daño con tentativas posteriormente descartadas. Esta semántica es crítica para problemas con plasticidad o daño: en su ausencia, una iteración no convergida que casualmente quedase dentro de tolerancia contaminaría el historial de forma irreversible.

5.9 Ensamblaje disperso con topología cacheada

El primer ensamblaje de la matriz global calcula los pares de índices (i, j) de cada contribución elemental en formato Coordinate (COO), deriva de ellos la estructura comprimida por filas (CSR) de la matriz y un mapa que lleva cada entrada COO a su posición CSR. En las iteraciones siguientes se reescribe únicamente el vector de datos sobre la misma topología (las matrices elementales de cada familia se escriben directamente en su bloque) y se reduce a CSR con un kernel compilado sobre el mapa inverso —las entradas COO que alimentan cada posición CSR, en orden estable—, serie o paralelo, sin recalcular índices ni reordenar: la conversión COO a CSR que `scipy` rehacía en cada ensamblaje (unos 115 ms para 8 000 hexaedros) desaparece. Se trata del patrón "calcular una vez, reutilizar" típico en programas de MEF compilados, implementado aquí sobre `scipy.sparse` (ADR 0014).

5.10 Ensamblaje por lotes: familias y un único kernel compilado

Los elementos que comparten clase, instancia de material, regla de cuadratura y número de nodos forman una *familia de lote* (`solidum/math/batch/`). La clave se deriva de atributos que ya existen; un elemento nuevo cae en su propia familia sin tocar el ensamblador. Cada familia guarda su estado interno como arreglos (`FamilyState`: variables internas y esfuerzos, committed y trial, una fila por punto de Gauss) y se evalúa completa dentro de un único bucle compilado, `solid_family_`

`kernel`, que recibe la cinemática del elemento (`BATCH_KINEMATICS`) y la constitutiva del material (`BATCH_KERNEL`) como funciones tipadas por su firma. Esa tipificación (`numba.types.FunctionType`) es lo que permite compilar el kernel una sola vez para todas las parejas elemento × material y reutilizarlo desde el caché en disco: un despachador tiene identidad propia por proceso y obligaría a recompilar por pareja en cada ejecución.

El camino por elemento (`compute_element_state`, `compute_state`) sigue siendo el contrato obligatorio y la referencia física. El camino por lotes es una capacidad que el componente declara: los elementos sin cinemática compilada (estructurales 1D), los elementos con estado propio (como el de discontinuidad embebida del módulo de usuario `discontinuities`) y los materiales sin esquema de estado se ensamblan como siempre dentro del mismo `Assembler`. Ambos caminos ejecutan las mismas funciones compiladas por punto de Gauss; el barrido de contratos exige que `K` y `F_int` coincidan a precisión de máquina en toda combinación registrada. El kernel recibe siempre coordenadas de referencia y desplazamientos y recomputa jacobiano y `B` en cada evaluación: nada derivado de la configuración deformada se cachea, de modo que el diseño vale para elementos lineales, corotacionales y formulaciones lagrangianas futuras. Medido sobre 10 000 Quad4 con plasticidad J2: 157 → 5,7 µs por elemento y ensamblaje con el kernel serie, y 1,4 µs con el paralelo (ADR 0014).

El kernel de familia existe en dos variantes con la misma aritmética: serie y paralela (`prange` sobre bloques de elementos, con espacio de trabajo por bloque y sin estado compartido entre hilos). Como cada elemento suma en el mismo orden, el resultado es bit a bit el mismo con cualquier número de hilos, cosa que el barrido de contratos exige. Una limitación de Numba condiciona el contrato de la cinemática: una excepción lanzada dentro de una región paralela se pierde en silencio, así que las cinemáticas por lotes no lanzan sino que devuelven un jacobiano no positivo, el kernel marca el punto y el ensamblador convierte la marca en la excepción de siempre, ahora con el identificador del elemento. El mismo protocolo rige la variante serie.

El post-proceso sigue el mismo camino. Cada familia deja en sus elementos una referencia a sí misma al adoptar sus estados, de modo que el exportador VTK y `gauss_states(domain, U)`, que sólo ven el dominio, evalúan por familia —un kernel que devuelve deformación y esfuerzo por punto de Gauss desde el estado committed, sin tocar el trial— y caen al `compute_gauss_state` de cada elemento sólo en los que no pertenecen a ninguna familia. La rigidez inicial (`assemble_system`, `compute_global_stiffness`) es una evaluación auxiliar y restaura el estado trial previo al salir.

5.11 Eliminación directa de condiciones de frontera

Las condiciones de frontera, tanto Dirichlet nodales como restricciones multipunto lineales (MPC), se imponen por eliminación directa (ADR 0004). Toda restricción se expresa en forma afín `u_s = g_s + Σ α_si · u_mi` y se acumula en un `ConstraintSet` (subpaquete `solidum.bc`). El ensamblador construye un operador disperso `T` y un vector `g` tales que `u = T · u_libre + g`: en filas correspondientes a DOF libres, `T` es la matriz identidad rectangular; en filas esclavas, lleva los coeficientes α`_si` en las columnas de los maestros. El sistema entregado al subsistema algebraico es `K_red` $= T^\top K T$, `F_red` $= T^\top$`(F − K · g)`. La imposición es exacta a redondeo, preserva la simetría y la positividad definida de `K`, y deja el sistema reducido apto para solvers iterativos (CG, GMRES).

La forma afín cubre con la misma maquinaria los casos de empotramiento, asentamiento prescrito, apoyo en plano oblicuo, periodicidad de celda unitaria y unión rígida master-slave entre nodos. Las restricciones declaradas como cadenas (un esclavo cuyo maestro es a su vez esclavo de otra restricción) se resuelven por cierre transitivo en el momento de construir `T`; los ciclos y las redeclaraciones

inconsistentes se detectan en validación temprana. La declaración se hace mediante `Domain.add_linear_constraint` o desde el bloque `linear_constraints` del archivo YAML.

El cierre parte siempre de las restricciones tal como se declararon, que el `ConstraintSet` conserva, y ofrece dos inspecciones independientes de él: `dirichlet_dofs`, los grados de libertad con valor prescrito declarado (sin los esclavos de las MPC, aunque el cierre deje sin maestros a un esclavo atado a un apoyo), y `build_g`, el vector `g` para unos valores de Dirichlet dados con la composición del cierre (`g_s = g_mpc,s + Σ α·ḡ_d`), que usa el transitorio térmico con temperatura impuesta variable en el tiempo. Los problemas de Newton de los solvers iterativos miden la convergencia con el equilibrio reducido $\|\mathtt{T}^\top \mathtt{R}\|$: con una MPC, el residuo no se anula en los grados de libertad libres por separado, porque la fuerza de cada esclavo pasa a sus maestros. En el post-proceso, `ConstraintSet.multipliers` recupera los multiplicadores de las restricciones declaradas sin cerrar ($\mathtt{C}^\top\mu$ `= F_int − F_applied`, por sustitución en orden de dependencia), y `build_solve_result` reparte su fuerza entre las reacciones de los apoyos (Dirichlet y MPC que impiden un movimiento de sólido rígido, clasificadas con la base rígida del dominio) y las fuerzas de restricción de las uniones (`SolveResult.constraint_forces`), con la convención que decidió el usuario (ADR 0004 §3).

5.12 Restricciones generadas y localización de puntos

Algunas restricciones no se declaran nudo a nudo sino que las pide una geometría: una barra de refuerzo que atraviesa una malla de sólidos sin coincidir con sus nodos. Un **generador de restricciones** (ADR 0032; `ConstraintRegistry`, carpeta `solidum/bc/`, bloque YAML `constraints`) recibe el dominio ya mallado y crea, antes de numerar las ecuaciones, los nodos, los elementos y las MPC que su geometría pide; el resto del programa los trata como si se hubieran declarado. Para atar un punto al campo de un sólido hay que saber en qué elemento cae y con qué coordenadas naturales: `solidum/mesh/point_location.py` lo resuelve invirtiendo el mapeo isoparamétrico por Newton (`natural_coordinates`) con lo que todo sólido declara (`REFERENCE_CELL` y sus funciones de forma), sin código por tipo de elemento, y `PointLocator` filtra los candidatos por su caja envolvente. Los nodos generados se registran en `Domain.generated_nodes`, que los aparta de los selectores por coordenada del YAML. Un generador puede crear también elementos no registrados cuando nadie los cita por `type` (las interfaces de adherencia del refuerzo embebido, ADR 0032 ampliación 2).

5.13 Etapas y continuidad de los análisis

Un análisis puede partir del estado que dejó otro (ADR 0026, 0029 y 0031). El estado son los desplazamientos (o temperaturas) y lo que guardan los elementos (variables internas, estado propio), más la carga que ya actúa. Los estáticos incrementales lo reciben con `solve(F_ref, u0=…, constant_load=…)`: el residuo es $\mathbf{F}_c + \lambda\mathbf{F}_{\text{ref}} - \mathbf{F}_{\text{int}}$ con $\lambda = 0$ al empezar, los valores impuestos crecen desde los de partida y se comprueba el equilibrio inicial. Los integradores en el tiempo lo reciben con `continue_from(previous)` (mezcla `TransientTerminationMixin`), desde un estático en reposo o desde un transitorio del mismo tipo con su aceleración final. El modal, el armónico, el espectral y el lineal linealizan alrededor del estado con la rigidez tangente de los elementos sin cambiarlo (mezcla `LinearizedStateMixin`). Desde Python las etapas se encadenan con `solidum.run(..., previous=resultado)`, que pasa como constante la carga de la etapa anterior y sus cargas equivalentes; desde el YAML, con el bloque `stages`, que devuelve un `StagedResult`.

5.14 Terminación y política del registro

Todo solver incremental y todo integrador en el tiempo deja cómo terminó en atributos que no exigen leer el registro (ADR 0025): `termination` (`max_lambda`, `control_target`, `max_steps`, `t_end` o `diverged`), `completed`, el número de pasos y el último estado convergido; se fijan también tras una excepción y se reinician en cada `solve`. Un fracaso llega como excepción tipada (`SolverDivergedError`, `StepConstraintError`, `TimeIntegrationInstabilityError`, …) con el resultado parcial. El registro (`solidum.logging`) sigue una política de niveles única: una línea `INFO` por paso, las iteraciones en `DEBUG`, lo que el solver recupera (una bisección, una subdivisión del paso) en `WARNING` con su motivo, y `ERROR` sólo al detenerse. El manejador de consola adapta los mensajes a la codificación del flujo (translitera lo que no cabe, por ejemplo en una consola cp1252), y el nivel se fija con `solidum.logging.set_log_level` o con `output.log_level` en el YAML.

5.15 Cargas que llevan sus cargas equivalentes

Las fuerzas internas de un elemento con carga distribuida son $\mathbf{F}_{\text{int}} - \mathbf{f}_{\text{eq}}$, así que el post-proceso necesita las cargas equivalentes de **la carga que actúa**, no de la última ensamblada. La carga las lleva consigo (ADR 0027): `LoadVector` (`solidum/core/load_vector.py`) es un `np.ndarray` con las cargas equivalentes por elemento de su parte distribuida, y la aritmética de las cargas las combina linealmente (`F_a ± F_b`, `a·F`, `F/a`); cualquier otra operación devuelve un arreglo simple. Así una corrida con varias cargas de cuerpo, varios casos con el mismo ensamblador, las etapas y los historiales de los transitorios dan fuerzas internas correctas sin cambiar la API de las cargas.

5.16 Longitud característica del punto de integración

Un material con ablandamiento regularizado por energía de fractura necesita la longitud h de cada punto de integración, que sólo conoce el elemento (ADR 0028). La longitud viaja en el estado del punto: el material declara si la usa (la propiedad `uses_characteristic_length`, que depende de la instancia) y construye el estado inicial con `initial_point_state(h)`, donde la valida; el elemento, al terminar de construirse, siembra una longitud por punto con `_seed_characteristic_lengths` (`L` en las barras de dos nodos; `w_g·L` por punto de Gauss, con los pesos normalizados a la unidad, en las barras de tres nodos y en los marcos con sección, que la entregan a sus fibras). Un h inadmisible se rechaza al construir el elemento, no durante el cálculo. Primer consumidor: `ManderConcrete1D`.

5.17 Compilación Just-In-Time mediante Numba

Las funciones críticas (cinemática de los elementos, núcleos de los algoritmos de retorno, el kernel de familia del ensamblaje por lotes) se decoran con `@njit(cache=True)` y se compilan a código nativo en su primera invocación; el binario se guarda en `__pycache__` y las ejecuciones siguientes lo cargan en centésimas de segundo. Lo que se compila es el procedimiento, no los datos: la configuración deformada, los parámetros y las variables internas entran como argumentos en cada llamada, así que un cambio de geometría o de estado nunca exige recompilar. La restricción consiste en que el código compilado solo admite tipos primitivos y arreglos NumPy, no objetos arbitrarios de Python; por eso los materiales exponen su estado como filas de arreglo (`STATE_SCHEMA`) y sus constantes como un vector de parámetros (`batch_params`).

5.18 Variable principal de visualización

Cada material declara como atributo de clase la variable interna principal a efectos de visualización: por ejemplo `'damage'` en un modelo de daño o `'alpha'` en plasticidad con endurecimiento. El componente `VtkExporter` la lee de forma genérica sin conocimiento del material que la origina. Este mecanismo permite la incorporación de materiales nuevos con visualización automática, sin modificación del exportador.

5.19 Cuadraturas centralizadas

Las tablas y reglas de cuadratura de Gauss-Legendre 1D, 2D y 3D residen centralizadas en `solidum/math/integration.py`. Cada elemento declara su `N_INTEGRATION_POINTS` y consume los puntos y pesos correspondientes, sin reproducción de tablas en cada subclase.

5.20 Política unificada de tolerancias: patrón `atol + rtol · escala`

Toda comparación con significado físico o iterativo del proyecto — admisibilidad constitutiva, convergencia de Newton-Raphson, convergencia de arc-length, futuros criterios de cierre de gap en contacto o de inversión del jacobiano — sigue una misma fórmula estructural:

```
magnitud ≤   atol  +  rtol · escala(estado)
```

donde `atol` es un piso absoluto en las unidades físicas del problema, `rtol` es una banda relativa adimensional y `escala(estado)` es la magnitud característica del criterio en el estado corriente. Esta forma única garantiza tres propiedades simultáneamente: invariancia bajo cambio de unidades (la `escala` se ajusta), adaptatividad al estado (la tolerancia crece con la evolución del problema, p. ej. con el endurecimiento plástico) y robustez en regímenes degenerados (cuando la escala colapsa transitoriamente, `atol` mantiene la comparación significativa).

El patrón vive centralizado por subsistema, no replicado: la admisibilidad constitutiva se aplica en `Material.is_admissible` y `Material.admissibility_tol` (ADR 0006); la convergencia de los solvers no lineales reside en la clase `ConvergenceCriterion` de `solidum/math/convergence.py` (ADR 0007). Cada material o solver declara su `escala` mediante un método ligero (por ejemplo `Material.admissibility_scale`, que devuelve la fluencia corriente o el umbral de daño), y la fórmula no se reproduce a mano en ningún sitio salvo cuando una restricción técnica (kernel compilado con Numba) obliga a precomputar la tolerancia fuera del kernel — y, aun así, vía el método centralizado.

Adicionalmente, los términos absolutos `atol` se autoderivan de las escalas del problema en su primer ensamblaje, no se codifican como constantes globales con unidades. Las constantes globales del proyecto que rigen esta política son adimensionales (`CONVERGENCE_RTOL_FORCE`, `CONVERGENCE_ATOL_FORCE_FACTOR`, `ADMISSIBILITY_TOL_REL`, etc.), lo que mantiene el código independiente del sistema de unidades elegido por el usuario.

La importancia de este mecanismo es estructural: la convergencia es lo que marca el éxito de un análisis numérico, y la diferencia entre un código robusto y uno frágil está en la disciplina con que se construyen estas comparaciones. Toda extensión futura que introduzca un nuevo criterio de comparación contra cero adopta este patrón.

5.21 Modos de cuerpo rígido derivados del nombre de los grados de libertad

Los modos de cuerpo rígido de un modelo —traslaciones, giros infinitesimales y, para un campo escalar como la temperatura, el vector constante— se derivan **sólo del nombre de los grados de libertad** (`ux`, `uy`, `uz`, `rx`, `ry`, `rz`, `T`) y de las coordenadas nodales, sin información del tipo de elemento (`solidum/math/linalg/nullspace.py`). Por eso cubren cualquier combinación del catálogo, incluidos los dominios mixtos, y un elemento nuevo los hereda sin hacer nada. Son el núcleo exacto de la matriz de rigidez de un modelo sin apoyos, verificado en todas las familias.

Los consumen dos piezas: el precondicionador multimalla del solver iterativo, que sin ellos converge peor que sin precondicionar (ADR 0018), y la detección de mecanismos del análisis estático (ADR 0019). El ensamblador los calcula una vez y los cachea mientras no cambie la topología.

5.22 Red de seguridad del análisis estático

El usuario de Solidum no elige el solver algebraico ni tiene por qué conocer cómo falla. Un solver directo ante una matriz singular devuelve resultados absurdos sin avisar (medido: desplazamientos de miles de kilómetros). El análisis estático interpone tres capas (ADR 0019, `solidum/math/solvers/model_checks.py`):

1. **Antes de resolver**: los modos de cuerpo rígido que las restricciones no impiden son un **mecanismo**. Se calculan como el núcleo de la matriz que evalúa cada modo sobre las restricciones —incluidas las lineales, que entran por el operador de reducción del ADR 0004— y se describen en términos del modelo: *"traslación en la dirección x"*, *"giro alrededor del eje z que pasa por (0, 0.5)"*, *"el campo 'T' no tiene ningún valor prescrito"*. No se aplica a análisis modales o dinámicos, donde un modelo libre es legítimo.

2. **Después de resolver** (estático lineal): el solver debe haber resuelto el sistema que se le dio: el error hacia atrás normado de la solución, escalado por la diagonal, η = `‖r̃‖/(‖K‖·‖ũ‖ + ‖F‖)` $\leq$ `10⁻¹⁰` (`K = D^-½·K·D^-½`, `D = diag(K)`) (`backward_error`, `BACKWARD_ERROR_RTOL`; desde el 2026-10-20, antes el residuo relativo `‖F − K·u‖` $\leq$ `10⁻⁸·‖F‖`, que un solver directo estable no garantiza con rigideces muy desproporcionadas). Los mecanismos internos cargados, que delataba el residuo, los rechaza antes la capa 3.

3. **Después de factorizar** (estático lineal): el modelo no debe tener un **mecanismo interno** (`find_internal_mechanisms`). Lo delata aunque ninguna carga lo active. Desde el 2026-10-20 decide la energía, no el tamaño de un pivote, en dos pasos. Un **filtro** barato: cada backend directo expone `n_small_pivots`, sus pivotes pequeños frente a la diagonal (`|U_ii|/√(|K_rr|·|K_cc|) < SUSPECT_PIVOT_RTOL`, invariante a las unidades para una secuencia de pivotes dada; Pardiso los lee de su factor con `pardiso_getdiag`). Si salta, el **veredicto**: el modo de menor energía de `K·v = λ·diag(K)·v`, calculado con la factorización existente por iteración inversa en bloque, es un mecanismo si todos los elementos que mueve tienen energía relativa `v_e^⊤·K_e·v_e / v_e^⊤·D_e·v_e < MECHANISM_ENERGY_RTOL`. Los modos que mueven los mismos elementos se agrupan y se describen en una base legible (traslaciones puras, giros, rótula esférica, campo escalar sin valor prescrito). El pivote de un mecanismo real es ruido de redondeo que

crece con el tamaño del modelo y se solapa con el de un contraste de rigidez legítimo; la energía por elemento los separa por muchos órdenes de magnitud.

Dentro de un Newton no se rechaza nada a lo largo del trazado: cerca de un punto límite resolver un sistema casi singular es parte del algoritmo, y la garantía la da el propio Newton, que exige equilibrio real. Allí el residuo del sistema lineal sólo se usa para diagnosticar mejor la divergencia. La excepción es la **tangente inicial** (2026-09-30): el corrector compartido entrega su primera factorización, la de `U = 0`, a un inspector que la red de seguridad arma al empezar el análisis (`check_initial_tangent`), así que la comprobación no factoriza dos veces. Qué hacer con un mecanismo en la tangente inicial depende de un **contrato declarativo** nuevo, `Element.GEOMETRICALLY_NONLINEAR`, y del ya existente `Material.IS_UNILATERAL`: si ningún componente puede ganar rigidez al deformarse, el mecanismo es permanente y el análisis se rechaza; si alguno puede, se avisa. Los solvers de los módulos de usuario llaman a la misma pieza pública.

Un backend que no tiene pivotes (el iterativo) lo declara con `n_small_pivots = None`, y la red avisa de que la comprobación no se hace en vez de leer un cero.

El principio que el mecanismo materializa es general: **un problema mal planteado lo detecta el sistema y lo explica en el lenguaje del modelo**, no lo diagnostica el usuario a partir de un síntoma numérico.

Capítulo 6

Convenciones de signos

La convención de signos es única para el conjunto del proyecto y de aplicación obligatoria en toda formulación, prueba, interfaz pública (`internal_forces`, `SolveResult`), documentación y catálogo. Si una referencia bibliográfica adopta otra convención, la traducción se realiza al implementar y se anota en la prueba correspondiente.

6.1 Ejes y giros

Estática 2D. Eje `x` positivo a la derecha; eje `y` positivo hacia arriba. Giro y momento positivos en sentido antihorario (regla de la mano derecha (RHR) con `z` saliente del plano).

Estática 3D. Aplicación de la RHR a todos los ejes (locales y globales) y a todos los momentos (`Mx`, `My`, `Mz`, `T`).

6.2 Fuerzas internas en 2D — convención de viga estructural

`N`, `V`, `M` son **fuerzas internas** (resultantes integradas del tensor de esfuerzos σ sobre la sección), no esfuerzos. Sobre un elemento diferencial, con `N` positivo en tracción, `V` positivo cuando tiende a rotar el diferencial en sentido horario y `M` positivo en flexión sagitada (tracción en la fibra inferior con `+y` hacia arriba):

- *Cara izquierda* (normal saliente en `-x`): `N` apunta en `-x`, `V` apunta en `+y`, `M` actúa en sentido horario.
- *Cara derecha* (normal saliente en `+x`): `N` apunta en `+x`, `V` apunta en `-y`, `M` actúa en sentido antihorario.

Esta es la convención clásica de los diagramas de fuerzas internas en vigas.

6.3 Fuerzas internas en 3D — convención de resultantes de esfuerzo bajo RHR

En la cara con normal saliente `+x_local`, las fuerzas internas positivas se definen como:

- `N` en `+x_local` (tracción positiva).
- `Vy` en `+y_local`, `Vz` en `+z_local`.
- `T` ≡ `Mx`, `My`, `Mz`: vectores de momento en `+x_local`, `+y_local`, `+z_local` respectivamente, conforme a la RHR.

En la cara con normal saliente `-x_local`, todos los sentidos se invierten (tercera ley de Newton).

Justificación: esta es la convención que resulta de integrar directamente el tensor de esfuerzos sobre la sección. Es la adoptada por Bathe, Crisfield, Cook-Malkus-Plesha, SAP2000, OpenSees, ANSYS y Abaqus para vigas 3D. La convención estructural de "flexión sagitada positiva" no admite extensión canónica a 3D y se utiliza únicamente en 2D.

Convención de fibra para los flectores en 3D, consecuencia directa del signo:

- `Mz > 0` ⇒ tracción en fibras con `y < 0` (equivalente a flexión sagitada en el plano `xy` con `+y` hacia arriba).
- `My > 0` ⇒ tracción en fibras con `z > 0`.

6.4 Capa interna y capa pública

Internamente, todos los elementos (2D y 3D) operan en la convención de resultantes de esfuerzo bajo RHR. Las matrices `B`, las fuerzas internas `f_int` = $\int B^\top \sigma \; d\Omega$, los jacobianos y los residuos se calculan en dicha convención. Esta uniformidad evita signos especiales por dimensión dentro del código de formulación.

En la interfaz pública (`internal_forces`, diagramas, catálogo), los elementos 3D la exponen sin transformación. Los elementos 2D, en cambio, exponen la convención de viga estructural (`V` con signo opuesto al interno) por ser la representación clásica en los diagramas de fuerzas internas. La traducción consiste en un simple cambio de signo en `V`, aplicado dentro del método `internal_forces()` del elemento 2D.

Relación 2D ↔ 3D en la interfaz pública:

- `M_2D` ≡ `Mz_3D` (mismo signo; ambos corresponden a flexión sagitada positiva con `+y` hacia arriba).
- `V_2D` y `Vy_3D` difieren en signo en la interfaz pública por la razón anterior.

Los valores se exponen siempre en ejes locales del elemento; la transformación a ejes globales constituye una capa separada.

6.5 Interfaces: orden de las componentes

Los materiales de interfaz (ADR 0033) reciben el desplazamiento relativo y devuelven la tracción en el sistema local de la interfaz, con las componentes **normales primero y las tangenciales después**: `(n, s)` o `(n, s1, s2)` en una superficie (grieta, junta) y `(n, s)` o `(n1, n2, s)` en la línea de una barra embebida. El desplazamiento relativo es el del lado positivo menos el del negativo (en una grieta, apertura normal positiva; en una barra, la barra menos el sólido que la rodea), y la tracción tangencial lleva el signo del deslizamiento. El paso a ejes globales es del elemento que consume la ley.

6.6 Reacciones en los análisis dinámicos

La reacción de un apoyo sigue la convención del estático (ADR 0004 §3), extendida a la ecuación de movimiento en las filas de apoyo (ADR 0030): $\mathbf{R} = \mathbf{F}_{\text{int}}(\mathbf{u}) + \mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} - \mathbf{F}$, de modo que la suma de las reacciones equilibra la carga y las fuerzas de inercia y de amortiguamiento de la estructura entera. La parte $\mathbf{C}\dot{\mathbf{u}}$ se reporta por separado; en el armónico, la misma regla en amplitudes complejas, y en el espectral, la ecuación de cada modo antes de combinar.

Capítulo 7

Crecimiento del sistema

Los capítulos anteriores describen el estado actual del programa. Este capítulo describe el procedimiento por el que se incorporan piezas nuevas — el aspecto del proyecto que la arquitectura optimiza explícitamente.

7.1 Puntos de extensión canónicos

Componente a incorporar	Ubicación	Mecanismo de registro	Modifica otros archivos
Material	`solidum/materials/<snake>.py`	`@MaterialRegistry.register`	No
Material térmico	`solidum/materials/<snake>.py`	`@ThermalMaterialRegistry.register`	No
Material de interfaz	`solidum/materials/interface/<snake>.py`	`@InterfaceMaterialRegistry.register`	No
Sección de marco	`solidum/sections/<snake>.py`	`@SectionRegistry.register`	No
Elemento	`solidum/elements/<familia>/<snake>.py`	`@ElementRegistry.register`	No
Solver	`solidum/math/solvers/<snake>.py`	`@SolverRegistry.register`	No
Generador de restricciones	`solidum/bc/<snake>.py`	`@ConstraintRegistry.register`	No

Los tres primeros más frecuentes son el material, el elemento y el solver, los que cubre la *skill* de plantillas; los demás siguen el mismo patrón con su propia clase base (`InterfaceMaterial`, `Section`, el generador de restricciones del ADR 0032) y su tipo de spec.

La columna "modifica otros archivos" es deliberadamente "No". Si la incorporación de un componente exigiera la modificación del intérprete, el ensamblador o la inicialización, dicha situación constituiría un síntoma de degradación de la arquitectura y procedería un refactor con su correspondiente Architecture Decision Record (ADR).

La tabla vale para los componentes **estándar**, que van al programa principal. El **criterio de admisión** (Reglas.md §1) decide la frontera: una formulación es estándar si aparece en los textos de referencia del área y está implementada en al menos un código general establecido; la spec lo

declara en su bloque `admission` (`textbooks`, `general_codes`, con los identificadores canónicos de `solidum/tools/spec.py`). Un componente **no estándar** (una formulación de investigación que no cumple el criterio: la discontinuidad embebida con cinemática KOS de Retama (2010), el seguimiento de trayectoria de grieta, el control del paso por liberación de energía de Gutiérrez (2004)) va a un **módulo de usuario** (ADR 0020). XFEM y las leyes cohesivas tracción-salto genéricas con elementos de interfaz, en cambio, son estándar según el criterio. El módulo tiene su código en `solidum/user/<nombre>/`, con los mismos decoradores y, si lo necesita, una familia de material propia declarada con una subclase de `Registry`; specs en `docs/user/<nombre>/specs/` y pruebas en `tests/user/<nombre>/`. No modifica ningún archivo del programa principal ni se carga solo: el caso lo pide con `user_modules: [<nombre>]`. Sólo añade entradas a los registros: si reasigna una existente, la carga falla y se deshace. Si le hace falta una pieza que el programa principal no ofrece, esa pieza entra en el programa principal como mecanismo genérico, justificado por casos reales ajenos al módulo, nunca como un caso especial que lo nombre. Lo vigilan el test de pureza (`tests/test_main_program_purity.py`, con sus controles estructural, léxico y en ejecución; el módulo nuevo no se da de alta en él) y, en sentido inverso, `tests/test_user_module_public_imports.py`, que exige al módulo usar sólo el contrato público del programa principal (ver «Programa principal y módulos de usuario»).

7.2 Skill `/solidum-new`

Para reducir también la fricción ergonómica de la incorporación, el repositorio incluye una *skill* versionada (`.claude/skills/solidum-new/`) que el asistente invoca cuando el usuario solicita un material, elemento o solver nuevo. La invocación es `/solidum-new material|element|solver <Name>`, con `--user <modulo>` para generar el componente dentro de un módulo de usuario, y produce automáticamente:

- El archivo en su carpeta canónica con la plantilla del tipo correspondiente.
- El decorador `@register` correctamente posicionado.
- Una prueba esqueleto en `tests/` con la estructura habitual (configuración, expectativas, validación contra solución analítica si aplica).

La *skill* no sustituye al razonamiento físico ni a la formulación; constituye estrictamente un mecanismo de generación de plantillas. Cierra el ciclo entre la arquitectura (optimizada para extensión) y la herramienta operativa que la materializa.

7.3 Ciclo especificación → código → catálogo

Cuando el componente a incorporar es físico (no infraestructura), el flujo es:

1. *Apertura de la especificación.* El usuario abre una especificación en `docs/specs/<Nombre>.md` a partir de la plantilla `_template_<kind>.md`. La especificación contiene: especificación física, formulación numérica y contrato YAML (`interface`, `parameters`, `acceptance`, `references`).
2. *Bloqueo en ausencia de especificación completa.* El asistente no escribe código hasta que la especificación esté completa. Si falta información, formula consultas en la sección *Diálogo* de la

propia especificación. Esto convierte la especificación, sucesivamente, en orden de trabajo y en referencia detallada del componente.

3. *Implementación.* El asistente implementa el componente y añade pruebas conforme a los criterios de `acceptance`.

4. *Cierre.* Tras la validación (todas las pruebas verdes contra los criterios), el asistente actualiza `status: validated` en la especificación e inserta una entrada en el catálogo correspondiente (`docs/catalogo_<elementos|materiales|solvers|restricciones>.md`; los materiales térmicos y de interfaz tienen su apartado en el de materiales, y las secciones de marco en el de elementos).

La relación es: la especificación constituye la orden de trabajo y la referencia detallada por componente; el catálogo constituye el índice navegable del conjunto. El manual de referencia (`Reference_manual.pdf`) se genera de forma automática a partir de las especificaciones. Un componente de un módulo de usuario sigue el mismo ciclo con su spec en `docs/user/<nombre>/specs/`, su entrada en la sección «Módulos de usuario» del catálogo y su capítulo en el apéndice «Módulos de usuario» del manual de referencia.

7.4 Architecture Decision Records: memoria de arquitectura persistente

Para los cambios que afectan al diseño del programa (no a la implementación interna de un módulo), el asistente crea un Architecture Decision Record en `docs/adr/000N-titulo.md` antes del *commit* o de forma simultánea al mismo. Los ADR son persistentes y consultables; sustituyen a las explicaciones efímeras del *chat* a fin de que el sistema permanezca comprensible meses o años después. Los ADR vigentes se enumeran en el siguiente capítulo.

7.5 Mantenimiento de los documentos vivos

Los archivos `docs/arquitectura.md`, `docs/conceptos_clave.md` y este manual se mantienen sincronizados con el estado del código. El asistente los actualiza cuando se modifica la topología del sistema (introducción de una capa nueva, una carpeta canónica nueva o una responsabilidad transversal nueva), no cuando se modifica la implementación interna de un módulo. Si tras un refactor de tamaño medio cualquiera de estos documentos no refleja la realidad del código, dicha discrepancia constituye un defecto del propio refactor.

Capítulo 8

Decisiones de arquitectura (ADR)

Los Architecture Decision Records (ADR) son el registro persistente de las decisiones que han modelado la arquitectura de Solidum FEM. Cada ADR responde, en una página, a tres cuestiones: contexto, decisión y consecuencias. En este capítulo se resumen los ADR vigentes; el texto completo reside en `docs/adr/`.

8.1 ADR 0001 — Mecanismos de transparencia conceptual

Fecha: 19 de abril de 2026. **Estado**: aceptado.

Contexto. El documento `Reglas.md` §2 establece el modelo "calculadora con manual disponible": el usuario delega la escritura del código pero conserva su comprensión conceptual. Para que dicho principio se opere, y no permanezca como mera declaración de intenciones, el proyecto requiere mecanismos prácticos que prevengan la deriva hacia la opacidad a medida que el código crece.

Decisión. Se introducen tres mecanismos vivos, de mantenimiento económico y mutuamente reforzantes:

1. *Mapa de arquitectura.* El archivo `docs/arquitectura.md` consiste en una página con diagrama de bloques y flujo de datos. El asistente lo actualiza al modificarse la topología del sistema.

2. *Conceptos clave.* El archivo `docs/conceptos_clave.md` contiene un inventario explícito de los conceptos sostenedores del sistema, cada uno con dos o tres frases conceptuales. Define el "test de no opacidad": el usuario debe poder explicar cualquier entrada en términos generales en cualquier momento.

3. *Modo "explícame X" bajo demanda.* Protocolo conversacional, no artefacto en repositorio. Constituye la válvula de escape ante opacidad puntual sin necesidad de sesiones programadas.

Consecuencia para este manual. El presente `Architecture_manual.pdf` es el siguiente eslabón en la cadena de transparencia conceptual: amplía el mapa de arquitectura a una lectura cómoda en formato de libro, manteniendo la disciplina de no entrar al detalle de cada componente.

8.2 ADR 0002 — Interfaz pública de resultados para consumidores externos

Fecha: 22 de abril de 2026. **Estado**: aceptado.

Contexto. Solidum FEM se consume desde una GUI externa (SolidumBAR, llamado FenixBAR hasta el 2026-09-30, programa de análisis estructural con elementos barra). Tras una solución, el consumidor requiere información suficiente para visualizar el post-proceso estándar de Computer-Aided Engineering (CAE): deformada escalada, reacciones en apoyos y diagramas de fuerzas internas sobre cada barra. Esta información no se exponía de forma uniforme: el componente `VtkExporter` cubría únicamente algunos elementos y las fuerzas internas carecían de método público homogéneo.

Decisión. Interfaz pública estructurada en tres niveles:

1. *Contrato por elemento.* Todos los elementos tipo armadura, cable o marco implementan `internal_forces(U) -> ElementForces`, que devuelve las fuerzas internas en ejes locales con claves homogéneas por familia (`{N}` para armadura y cable; `{N, V, M}` para marco 2D; `{N, Vy, Vz, T, My, Mz}` para marco 3D). La convención de signos es la del proyecto (capítulo anterior).

2. *Agregado en `Domain`.* El objeto `SolveResult` se calcula de forma anticipada al final de `solver.solve()` y es inmutable. Expone los desplazamientos globales, las cargas aplicadas, las reacciones y un diccionario de fuerzas internas por elemento.

3. *Puntos de entrada oficiales.* Las funciones `solidum.run(case)` y `solidum.run_yaml(path)` devuelven directamente un `SolveResult`. Cualquier consumidor externo opera contra esta interfaz.

Consecuencia. El consumidor externo no recalcula las fuerzas internas a partir de `U`; la lógica de MEF reside íntegramente en Solidum FEM. La incorporación de un elemento nuevo obliga a implementar `internal_forces` con las claves de su familia: el contrato es explícito.

8.3 ADR 0003 — Despachador de algoritmo algebraico

Fecha: 26 de abril de 2026. **Estado**: aceptado; fases 1 y 2 implementadas.

Contexto. Dentro de cada iteración del método de Newton-Raphson se requiere la resolución del sistema lineal $\mathtt{K} \cdot \delta\mathtt{U} = \mathtt{R}$. El algoritmo utilizado hasta el momento era SuperLU (factorización LU general), válido para todo caso pero subóptimo cuando `K` es simétrica y definida positiva, situación habitual en estática lineal y en numerosas no linealidades suaves. La factorización de Cholesky es típicamente entre 1.5 y 2 veces más rápida y consume considerablemente menos memoria en dichos casos.

Decisión. Se introduce un subsistema algebraico independiente del solver no lineal, con varios algoritmos disponibles y un despachador que selecciona el adecuado de forma automática:

- `LUSolver` (SuperLU) — algoritmo universal.
- `CholeskySolver` (CHOLMOD) — para matrices simétricas y definidas positivas; opcional según disponibilidad de `scikit-sparse`.

- `LDLTSolver` — reservado para la fase 2 (matrices simétricas indefinidas).

El despachador `select_solver(props, override)` recibe un objeto `StiffnessProperties` (atributos `is_symmetric`, `is_positive_definite`) y selecciona la factorización de Cholesky cuando aplica, o la factorización LU en caso contrario. El usuario puede forzar un algoritmo desde el archivo YAML mediante el campo `linear_algebra` en calidad de herramienta de diagnóstico, no como decisión de modelado.

Consecuencia. Los solvers no lineales (`LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`) ya no invocan SuperLU directamente: solicitan un solver algebraico al despachador y se desentienden del algoritmo numérico subyacente. La incorporación de un nuevo algoritmo (Pardiso, MUMPS, métodos algebraicos multimalla iterativos) afecta exclusivamente a `solidum/math/linalg/`. La separación entre estrategia (solver no lineal) y táctica (subsistema algebraico) queda explícita en el código.

8.4 ADR 0004 — Imposición de condiciones de Dirichlet por eliminación directa

Fecha: 30 de abril de 2026. **Estado**: aceptado; fases 1 (Dirichlet nodal) y 2 (restricciones lineales afines) implementadas. Queda abierta la interpretación de las reacciones con restricciones multipunto (punto 10 de la fase 2).

Contexto. Las condiciones de contorno de Dirichlet, tanto los empotramientos nodales clásicos como las restricciones multipunto (MPC: asentamientos prescritos, apoyos en plano oblicuo, periodicidad de celda unitaria, unión rígida master-slave), se imponían previamente por método de penalidad: añadir términos diagonales muy grandes al sistema. Dicho método introduce condicionamiento numérico artificialmente malo, contamina la simetría y la positividad de la matriz de rigidez y no es exacto a redondeo.

Decisión. Se adopta la imposición por eliminación directa. Toda restricción se expresa en forma afín `u_s = g_s + Σ α_si · u_mi` y se acumula en un `ConstraintSet` del subpaquete `solidum.bc`. El ensamblador construye un operador disperso `T` y un vector `g` tales que `u = T · u_libre + g`. El sistema entregado al subsistema algebraico es $K_{red} = T^{\top} K T$, $F_{red} = T^{\top}(F - K \cdot g)$. Las restricciones encadenadas (un esclavo cuyo maestro es a su vez esclavo) se resuelven por cierre transitivo al construir `T`; los ciclos y redeclaraciones inconsistentes se detectan en validación temprana.

Consecuencia. La imposición es exacta a redondeo, preserva simetría y positividad definida de `K`, y deja el sistema reducido apto para los algoritmos del subsistema algebraico, incluidos solvers iterativos como gradiente conjugado o GMRES. La misma maquinaria cubre empotramiento, asentamiento, apoyo oblicuo, periodicidad y unión rígida sin ramificación específica por tipo.

8.5 ADR 0005 — Logging configurable en lugar de `print` directo

Fecha: 7 de mayo de 2026. **Estado**: aceptado.

Contexto. La trazabilidad del solver durante un análisis (incrementos de carga, iteraciones de Newton, ratios de convergencia, factorizaciones reusadas) se emitía mediante `print` directo en la salida estándar. Esto impedía silenciar el ruido en pipelines de batch testing, redirigir la traza a un archivo de log paralelo y diferenciar niveles de gravedad (información ordinaria, advertencias, errores recuperables).

Decisión. Se introduce un *logger* configurable basado en el módulo estándar `logging` de Python, expuesto al resto del proyecto vía `solidum.logging.get_logger(name)`. Todos los componentes consumen este *logger* en lugar de `print`. El nivel y el formato se configuran de forma centralizada y pueden sobreescribirse desde el archivo YAML del caso o programáticamente.

Consecuencia. Tests automatizados pueden silenciar la traza sin perder la información de errores. La integración con herramientas externas (GUI SolidumBAR, batería de tests, pipelines CI) se hace mediante la API estándar de `logging`, sin acoplamiento a `sys.stdout`. Las advertencias internas relevantes (degradación de Cholesky a LU, bisección de incremento) quedan marcadas con su nivel adecuado.

8.6 ADR 0006 — Tolerancias del criterio de admisibilidad constitutiva

Fecha: 12 de mayo de 2026. **Estado**: aceptado.

Contexto. Cada modelo constitutivo no lineal tiene un criterio de admisibilidad — una función `f(estado)` que decide si el material está en régimen reversible (`f` $\leq$ `0`) o disipativo (`f > 0`). En aritmética de doble precisión la comparación no puede ser exacta: tras un return mapping, el ruido de redondeo deja `f` cerca de cero pero no exactamente cero, por lo que la comparación se escribe `f` $\leq$ `tol`. La implementación anterior usaba una tolerancia absoluta hardcodeada (`PLASTIC_YIELD_TOL = 1e-9`), frágil bajo cambio de unidades y no adaptativa al estado en modelos con endurecimiento severo o sin parámetro de referencia inicial.

Decisión. Se adopta el esquema combinado absoluto + relativo, estilo solver de ecuaciones diferenciales ordinarias: `f` $\leq$ `ADMISSIBILITY_TOL_ABS + ADMISSIBILITY_TOL_REL` $\cdot$ `escala(estado)`. Cada material declara su escala característica vía el método `admissibility_scale(state_vars)`, devolviendo un esfuerzo positivo característico del criterio en el estado corriente: σ`_y + H`$\cdot\alpha$ para plasticidad J2 1D, `E`$\cdot\kappa$`_0` para daño isótropo, $\sqrt{}$`(2/3)`$\cdot$`(`σ`_y + H`$\cdot\alpha$`)` para Von Mises 2D. La comparación se encapsula en `Material.is_admissible(f, state_vars)` y en `Material.admissibility_tol(state_vars)` para que los modelos no la repliquen.

Consecuencia. Misma física en MPa, Pa, GPa o adimensional produce idéntico estado interno hasta precisión de doble. En endurecimiento severo la tolerancia crece junto con la fluencia. Modelos energéticos futuros (phase-field, gradient damage) encajan declarando su escala ($\sqrt{}$`(2·E·g_c)`) sin tocar la política. Los modelos con kernel compilado (Numba) consumen `admissibility_tol` desde código Python antes de invocar el kernel, preservando la centralización de la fórmula.

8.7 ADR 0007 — Tolerancias de convergencia en solvers no lineales

Fecha: 13 de mayo de 2026. **Estado**: aceptado.

Contexto. El criterio de convergencia del `NonlinearSolver` y del `ArcLengthSolver` era puramente relativo: ‖`residuo`‖ `/ ref_force` $\leq$ `tol` y ‖δ`U`‖ `/` ‖`U`‖ $\leq$ `tol`, con un único parámetro `tol` para los dos criterios y sin término absoluto real. Esta forma arrastraba tres limitaciones estructurales: no había tolerancia absoluta cuando la escala relativa colapsaba transitoriamente, una sola tolerancia mezclaba dos criterios físicamente distintos (incrementabilidad del iterado vs equilibrio), y la fórmula se replicaba en los dos solvers a mano.

Decisión. Se aplica el mismo patrón del ADR 0006 a la convergencia, separado por criterio.

La política reside en una clase pequeña `ConvergenceCriterion` en `solidum/math/convergence.py`, que encapsula configuración (cuatro tolerancias: `rtol_force`, `rtol_disp`, `atol_force_factor`, `atol_disp_factor`) y estado calibrado (los `atol` efectivos derivados de las escalas del primer ensamblaje). Los solvers no lineales reciben una instancia en el constructor (parámetro `convergence`), calibran al inicio de `solve()` y delegan en `evaluate()` la comparación. La semántica es AND, no OR (ambos criterios deben cumplirse simultáneamente), coherente con la convención de Bathe, Crisfield y Owen-Hinton.

Consecuencia. Invariancia bit-paritaria bajo cambio de unidades (el mismo problema en N/m, kN/mm o MN/m converge en el mismo número de iteraciones). Régimen degenerado (carga inicial nula, control en desplazamiento puro) deja de oscilar espuriamente: el piso absoluto `atol_force` autoderivado define cuándo se da por convergido un residuo numéricamente cero. Problemas casi-rígidos y plásticos perfectos se afinan independientemente vía `rtol_disp` vs `rtol_force` sin compromisos. La política vive en un único punto del código; un solver no lineal futuro la consume sin replicarla.

8.8 ADR 0008 — Densidad como propiedad del material

Fecha: 13 de mayo de 2026. **Estado**: aceptado.

Contexto. El cableado de fuerza de cuerpo introducido previamente aceptaba un vector `body_force: [bx, by, bz]` global, aplicable uniformemente a todos los elementos. Esto servía para estructuras monomaterial — el usuario calculaba $\rho \cdot g$ una vez y lo declaraba — pero fallaba en estructuras multimaterial: con un único vector global, no había forma de distinguir el peso propio del acero del hormigón o de un cable tensado. Adicionalmente, la densidad es la magnitud física que abre la puerta a la matriz de masa elemental y por tanto a todo el subsistema dinámico futuro.

Decisión. La densidad es propiedad del material, no del elemento. Se introduce como atributo `density` en la clase base `Material` con valor por defecto `None` (no declarado). La densidad es físicamente intrínseca al medio continuo; vive en el mismo nivel jerárquico que `E`, ν o `σ_y`. En el YAML se acepta el bloque atajo `gravity: [0, -9.81]` complementario a `body_force` (mutuamente excluyentes). El ensamblador incorpora `assemble_self_weight(g)` que itera sobre elementos y aplica `b_element = material.density · g`. Si algún material tiene `density=None`, el método falla con `ValueError` listando los materiales afectados — enforcement diferida al uso, sin posibilidad de propagar masa cero silenciosa a un análisis dinámico.

Consecuencia. Peso propio físicamente correcto en estructuras multimaterial. API del YAML más intuitiva. Tests existentes con `body_force` siguen pasando. La fase de análisis modal y dinámico hereda la propiedad sin rediseño: `Assembler.assemble_mass_matrix` reutiliza el mismo patrón de validación.

8.9 ADR 0009 — Análisis modal y dinámico

Fecha: 13 de mayo de 2026. **Estado**: aceptado; **todas las fases (1-7) implementadas** al 18 de mayo de 2026, junto con la variante HHT-α y las reglas C y D de refactor arquitectural aplicadas.

Contexto. El ADR 0003 había identificado análisis modal y dinámica implícita entre los análisis futuros que la capa algebraica iba a habilitar; el ADR 0008 había puesto la densidad como propiedad del material disponible. Quedaba abrir el subsistema con dos análisis distintos pero relacionados: el análisis modal — problema algebraico de valores y vectores característicos generalizado $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ —, cuyas soluciones admiten interpretación dinámica pero no constituyen análisis dinámico

per se; y el análisis dinámico propiamente dicho, con dependencia temporal explícita — integración de `M · ü + C · u̇ + K · u = F(t)`.

Decisión. Se acometen ambos análisis con una hoja de ruta común y siete decisiones arquitecturales fijadas hoy para no reabrir debates en fases posteriores:

1. Matriz de masa **consistente** en la fase 1 (con contrato extensible a `lumped` mediante un parámetro `lumping` ya presente en la firma de `compute_mass_matrix`, suficiente para la futura integración explícita).

2. Ensamblaje de M paralelo al de K, reutilizando la topología COO cacheada del `Assembler` y la enforcement de densidad del ADR 0008.

3. `ModalSolver` registrado en el mismo `SolverRegistry` que los solvers estáticos: no se crea un `IntegratorRegistry` paralelo, manteniendo el patrón "un solver = un análisis = una clase registrada".

4. Dirichlet en el problema modal por eliminación directa (mismo mecanismo del ADR 0004); el término independiente `g` de restricciones lineales no aplica en modal.

5. Estado dinámico transitorio `(u, u̇, ü)` fuera de `Node`: vive en un `TransientResult` específico, preservando la pureza topológica/geométrica de la clase nodal y evitando que análisis estáticos paguen el coste de campos que no usan.

6. Amortiguamiento Rayleigh `C = α · M + β · K` como entrada estándar para la fase de dinámica transitoria, con coeficientes calibrables modalmente a partir de dos pares `(ξ, ω)`. Amortiguamiento modal por modo y otros esquemas (Caughey, local) diferidos.

7. Convención de unidades heredada del modelo (ADR 0008): el usuario es responsable de la consistencia; las tolerancias adimensionales del ADR 0007 garantizan invariancia bajo cambio de unidades.

Las siete fases quedan implementadas y validadas con tests contra solución analítica y recuperación a paridad de bits del caso lineal en ausencia de plasticidad:

- **Fase 1** (`ModalSolver` + ARPACK Lanczos con shift-invert). Cerrada 2026-05-13.
- **Fase 2** (mass lumping HRZ canónico en sólidos + nodal directo en frames). Helper centralizado `solidum.math.mass_lumping.lump_hrz`. Cerrada 2026-05-18 (commit `c92bf4e`).
- **Fase 3** (`NewmarkSolver` con `(β, γ)` parametrizables, factorización única, Rayleigh, cargas por callback). Cerrada 2026-05-13.
- **Fase 3-bis** (`HHTSolver` como variante de Newmark con disipación numérica controlada por $\alpha \in$ `[-1/3, 0]`). Cerrada 2026-05-18 (commit `73742f4`).
- **Fase 4** (`NewtonNewmarkSolver` con Newton interno, jacobiano `J = M + γΔt·C + βΔt²·K_t`, convergencia dual ADR 0007, Rayleigh constante con `K_0`; `NewtonHHTSolver` análogo). Cerrada 2026-05-13/-18.

- **Fase 5** (`CentralDifferenceSolver` — leapfrog explícito Belytschko-Liu-Moran, lineal y no lineal en una clase con `nonlinear`, estabilidad condicional CFL con detección a posteriori). Cerrada 2026-05-18 (commit `e66473d`).
- **Fase 6** (`HarmonicSolver` — respuesta forzada armónica $(-\omega^2\mathbf{M}+i\omega\mathbf{C}+\mathbf{K})\cdot\hat{\mathbf{u}}=\mathbf{F}$ con barrido configurable, aritmética compleja). Cerrada 2026-05-18 (commit `47a1333`).
- **Fase 7** (`ResponseSpectrumSolver` — combinación modal SRSS/CQC sobre espectro de respuesta). Cerrada 2026-05-18 (commit `5151d7d`).

Reglas arquitecturales aplicadas durante el cierre del ADR:

- **Regla C** (auditoría 2026-05-13). Disparada por `CentralDifferenceSolver` como primer solver fuera de la jerarquía de Newmark. `entry.py::run_yaml` despacha por atributo de clase `PIPELINE_KIND` ∈ `{"static", "modal", "transient", "harmonic", "spectrum"}` en lugar de cadena de `isinstance`. Solvers no clásicos futuros no requieren tocar `entry.py`.
- **Regla D** (auditoría 2026-05-13). Disparada por `ResponseSpectrumSolver` como segundo método de cómputo sobre `ModalResult`. El algoritmo de `free_vibration` migra a `solidum/math/modal_response.py` junto con los nuevos `participation_factors`, `response_spectrum_srss`, `response_spectrum_cqc`, `spectrum_from_sa`, `spectrum_tabulated`. `ModalResult.free_vibration` queda como wrapper delgado, preservando la API histórica.

Consecuencia. El subsistema modal/dinámico/espectral queda **completo en su totalidad**. Las extensiones futuras (excitación sísmica multi-direccional simultánea CQC3, multi-support seismic, generalized-α) requieren caso de uso específico — no son piezas del ADR pendientes. La clase `Node` conserva su semántica original.

Ampliaciones (9 de noviembre de 2026). (1) §8, decisión del usuario: la razón de masa efectiva acumulada se mide contra la masa que puede participar, $M_{\text{part}} = \mathbf{b}^\top \mathbf{M}_{LL}^{-1}\mathbf{b}$, la suma de las masas efectivas de todos los modos, de modo que con todos los modos vale 1 exactamente (con masas de nudo, grados de libertad sin masa y restricciones multipunto); `ModalResult.participating_mass` y `cumulative_effective_mass_ratio`. (2) Paso adaptativo en `NewtonNewmarkSolver` y `NewtonHHTSolver`, decisión del usuario: el paso cuyo Newton no converge se repite desde el último estado convergido en dos mitades, hasta `max_subdivisions` niveles (6 por omisión, $\Delta t/64$); el historial queda en la malla de tiempos del usuario. (3) Los transitorios implícitos admiten grados de libertad sin masa (ADR 0024, §6 bis).

8.10 ADR 0010 — Discontinuidades interiores embebidas

Fecha: 13 de mayo de 2026. **Estado**: aceptado; fases 1, 2, 3 y 3b implementadas; fase 4 (curva de Van Vliet) validada con grieta prescrita el 29 de septiembre de 2026; fase E (seguimiento de grieta) completada el 10 de noviembre de 2026; las demás, diferidas con justificación.

Contexto. Abrir la línea de fractura computacional. Entre las alternativas (fisura difusa, elementos cohesivos de interfaz, XFEM, discontinuidades embebidas) se elige la que el autor domina a nivel doctoral: discontinuidades interiores embebidas en aproximación discreta (Retama, 2010).

Decisión. Cinco abstracciones transversales: (1) una familia `CohesiveMaterial` **paralela** a `Material`, no subclase, porque relaciona tracción con salto de desplazamiento y no esfuerzo con

deformación; (2) elementos con **grados de libertad enriquecidos a nivel elemental**, invisibles para el ensamblador; (3) **condensación estática local** del salto dentro del elemento, de modo que el sistema global no cambia de tamaño; (4) un `DiscontinuityState` que guarda la orientación y la historia de la discontinuidad; (5) una semántica de **elemento que cambia de modo** (continuo a agrietado), cuya activación se decide entre pasos con el hook `prepare_step` y no dentro del Newton, para evitar oscilaciones sobre el umbral. Se fija además la longitud de discontinuidad `l_d = (A/h)·cos(`$\theta-\alpha$`)`, aportación del capítulo 6 de la tesis, validada numéricamente por primera vez.

Consecuencia. Fractura computacional sin tocar el ensamblador, el despachador algebraico, el Newton ni el parser. Las cuatro abstracciones son reutilizables para modelos cohesivos clásicos, modos incompatibles o EAS. El límite conocido es el solver para ablandamiento severo con penalización cohesiva rígida (deuda técnica). *(Nota 2026-09-29: ese límite se diagnosticó antes de corregir la tangente cohesiva el 2026-09-23; con la tangente corregida, `NonlinearSolver` con control por desplazamiento recorre el ablandamiento hasta la separación, y el fallo de `ArcLengthSolver` no se ha vuelto a comprobar.)*

Ubicación. Desde el ADR 0020 la formulación vive fuera del programa principal, en el módulo de usuario `discontinuities` (`solidum/user/discontinuities/`), y un modelo la carga con `user_modules: [discontinuities]`. El ADR se conserva como registro de la formulación.

Actualizaciones (10 de noviembre de 2026). *Seguimiento de grieta* (fase E, decisiones del usuario): el nodo solitario se elige por la trayectoria de la grieta y no elemento a elemento, con un seguimiento global por región conexa (`GlobalCrackTracking`) o una trayectoria prescrita (`PrescribedCrackPath`), y `l_d` lleva signo. *Familia cohesiva*: pasó al programa principal como familia de materiales de interfaz (ADR 0033); lo que aquí se dice de `CohesiveMaterial` vale para `InterfaceMaterial`. El elemento acepta también la ley de modo mixto `CohesiveDamageMixedMode`, que transmite cortante por la grieta; la cinemática sigue siendo KOS. El módulo conserva el elemento, el estado de la discontinuidad y el seguimiento.

8.11 ADR 0011 — Robustez del Newton: line search y diagnóstico de divergencia

Fecha: 18 de mayo de 2026. **Estado**: aceptado con una enmienda.

Contexto. Los solvers no lineales divergían con un `RuntimeError` genérico que no distinguía entre causas cualitativamente distintas, y no tenían globalización.

Decisión. Dos mejoras coordinadas: un **line search** de backtracking con condición de descenso sobre el residuo, y una **familia de excepciones tipadas** que clasifica la divergencia (oscilación del Newton, tangente singular, carga por encima de la capacidad, modo desconocido) con métricas y una pista textual. **Enmienda**: el line search, previsto activo por defecto, quedó **desactivado por defecto** al comprobarse que rompía casos de daño que convergían sin él; con Newton completo y tangente consistente el backtracking apenas aporta (de Borst y Sluys, tabla 3.1).

Consecuencia. Un fallo de convergencia se reporta con su causa probable y la acción típica, no con un mensaje genérico.

8.12 ADR 0012 — Sólidos 3D y notación de Voigt en seis componentes

Fecha: 19 de mayo de 2026. **Estado**: aceptado.

Contexto. Abrir la familia de elementos sólidos 3D obligaba a fijar la notación de Voigt tridimensional y a decidir qué hace el contrato de fuerzas internas en un sólido.

Decisión. (1) Voigt 3D con orden `[xx, yy, zz, xy, yz, xz]` y deformaciones angulares *engineering*, extensión natural del orden 2D (difiere de ABAQUS en la permutación `yz` ↔ `xz`); (2) el contrato `internal_forces()` **no se aplica a sólidos**: un continuo no tiene resultantes seccionales, su salida canónica es el campo de esfuerzos por punto de Gauss; (3) una API de tracciones y caras 3D paralela a la de bordes 2D.

Consecuencia. Especificaciones de sólidos 3D sin ambigüedad de notación, y el contrato de resultados del ADR 0002 cerrado por dominio explícito en vez de quedar como deuda.

8.13 ADR 0013 — Orientación material y ortotropía

Fecha: 9 de septiembre de 2026. **Estado**: aceptado.

Contexto. El primer material anisótropo (`Orthotropic2D`, para la investigación con bambú) plantea dónde vive el ángulo de fibra: en el material, en el elemento, o en un campo sobre la malla.

Decisión. El ángulo va **en el material**, como atajo consciente y desechable: es una entrega puramente aditiva que no toca los contratos `Material` ni `Element`, y resuelve el caso real (probetas a ángulo constante). Se declara explícitamente que la orientación es conceptualmente una propiedad del elemento, y se escribe la condición de migración: cuando aparezca un caso real con orientación variable en la malla. Se fijan también la validación de admisibilidad propia de la ortotropía ($|\nu_{12}| < \sqrt{(E_1/E_2)}$, no la de `Elastic2D`) y la convención de subíndices de Poisson.

Consecuencia. Calibración experimental desbloqueada sin riesgo de regresión. El isótropo queda como caso particular verificable.

8.14 ADR 0014 — Ensamblaje por lotes

Fecha: 22 de septiembre de 2026. **Estado**: aceptado; fases 1-3 implementadas el mismo día.

Contexto. El ensamblaje recorría los elementos uno a uno en Python y, dentro de cada uno, los puntos de Gauss uno a uno: más del 90 % del tiempo era sobrecarga de despacho, no aritmética (120–170 µs por Quad4 y ensamblaje; 0,4 s por commit copiando diccionarios), y el estado interno como diccionarios ocupaba más que la matriz global. La solución conocida desde los supercomputadores vectoriales es invertir el bucle y evaluar por lotes los elementos que comparten receta, pero el proyecto exige que la agrupación no fije hipótesis que un componente futuro tenga que deshacer.

Decisión. (1) La *familia* se deriva de los contratos declarativos —clase de elemento, instancia de material, cuadratura, número de nodos—, nunca de listas. (2) El camino por elemento sigue siendo el contrato obligatorio y la referencia física; el camino por lotes es una capacidad que el componente declara (`BATCH_KINEMATICS` en el elemento, `STATE_SCHEMA` + `BATCH_KERNEL` en el material) con fallback automático dentro del mismo ensamblador. (3) Un único kernel de familia, compilado una vez y cacheado en disco, recibe la cinemática y la constitutiva como funciones tipadas por su firma. (4) Ambos caminos ejecutan las mismas funciones compiladas por punto de Gauss, y el barrido de

contratos exige equivalencia a precisión de máquina. (5) El estado interno se declara (`STATE_SCHEMA`, obligatorio) y se almacena por arreglos con vistas compatibles con `ElementState`. (6) Nada derivado de la configuración deformada se cachea. (7) Trozos con presupuesto de memoria y mapa COO → CSR cacheado. (8) Cierre del mismo día: kernel paralelo con `prange` bit a bit igual al serie, señalización del jacobiano degenerado por bandera (una excepción dentro de una región paralela se pierde) y post-proceso por familia con el mismo kernel de cinemática y constitutiva.

Consecuencia. Quad4 + J2 ×27 y Hex8 + J2 ×13 en el ensamblaje con el kernel serie, ×110 y ×84 con el paralelo en 16 hilos; la exportación VTK ×6–8; el commit pasa de cientos de milisegundos a una copia de arreglo; el estado interno ocupa 80 B por punto de Gauss en vez de 462. Un material nuevo declara `STATE_SCHEMA` (una línea) y, si quiere acelerarse, un adaptador de diez líneas sobre su propio return mapping; un elemento nuevo escribe su cinemática compilada con las mismas funciones que su `compute_element_state`. Diferidos: los elementos estructurales 1D y los adaptadores sin asignaciones de Drucker-Prager y daño.

8.15 ADR 0015 — Corrector de Newton compartido

Fecha: 22 de septiembre de 2026. **Estado**: aceptado; los cinco solvers migrados el mismo día.

Contexto. Los cinco solvers iterativos escribían el mismo bucle de corrección cinco veces (ensamblaje, residuo, criterio dual, backend con degradación a LU, Newton modificado, line search, historial de divergencia, commit), con dos variantes del line search y tres gestiones del backend. La auditoría de esa mañana tuvo que aplicar la misma corrección semántica en los cinco, y todo lo previsto (cargas con historia, acoplamiento, backend multihilo) los tocaría de nuevo.

Decisión. Un solo `NewtonCorrector` posee el bucle y lo que persiste entre pasos (backend, degradación a LU, factor congelado); cada solver construye por paso un `NewtonProblem` —ensamblar, residuo y su norma, escalas del criterio, corrección con el `solve` del corrector, actualización, commit— y conserva su control de paso. El iterado es opaco (`U`; `(U, λ, ΔU)`; `(u, u̇, ü, ü_libre)`). La restricción del arc-length es un método del problema que la variante por disipación sobreescribe (después también `IndirectDisplacementSolver`; desde el 2026-09-29 la variante por disipación vive en el módulo de usuario `dissipation_control` y el problema se exporta como `solidum.math.solvers.ArcProblem`). `CorrectionAborted` permite abandonar un paso sin marcarlo como tangente singular.

Consecuencia. Ningún resultado cambia (las 1 447 pruebas de entonces, sin tocar un valor esperado). Un solver iterativo nuevo escribe su problema y su control de paso; un cambio del bucle, del backend o de la globalización se hace una vez. Retirados los ayudantes privados duplicados; `solver.corrector` es la superficie de instrumentación.

8.16 ADR 0016 — Acoplamiento termomecánico (estudio de viabilidad)

Fecha: 22 de septiembre de 2026. **Estado**: aceptado; sin código.

Contexto. El acoplamiento termomecánico era la decisión pendiente que el problema térmico (Etapa 8) había dejado abierta: acoplamiento débil o monolítico, y cómo entra en el contrato de material.

Decisión. Una verificación previa sobre el código mostró que la infraestructura ya admite nodos

con desplazamiento y temperatura a la vez: numeración, ensamblaje en una sola matriz, condiciones mixtas, masa y capacidad por el mismo operador y resolución estática completa. Lo que falta es sólo la física de los bloques fuera de la diagonal. Se decide: **acoplamiento débil unidireccional primero**, con el monolítico como extensión aditiva; una **deformación propia genérica** ε_0 con σ `= C:(`ε `-` ε_0`)`, de la que la térmica es un caso particular (la higroscópica, formalmente idéntica, domina en madera y bambú); ε_0 almacenada **por punto de Gauss**; y sin familia de elementos acoplados.

Consecuencia. El acoplamiento deja de ser una decisión pendiente y pasa a ser una etapa de trabajo acotada que no necesita solver nuevo.

8.17 ADR 0017 — Backend algebraico multihilo

Fecha: 22 de septiembre de 2026. **Estado**: aceptado.

Contexto. Medido con un perfilado de extremo a extremo, cerrado el ADR 0014 la factorización del sistema se llevaba el 97 % del tiempo de un análisis y el ensamblaje el 1 %. La causa es el relleno de los factores, que crece con la talla.

Decisión. Un backend opcional sobre Intel MKL Pardiso (factorización multihilo con reordenamiento por disección anidada), con el patrón de Cholesky: si la dependencia falta, el sistema funciona igual con SuperLU. Ocupa el lugar de LU en la regla automática (Cholesky, luego Pardiso, luego LU). Un único manejador de la MKL por proceso, y una factorización invalidada falla de forma ruidosa en lugar de devolver la solución de otro sistema.

Consecuencia. Aceleraciones de $\times 5$ a $\times 56$ según la talla, crecientes con ella, con la misma solución a precisión de máquina.

8.18 ADR 0018 — Solver iterativo

Fecha: 23 de septiembre de 2026. **Estado**: aceptado; la tolerancia (10^{-10} sobre el residuo verdadero, fija y no configurable desde el YAML), validada por el usuario.

Contexto. Los programas comerciales ofrecen un solver directo por defecto y uno iterativo para modelos grandes. La ventaja del iterativo no es la velocidad sino la memoria: la del directo crece más deprisa que el modelo.

Decisión. Un backend iterativo **sólo a petición** (`linear_algebra: iterative`): gradiente conjugado para matrices simétricas definidas positivas, MINRES para simétricas indefinidas (el cambio es automático al detectar curvatura negativa) y solver directo para las no simétricas. El precondicionador es un multimalla algebraico que recibe los **modos de cuerpo rígido** del modelo, derivados del nombre de los grados de libertad; sin ellos rinde peor que no precondicionar. Tolerancia sobre el residuo verdadero cinco órdenes por debajo de la del Newton, de modo que el método de Newton no cambia; si no se alcanza, el análisis se detiene con la causa.

Consecuencia. Número de iteraciones casi independiente de la talla y memoria proporcional al modelo; por encima de unos 10^5 grados de libertad gana también en tiempo.

8.19 ADR 0019 — Red de seguridad del análisis estático

Fecha: 23 de septiembre de 2026. **Estado**: aceptado.

Contexto. El usuario de Solidum no elige el solver algebraico ni tiene por qué conocer sus fallos. Medido: con un modelo mal apoyado, los solvers directos devolvían en silencio desplazamientos de miles de kilómetros.

Decisión. Tres capas en el análisis estático: antes de resolver, los **mecanismos rígidos** se detectan a partir de los modos de cuerpo rígido que las restricciones dejan libres y se describen en términos del modelo (traslación, giro alrededor de un eje por un punto, campo escalar sin valor prescrito); después de factorizar y de resolver, en el análisis lineal, se verifican la ausencia de **mecanismos internos** (desde el 2026-10-20, por la energía del modo de menor energía y no por el tamaño de un pivote, con los elementos que se mueven nombrados) y que el solver **resolvió el sistema** (desde el 2026-10-20, por el error hacia atrás normado, que no depende de las unidades ni del contraste de rigideces). Dentro de un Newton no se rechaza nada —cerca de un punto límite resolver un sistema casi singular es legítimo—, sólo se mejora el diagnóstico.

Consecuencia. Un modelo mal planteado se detiene con una explicación en vez de devolver resultados absurdos, sin falsos positivos en toda la batería de pruebas.

Capa 4 (27 de octubre de 2026, decisión del usuario). Un sistema bien resuelto y no singular puede haber perdido cifras si está mal condicionado, y el residuo no lo delata: un voladizo de 10 000 elementos daba la flecha errónea en un 51 % sin aviso. El análisis lineal estima ahora el número de condición de la rigidez escalada por la diagonal con la factorización ya hecha (Higham y Tisseur; unas seis resoluciones) y avisa de las cifras perdidas por encima de 10^{12}, cuando la solución baja a unas 5 cifras; el valor queda en el resultado.

8.20 ADR 0020 — Módulos de usuario: las formulaciones no estándar fuera del programa principal

Fecha: 24 de septiembre de 2026. **Estado**: aceptado.

Contexto. Al decidir completar su formulación de discontinuidades interiores, el usuario fijó que un elemento no estándar no debe contaminar el programa principal, como en FEAP, que ofrece formulaciones estándar y deja al usuario añadir las suyas sin tocarlo. La discontinuidad embebida (ADR 0010) sí lo contaminaba: su familia cohesiva y su estado vivían en `core/`, su registro en `registry.py`, el lector YAML la trataba como caso especial (una sección y una clave propias) y la nombraban también la raíz del paquete, el descubrimiento automático, la herramienta de specs y el manual de referencia. La familia térmica sufría la misma rigidez: el lector repetía tres veces el mismo bloque, y los elementos térmicos resolvían su material con una heurística y no podían salir de gmsh.

Decisión. Dos términos, los de FEAP: **programa principal** (Solidum estándar, todo `solidum/` salvo `solidum/user/`) y **módulo de usuario** (una formulación no estándar en `solidum/user/<nombre>/`, análoga a los elementos y materiales de usuario de FEAP). La embebida pasa íntegra al módulo `discontinuities`. La carga es explícita, nunca automática: `user_modules: [discontinuities]` en el YAML o `solidum.load_user_module("discontinuities")` en Python; `import solidum` no carga nada de `solidum/user/`. El programa principal ofrece a los módulos piezas genéricas, cada una justificada por casos reales ajenos a la embebida: un `Registry` público con metadatos de familia (`YAML_SECTION`, `YAML_LABEL`, `SPEC_KIND`) y `Registry.families()`; un lector YAML con un solo bucle sobre las familias, en el que una sección de primer nivel desconocida es un error; `Element.REFERENCE_KWARGS` para las referencias de un elemento a otras familias; la carga de módulos; el gancho `prepare_step`, documentado como contrato genérico e idempotente; la cinemática pública (`compute_kinematics_tri3`, `compute_integrands`); la receta de elemento

por grupo físico de gmsh; y la exclusión del camino por lotes de los elementos con estado propio. Quedan fuera a propósito, sin un segundo caso real, el servicio de seguimiento de trayectoria, un bus de eventos y la carga automática por *entry points*. Se descartaron un paquete Python aparte (dos instalaciones en cada PC y un nombre que sugiere otro programa), un repositorio aparte y la carga automática al importar Solidum (el YAML dejaría de decir qué formulación necesita).

Consecuencia. El programa principal no conoce la embebida, y un test de pureza (`tests/test_main_program_purity.py`) lo vigila: ningún archivo del programa principal nombra al módulo, y tras `import solidum` no hay ningún módulo de usuario cargado ni ninguna de sus clases registrada. Un módulo de usuario nuevo no toca el lector YAML, gmsh ni la herramienta de specs; la familia térmica gana validación de parámetros y referencias y puede salir de gmsh. Ningún resultado cambia. En contra: `from solidum import CST_Embedded2D` deja de funcionar, sin transición (el import es `from solidum.user.discontinuities import CST_Embedded2D`), los YAML de la embebida necesitan `user_modules`, y las piezas genéricas pasan a ser contrato estable del programa principal.

Revisión del 2026-09-29. El criterio de admisión de Reglas.md §1 (textos de referencia y código general establecido) fija la frontera, y cada spec del programa principal lo declara en su bloque `admission`. Por él sale del programa principal un segundo módulo, `dissipation_control`, con `DissipationArcLengthSolver`; el programa principal gana una sola pieza genérica, el nombre público `solidum.math.solvers.ArcProblem` para el problema por paso del arco, que ya extendía `IndirectDisplacementSolver`, y el lector YAML da la pista de `user_modules` también ante un tipo de solver desconocido. La frontera se endurece en los dos sentidos: el test de pureza pasa a tres controles (estructural por `ast`, léxico con los nombres derivados de lo que registra cada módulo, sin lista a mano, y en ejecución), `load_user_module` es atómica y sólo admite añadir entradas a los registros, y `tests/test_user_module_public_imports.py` exige a los módulos el contrato público: la cinemática se importa de `solidum.elements.solid_2d` (`compute_kinematics_tri3`, `compute_integrands`, sin guion bajo) y `YAML_LABEL` es la etiqueta de todo registro, en lugar del atributo privado `_kind`, retirado.

Actualización del 10 de noviembre de 2026. La familia cohesiva del módulo `discontinuities` volvió al programa principal (ADR 0033), como preveía Reglas.md §4, al entrar el primer elemento de interfaz del programa principal (la adherencia barra–sólido). El módulo conserva el elemento, el estado de la discontinuidad y el seguimiento de grieta.

8.21 ADR 0021 — Secciones de marco: familia de secciones y sección por fibras

Fecha: 29 de septiembre de 2026. **Estado**: aceptado.

Contexto. La auditoría de pureza verificó un defecto físico silencioso en los cuatro marcos: evaluaban el material con la deformación axial y construían toda la rigidez local con la E_t resultante, de modo que con fluencia axial el flector se anulaba aunque la sección estuviera curvada y con ablandamiento cambiaba de signo. El usuario decidió la física, una cuestión por vez: sección por fibras, elemento en desplazamientos, etapa 1 con los tres marcos Euler-Bernoulli (Timoshenko en una etapa 2), Gauss-Legendre con 3 puntos por defecto y mínimo 2, torsión elástica desacoplada con `GJ` de la sección y sin regularización del ablandamiento en esta etapa.

Decisión. Una familia nueva, `SectionRegistry` (sección YAML `sections`, `kind: section`), con la base abstracta `Section`, **sin estado**: el estado de cada fibra en cada punto de Gauss vive en el elemento. `FiberSection` es la primera sección. Los marcos Euler-Bernoulli aceptan

exactamente uno de dos caminos: `material` + propiedades geométricas (forma cerrada, sin cambio de resultados, sólo con materiales `IS_LINEAR_ELASTIC = True`) o `section` + `n_ip` (elemento en desplazamientos integrado en Gauss-Legendre); la guarda de material elástico lineal protege también a `Frame2DTimoshenko`. Piezas genéricas nuevas, cada una con casos reales: `Material.IS_LINEAR_ELASTIC`, `Element.materials()` para las comprobaciones agregadas de masa, peso propio y simetría, la referencia `section` en `REFERENCE_KWARGS` (la primera con nombre distinto de `material`) y el gancho `from_yaml(data, resolve)` del lector YAML para objetos de familia con referencias anidadas. Se descartaron elementos nuevos por cada marco (duplicarían cinemática, catálogo y tests para cambiar sólo la ley de sección), la sección como material (contratos distintos) y la sección integrada dentro de cada marco (no se podría declarar una vez ni compartir).

Consecuencia. Los marcos dejan de dar flectores incorrectos en silencio y ganan plasticidad distribuida en la sección con cualquier material 1D, secciones mixtas y flexión biaxial. En contra: un modelo con marcos y material inelástico que antes corría ahora falla al construir, y el ablandamiento en fibras depende de la malla hasta que exista la regularización genérica.

8.22 ADR 0022 — Cargas variables, mallas estructuradas y verificación por soluciones manufacturadas

Fecha: 27 de octubre de 2026. **Estado**: aceptado; el plan en cuatro fases y su criterio de aceptación (orden observado $\geq$ teórico $-$ 0.1), aprobados por el usuario.

Contexto. La validación de Solidum se apoyaba en soluciones analíticas y benchmarks publicados, que cubren casos simples y no detectan un error que sólo afecte al orden de convergencia. La carga de borde de los elementos cuadráticos se repartía con pesos fijos sobre la cuerda (deuda #13) y no había presión.

Decisión. Las cargas distribuidas de todos los elementos (sólidos, térmicos, armaduras y marcos) aceptan una constante o un campo de la posición (`b(x)`, `t(x, n)` con la normal exterior, `Q(x)`, $\bar{\mathbf{q}}$`(x, n)`), integrado sobre la geometría real. Un contrato declarativo, `REFERENCE_CELL` y `REFERENCE_NODES` (y, en los elementos de línea, su campo interior y las rigideces de la sección), permite un generador de mallas estructuradas (`solidum.mesh`) y la verificación por soluciones manufacturadas con SymPy (`solidum.verification`): se inventa una solución, se derivan las cargas que la hacen exacta y se mide el orden de convergencia al refinar.

Consecuencia. Los 17 elementos lineales del programa principal dan el orden teórico; en los de línea, además, los valores nodales son exactos. Los barridos recorren el registro, así que un elemento nuevo queda verificado sin escribir pruebas nuevas.

8.23 ADR 0023 — Cargas distribuidas y expresiones en el YAML

Fecha: 27 de octubre de 2026. **Estado**: aceptado; el convenio de signo de la presión ($p > 0$ comprime), decisión del usuario.

Contexto. Las cargas variables del ADR 0022 sólo estaban en el API de Python. El YAML no admitía tracciones de borde ni presión (deudas #23 y #24), y los grupos de gmsh se guardaban como conjuntos de nodos, de los que no se deduce con seguridad qué bordes cargar.

Decisión. Bloques `tractions` (por elemento) y `tractions_by_group` (por grupo físico de gmsh, con sus celdas de contorno exactas), con tracción `t` o presión `pressure` ($\mathbf{t} = -p\,\mathbf{n}$); `boundary_flux_`

`by_group` en el térmico. Todo valor de carga puede ser una expresión de la posición (y de la normal en las de contorno), compilada por un intérprete propio con lista blanca, sin ejecutar el texto como código. Las cargas se resuelven y validan al leer el archivo.

Consecuencia. El YAML alcanza al API de Python en cargas distribuidas; un archivo ajeno no puede ejecutar código, y las erratas se detectan antes de resolver.

8.24 ADR 0024 — Masas concentradas en nudos y análisis modal con grados de libertad sin masa

Fecha: 9 de noviembre de 2026. **Estado**: aceptado; las masas rotacionales en los nudos, decisión del usuario.

Contexto. La única fuente de masa era la densidad del material (ADR 0008), pero en un edificio la masa de cada nivel viene de la losa, los acabados y la carga viva, y la práctica estándar la concentra en los nudos. Con masas sólo en algunos nudos, o elementos de densidad cero, la matriz de masa es semidefinida y el análisis modal fallaba: ARPACK lanzaba un error sin causa y el camino denso exigía masa definida.

Decisión. La masa de nudo es un dato del dominio (`Domain.add_nodal_mass`, bloques YAML `nodal_masses`, `nodal_masses_by_group` y `nodal_masses_by_coord`), por grado de libertad mecánico, traslacional o rotacional. Se suma a la diagonal de la matriz de masa con los dos esquemas, sobre el patrón ya cacheado, así que la ven todos los consumidores de la masa. Es sólo inercia: `gravity` no genera su peso (decisión del usuario, como el `mass` de OpenSees: la masa sísmica y la carga de gravedad salen de combinaciones distintas), y el peso de los pisos se declara como carga. En el análisis modal, los grados de libertad sin masa se condensan estáticamente de forma exacta ($\boldsymbol{\phi}_0 = -\mathbf{K}_{00}^{-1}\mathbf{K}_{0m}\boldsymbol{\phi}_m$), sin formar la matriz condensada en los modelos grandes: el operador de desplazamiento e inversión del problema condensado es el bloque de $(\mathbf{K} - \sigma\mathbf{M})^{-1}$ del modelo completo. Si el problema no tiene solución, `MasslessDofsError` nombra los grados de libertad. Se descartaron un elemento de masa puntual (material, rigidez nula y catálogo para una entrada diagonal), la masa ficticia pequeña (frecuencias espurias) y el problema inverso con $\mathbf{K}$ definida (falla en estructuras libres).

Consecuencia. Análisis modal y espectral de edificios con masas de piso, con frecuencias exactas para el modelo discreto; los modelos con masa en todos los grados de libertad no cambian.

Ampliación (9 de noviembre de 2026, §6 bis, decisión del usuario). Los transitorios implícitos (Newmark, HHT-α y sus variantes de Newton) admiten grados de libertad sin masa: la matriz efectiva $\mathbf{M} + \gamma\Delta t\,\mathbf{C} + \beta\Delta t^2\mathbf{K}$ es regular, y la aceleración inicial se toma consistente, $\ddot{\mathbf{u}}_0 = -\mathbf{K}_{00}^{-1}\mathbf{K}_{0m}\ddot{\mathbf{u}}_m$ en los grados de libertad sin masa. Un mecanismo sin masa da `MasslessDofsError`.

8.25 ADR 0025 — Terminación de los solvers estáticos incrementales y registro por omisión

Fecha: 9 de noviembre de 2026. **Estado**: aceptado; amplía los ADR 0005 y 0011.

Contexto. Quien usaba los solvers de trayectoria desde Python no sabía, sin leer el registro, si el análisis terminó ni por qué se detuvo. Un trazado que agotaba `max_steps` regresaba con normalidad. El fracaso del arco era un `RuntimeError` genérico con el diagnóstico sólo en el registro, y un solver reutilizado conservaba el estado de la corrida anterior. El control indirecto no podía detenerse en un

desplazamiento. El registro escribía las iteraciones en `INFO`, registraba como `ERROR` eventos que el solver recuperaba y producía trazas con una consola en cp1252.

Decisión. Atributo `termination` (`max_lambda`, `control_target`, `max_steps` o `diverged`) con `completed`, `lambda_final`, `steps_done`, `n_bisections` y `U_final`. Se fijan también tras una excepción, se reinician en cada `solve` y llegan a `SolveResult.termination`. El fracaso es una `SolverDivergedError` tipada: `StepConstraintError` si la restricción del paso no tiene solución. `IndirectDisplacementSolver` admite `control_target`: el último paso se recorta y la llegada es exacta. Ampliado el mismo día al arco cilíndrico: `ArcLengthSolver(..., control=[...], control_target=c)`, donde `control` sólo mide; el paso que cruza el objetivo se repite con la restricción del control indirecto y la llegada también es exacta. La base ofrece los puntos de extensión `_target_reached`, `_limit_step` y `_step_summary`. Registro: una línea `INFO` por paso, iteraciones en `DEBUG`, lo recuperado en `WARNING` con su motivo y `ERROR` sólo al detenerse; un manejador de consola adapta los mensajes a la codificación del flujo.

Consecuencia. Cómo terminó un análisis se lee en un atributo y un fracaso llega con su causa. Un análisis de empuje lateral se detiene en el desplazamiento pedido. Ningún resultado numérico cambia.

Ampliación (9 de noviembre de 2026). Una auditoría de generalidad del mismo día extendió el contrato a los análisis en el tiempo: `termination = "t_end"` o `"diverged"`, `completed`, `steps_done`, `t_final`, `U_final` y `partial_result`; una línea `ERROR` y una excepción tipada al detenerse (`TimeIntegrationInstabilityError` para la integración inestable), con el historial parcial. El θ-método dejó de devolver un resultado con columnas en cero al divergir. `control_target` toma el sentido del objetivo, así que una etapa también descarga.

8.26 ADR 0026 — Etapas de carga en los solvers estáticos

Fecha: 9 de noviembre de 2026. **Estado**: aceptado.

Contexto. Los solvers estáticos arrancaban siempre en `U = 0` y su residuo no tenía término constante. No se podía aplicar la gravedad y empujar después, y una segunda corrida sobre un modelo plastificado empezaba en `U = 0` con el estado plástico de la primera.

Decisión. `solve(F_ref, u0=…, constant_load=…)` en `NonlinearSolver` y la familia del arco, con residuo $\mathbf{F}_c + \lambda\mathbf{F}_{\text{ref}} - \mathbf{F}_{\text{int}}$ y $\lambda = 0$ al empezar. Los valores impuestos crecen desde los de partida, $\mathbf{g}(\lambda) = \mathbf{g}_0 + \lambda(\mathbf{g} - \mathbf{g}_0)$. Cada etapa comprueba su equilibrio inicial, y un modelo con historia que empieza en cero lo avisa. `solidum.run(..., previous=resultado)` continúa la etapa anterior con su carga aplicada como constante y sus cargas equivalentes por elemento; el resultado separa `F_constant` de `F_variable`. `DissipationArcLengthSolver` rechaza las etapas: su restricción de disipación es física por decidir.

Consecuencia. Gravedad y empuje, P-Δ con la carga vertical y continuación de un análisis, verificados contra la viga-columna con acortamiento, la interacción N–M y la superposición elástica. Las etapas desde el YAML se añadieron el mismo día: bloque `stages`, una etapa por entrada con sus cargas y su solver, y `run_yaml` devuelve un `StagedResult` (deuda #36 saldada). Las dos cargas distribuidas en una misma etapa (deuda #35) las resolvió el ADR 0027.

8.27 ADR 0027 — Cargas que llevan sus cargas equivalentes

Fecha: 9 de noviembre de 2026. **Estado**: aceptado; sustituye el §5 del ADR 0026.

Contexto. Las fuerzas internas de extremo con carga distribuida son $\mathbf{F}_{\text{int}} - \mathbf{f}_{\text{eq}}$, y el ensamblador recordaba sólo las cargas equivalentes de la última carga de cuerpo ensamblada: dos cargas de cuerpo en una corrida, dos casos de carga con el mismo ensamblador y el historial de los transitorios daban fuerzas internas erróneas sin aviso (deuda #35).

Decisión. La carga lleva su propia información: `LoadVector`, un `np.ndarray` con las cargas equivalentes por elemento de su parte distribuida, que la aritmética de las cargas combina linealmente (`F_a ± F_b`, `a·F`, `F/a`; cualquier otra operación da un `np.ndarray` simple). Se guardan concatenadas con una disposición compartida por las cargas del mismo ensamblador, así que combinar es una operación vectorizada. Un `np.ndarray` simple que las perdió recurre al registro anterior y avisa si es ambiguo.

Consecuencia. La deuda #35 queda cerrada en estático, por etapas y en el tiempo, sin cambiar la API de las cargas.

8.28 ADR 0028 — Longitud característica del punto de integración

Fecha: 2026-11-09 · **Estado**: aceptado.

Un material con ablandamiento regularizado por energía de fractura necesita la longitud `h` de cada punto de integración, que sólo conoce el elemento. `h` viaja en el estado del punto: el material declara si la usa (`uses_characteristic_length`) y construye el estado inicial con `initial_point_state(h)`, donde la valida; el elemento, al terminar de construirse, siembra una longitud por punto con `_seed_characteristic_lengths` (`L` en las barras, `w_g·L` en los marcos con sección, que la reparte a sus fibras con `initial_states(characteristic_length=h)`). Un `h` inadmisible se rechaza al construir el elemento. Se descartó pasar `h` en cada `compute_state` (más de treinta llamadas y validación tardía) y guardarlo en el material (lo comparten puntos de longitudes distintas). Primer consumidor: `ManderConcrete1D`; regularizar los materiales de daño de los sólidos sólo exigirá que sus elementos siembren las longitudes.

8.29 ADR 0029 — Continuidad de los análisis en el tiempo

Fecha: 2026-11-09 · **Estado**: aceptado.

Extiende las etapas del ADR 0026 a los análisis en el tiempo. `continue_from(previous)`, común a los seis integradores (mezcla `TransientTerminationMixin`), parte de un estático (sus desplazamientos o temperaturas, en reposo, con su carga constante y sus cargas equivalentes; aviso si el estado no la equilibra) o de un transitorio del mismo tipo (su estado final en su instante final, en tiempo absoluto). Al continuar un transitorio se hereda la aceleración final, no la del equilibrio instantáneo, que con HHT-α no coincide: parar y continuar es igual que no parar. Los resultados guardan `F_constant`. Pendiente de una decisión física: la tangente con que se forma el amortiguamiento de Rayleigh al continuar.

8.30 ADR 0030 — Reacciones de los apoyos en los análisis dinámicos

Fecha: 2026-11-09 · **Estado**: aceptado.

Extiende el ADR 0004 §3 a los análisis dinámicos. Por decisión del usuario, la reacción es la ecuación de movimiento en las filas de apoyo, `R = F_int + M·ü + C·u̇ − F`, y la parte de amortiguamiento `C·u̇` se reporta aparte (OpenSees deja elegir entre ambas con `-rayleigh`). `TransientTrace` la registra en cada paso guardado, común a los cinco integradores mecánicos; el armónico la da en amplitudes complejas y el espectral por modo (`(K − ω_n²M)·u_n`), combinada con `combine_modal`. `total_reaction(dof)` da el cortante basal; en el espectral suma antes de combinar. Se guardan sólo los grados de libertad de apoyo, con el mismo reparto entre apoyos y uniones que el estático.

8.31 ADR 0031 — Análisis lineales sobre el estado de una etapa

Fecha: 2026-11-09 · **Estado**: aceptado.

Por decisión del usuario, el modal, el armónico y el espectral que continúan una etapa (`continue_from`, `previous=`, etapas `stages`) linealizan alrededor de su estado con la rigidez tangente que dan los elementos (P-Δ de los corrotacionales, descarga elástica de la plasticidad, rigidez dañada), sin cambiarlo. La mezcla `LinearizedStateMixin` lo comparte. Los autovalores negativos dejan de recortarse a cero: `ModalResult.eigenvalues` con signo y `unstable_modes`. El `LinearSolver` como etapa es una perturbación lineal estática que tampoco cambia el estado (`SolveResult.perturbation`): casos superponibles sobre la etapa, con reacciones y fuerzas internas lineales (`Element.internal_forces_from_nodal`).

8.32 ADR 0032 — Restricciones generadas por geometría y refuerzo embebido

Fecha: 2026-11-10 · **Estado**: aceptado.

Un generador de restricciones (`ConstraintRegistry`, tipo de spec `constraint`, bloque YAML `constraints`) crea sobre un dominio ya mallado los nodos, los elementos y las MPC (ADR 0004) que su geometría pide. La localización de puntos (`solidum/mesh/point_location.py`) invierte el mapeo isoparamétrico por Newton con lo que todo sólido declara (`REFERENCE_CELL`, funciones de forma), sin código por tipo de elemento. El primer generador es el refuerzo embebido (`EmbeddedReinforcement`): la barra, con su propia malla cortada en los bordes de los anfitriones, queda atada al campo interpolado del sólido. Sus MPC son de unión, y su fuerza es la adherencia (`EmbeddedBar.bond_forces`). `Domain.generated_nodes` aparta esos nodos de los selectores por coordenada del YAML.

Ampliaciones (10 de noviembre de 2026). (1) La barra admite elementos de tres nodos (`Truss2D3`, `Truss3D3`): el número de nodos se lee de `REFERENCE_NODES` y cada tramo recibe su nodo intermedio, atado como los demás. (2) Adherencia-deslizamiento: con `bond` el refuerzo no ata los nodos de la barra, sino que genera, uno por tramo, elementos de interfaz `BondSlipInterface` que unen la barra con su anfitrión y llevan una ley de interfaz de línea (ADR 0033). Esos elementos no se registran (los crea el generador, como crea las MPC), no tienen masa ni celda VTK (`VTK_EXPORT = False`, pieza genérica del exportador), y el lector YAML admite en una familia las claves que traduce su `from_yaml` (`YAML_KEYS`).

8.33 ADR 0033 — Materiales de interfaz en el programa principal

Fecha: 2026-11-10 · **Estado**: aceptado.

Una sola familia de leyes tracción–desplazamiento relativo, `InterfaceMaterial` (`InterfaceMaterialReg` tipo de spec `interface_material`, sección YAML `interface_materials`), para las dos geometrías de interfaz: superficie (grieta, junta: `(n, s)` / `(n, s1, s2)`) y línea (barra embebida: `(n, s)` / `(n1, n2, s)`), normales primero. El paso a ejes globales y el ancho de cada componente son del elemento. La familia cohesiva del módulo `discontinuities` pasa a ella sin cambio de física, como preveía Reglas §4, porque el programa principal tiene ahora un elemento de interfaz propio (la adherencia barra–sólido). La sección YAML `cohesive_materials` pasa a llamarse `interface_materials`, sin alias. Leyes de la familia: `CohesiveDamageIsotropic` (modo I) y `CohesiveDamageMixedMode` (modo mixto con apertura efectiva de Camacho–Ortiz, energía independiente del modo, ablandamiento lineal, exponencial o bilineal del fib Model Code 2010), de superficie; `BondSlipModelCode` y `BondSlipMultilinear` (adherencia-deslizamiento), de línea. Las dos leyes cohesivas comparten la envolvente de ablandamiento.

8.34 Evolución de esta lista

Cada decisión de arquitectura de gran calado — refactor transversal, subsistema nuevo, ruptura de contratos — produce un ADR adicional. Los siguientes ADR se prevén en las fases de diseño futuras:

- El problema térmico se incorporó en la Etapa 8 **sin ADR propio**, a diferencia de lo previsto aquí. La razón es que no rompió contrato alguno ni exigió la generalización del concepto de grado de libertad que se anticipaba: la infraestructura resultó ya agnóstica al campo físico, y la familia térmica reutilizó el patrón de familia paralela con registro propio que el ADR 0010 había establecido para los materiales cohesivos. Las convenciones específicas —signo del flujo de frontera saliente, ausencia de notación de Voigt en el problema escalar— se registraron en `Reglas.md` §5.
- El acoplamiento termomecánico, que sí obliga a decidir sobre el contrato de material, tiene ya su ADR (0016): estrategia decidida, implementación pendiente.

Capítulo 9

Evolución de la arquitectura

Este capítulo presenta la trayectoria prevista de la arquitectura de Solidum FEM. Su función es preservar la imagen de la arquitectura del programa a medida que crece: cada fase incorpora un nuevo tipo de análisis con sus contratos propios, sus variables de estado y, eventualmente, capas o subsistemas adicionales. La diferencia con los pendientes del capítulo 4 es de granularidad: aquellos enumeran componentes individuales; este capítulo describe los **estados sucesivos de la arquitectura** del programa, cada uno con su mapa de capas correspondiente.

Las fases aún no implementadas están marcadas como pendientes y se materializarán mediante los Architecture Decision Records (ADR) anunciados en el capítulo 8. La estructura de cada fase se redefinirá en su ADR de apertura; lo aquí descrito constituye una hipótesis razonada de partida.

9.1 Fase 1 — Estática lineal y no lineal

Estado: implementado.

Capas activas: las seis capas descritas en el capítulo 2 (entrada, inicialización, interpretación del caso, dominio, capa numérica, salida).

Tipos de análisis: estática lineal (`LinearSolver`) y estática no lineal incremental (`NonlinearSolver`, `ArcLengthSolver`).

Variables de estado: desplazamientos `U`. Variables internas del material (deformaciones plásticas, daño, historia) gestionadas mediante `ElementState` con semántica *trial* / comprometido.

Matrices globales: matriz de rigidez `K` (única).

Contrato de elemento: `K_local`, `f_internal`, `internal_forces(U)`. La matriz `B` y el residuo se calculan en cada iteración.

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, solvers] - [
    Salida]
```

9.2 Fase 2 — Análisis modal

Estado: implementado (ADR 0009 fase 1, sesión del 2026-05-13).

El análisis modal resuelve el problema generalizado de valores y vectores característicos $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$. Es un problema algebraico cuyas soluciones admiten **interpretación** como frecuencias

propias y formas de vibración libre no amortiguada, pero el análisis en sí es estático: no hay paso de tiempo ni evolución temporal. Se acometió antes que la dinámica transitoria porque solo requiere la matriz de masa nueva, aísla esa pieza, y deja la infraestructura preparada para Newmark.

Cambios respecto a la fase 1:

- Aparece la matriz de masa `M`, ensamblada con la misma infraestructura que `K` (mismo patrón de sparsity, misma topología COO cacheada).
- Cada elemento implementa `compute_mass_matrix(lumping)` — masa consistente translacional invariante en barras y cables, axial + Hermitiana cúbica + inercia rotacional propia de sección en frames 2D/3D, integral $\int \rho \cdot \mathtt{N}^\top \mathtt{N} \cdot \mathtt{t}\ \mathtt{dA}$ en sólidos 2D (Tri6 usa cuadratura específica `tri_6` por subintegración con `tri_3`).
- Nuevo subsistema en la capa algebraica: `EigenSolver` que envuelve ARPACK Lanczos con shift-invert (`scipy.sparse.linalg.eigsh`).
- Nueva clase de solver: `ModalSolver`, registrado en `SolverRegistry` (no se fragmenta el registro). Reutiliza Dirichlet por eliminación directa con un `reduce_pair(K, M)` que aplica `T` a ambas matrices simultáneamente.
- Nueva salida especializada: `ModalResult` (frozen) con `frequencies_rad/hz`, `periods`, `modes` M-ortonormales, y un método `free_vibration(M, u0, u0_dot, t)` que reconstruye analíticamente la respuesta libre sin amortiguamiento por superposición modal — sin necesidad de integrador temporal.
- `Material.density` (ADR 0008) ya disponía de la única propiedad física nueva necesaria.

Topología efectiva:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, M, eigensolver]
    - [Salida: ModalResult]
```

Riesgo de arquitectura

Bajo. El análisis modal añade un nuevo tipo de problema algebraico (autovalor generalizado en lugar de sistema lineal) y una nueva familia de resultados, sin modificar contratos existentes ni la infraestructura de Dirichlet.

9.3 Fase 3 — Dinámica estructural transitoria lineal

Estado: implementado (ADR 0009 fase 3, sesión del 2026-05-13).

Integración temporal directa de la ecuación de movimiento semidiscreta $\mathtt{M} \cdot \ddot{\mathtt{u}} + \mathtt{C} \cdot \dot{\mathtt{u}} + \mathtt{K} \cdot \mathtt{u} = \mathtt{F(t)}$ por el método de Newmark-β. Es el primer análisis con dependencia temporal real; aprovecha la matriz de masa introducida en la fase 2 y reutiliza las decisiones arquitecturales fijadas en el ADR 0009 (registro unificado, estado dinámico fuera de `Node`, amortiguamiento Rayleigh como entrada estándar).

Cambios respecto a la fase 2:

- Aparece la matriz de amortiguamiento `C = α·M + β·K` (Rayleigh proporcional). Los coeficientes pueden pasarse directos o calibrarse modalmente a partir de `(ξ₁, ω₁), (ξ₂, ω₂)` con `solidum.math.damping.rayleigh_from_modes`.
- Las variables de estado se amplían a `(u, u̇, ü)`. Coherente con el ADR 0009 §5, el estado dinámico **no contamina** la clase `Node`: vive en el nuevo `TransientResult` (frozen) como historiales `t_history`, `u_history`, `udot_history`, `uddot_history` indexados por paso temporal.
- Nuevo solver `NewmarkSolver` con esquema `(β, γ)` parametrizable (default 1/4, 1/2 — average acceleration, incondicionalmente estable, error `O(Δt²)`). Cargas dependientes del tiempo por callback Python (`F_func: t → F`).
- El subsistema algebraico recibe una matriz efectiva `A_eff = M + γΔt·C + βΔt²·K`. Con `Δt` constante y problema lineal, `A_eff` es invariante: se factoriza **una sola vez** al inicio (ADR 0003 — `FactorizedSolver`) y cada paso temporal se reduce a una sustitución triangular barata.
- Aceleración inicial calculada de forma consistente con la ecuación de movimiento: `M·ü₀ = F(0) − C·u̇₀ − K·u₀`.

Topología efectiva:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K, M, C, A_eff,
    solver] - [Salida: TransientResult]
                                                                              |
                                                                  [Integrador temporal
                                                                      Newmark]
```

Riesgo de arquitectura

Bajo. La decisión 5 del ADR 0009 (estado dinámico fuera de `Node`, en `TransientResult`) preserva la pureza topológica/geométrica de `Node` y evita que análisis estáticos paguen el coste de campos que no usan. La semántica *trial* / comprometido del estado interno de elementos (ADR sobre `ElementState`) sigue intacta porque la fase 3 es lineal: no hay variables internas que evolucionen.

Limitaciones declaradas en esta fase 3:

- Solo lineal: `K, M` constantes. No-linealidad (Newton dentro de Newmark) entregada en la fase 4.
- Apoyos Dirichlet constantes en el tiempo. Multi-support excitation sísmica diferida.
- `Δt` fijo; adaptativo diferido.
- Esquemas con amortiguamiento numérico controlado (HHT-α) entregados en 2026-05-18 como variante de la familia Newmark.

9.4 Fase 4 — Dinámica estructural transitoria no lineal

Estado: implementado (ADR 0009 fase 4, sesión del 2026-05-14).
Cambios respecto a la fase 3:

- Nueva clase `NewtonNewmarkSolver` como **variante** de `NewmarkSolver` (Reglas §4 — extensión documentada en spec corta, sin ADR nuevo porque reúsa todas las decisiones del ADR 0009): hereda predictores, correctores y reducción de Dirichlet; añade un bucle Newton-Raphson dentro de cada paso temporal.
- En cada paso, el residuo `R = F_ext(t_{n+1}) − F_int(u_{n+1}) − C·ü_{n+1} − M·ü_{n+1}` se anula iterativamente con jacobiano `J = M + γΔt·C + βΔt²·K_t`, donde `K_t = ∂F_int/∂u` es la rigidez tangente corriente. Convergencia dual del ADR 0007 calibrada al primer paso.
- Reaprovecha íntegramente la maquinaria del `NonlinearSolver` estático: `ConvergenceCriterion`, `freeze_tangent_after_iter` (Newton modificado, ADR 0003 fase 2), `commit_all_states()` tras converger cada paso temporal — el estado interno de los elementos recupera plenamente su semántica *trial* / comprometido también en dinámica.
- Amortiguamiento Rayleigh `C = α·M + β·K_0` **constante en el tiempo**, calibrado con la rigidez elástica de referencia `K_0` al inicio del análisis. Convención estándar (Abaqus, ANSYS, OpenSees); evita acoplamiento ad-hoc entre disipación viscosa y plástica.
- **Recuperación del caso lineal**: con materiales sin historia (o plásticos no plastificados), `K_t ≡ K_0` y el residuo se anula en una iteración. El resultado coincide a paridad de bits con `NewmarkSolver`, validado en tests.
- Despacho YAML automático: como `NewtonNewmarkSolver` es subclase de `NewmarkSolver`, `entry.run_yaml` lo detecta vía `isinstance` y enruta a `run_transient` sin tocar el dispatcher.

Topología efectiva: igual que la fase 3, con el bucle Newton añadido dentro del bucle temporal y la rigidez tangente reensamblada en cada iteración.

Riesgo de arquitectura

Bajo en la práctica: toda la infraestructura ya existía. El `NonlinearSolver` estático aportaba calibración de convergencia, fallback de positividad y Newton modificado; la fase 3 aportaba predictores/correctores Newmark y reducción de Dirichlet. La fase 4 es la composición exacta de ambas, formalizada como subclase para no duplicar código.

9.5 Fase 4 (extensión 2026-05-18) — Cierre completo del subsistema modal/dinámico/espectral

Estado: implementado (ADR 0009 fases 2, 5, 6, 7 + variante HHT-α + reglas C y D de auditoría aplicadas).

Las cuatro fases pendientes del ADR 0009 se cierran en una secuencia obligada (cada una precondición de la siguiente) sin reabrir las decisiones arquitecturales tomadas en el ADR original:

Mass lumping (fase 2) — `compute_mass_matrix(lumping="lumped")` operativo en todos los elementos. Helper centralizado `solidum.math.mass_lumping.lump_hrz` con esquema HRZ canónico (Hinton-Rock-Zienkiewicz 1976) para sólidos isoparamétricos y fórmula nodal directa para frames. Diagonalidad estricta en globales para todos los elementos excepto Frame3D con eje oblicuo (queda bloque-diagonal — limitación documentada, Cook-Malkus-Plesha §11.4).

HHT-α (variante) — `HHTSolver` y `NewtonHHTSolver` como variantes de la familia Newmark (Reglas §4 — spec corta sin ADR nuevo). Disipación numérica controlada por $\alpha \in$ `[-1/3, 0]` con (β, γ) auto-derivados canónicamente.

Central differences (fase 5) — `CentralDifferenceSolver` con leapfrog explícito Belytschko-Liu-Moran. Una sola clase cubre lineal y no lineal con parámetro `nonlinear`. Requiere masa lumped (`lumping="consistent"` rechazado con `ValueError`). Estabilidad condicional CFL con detección a posteriori de divergencia exponencial.

Harmonic (fase 6) — `HarmonicSolver` resuelve $(-\omega^2 M + i\omega C + K)\cdot\hat{u} = F$ con aritmética compleja para un barrido en ω. Factorización LU compleja por frecuencia. Nuevo `HarmonicResult` con métodos `.amplitude()` y `.phase()`. Cuarto valor del literal `PIPELINE_KIND` (ver regla C abajo).

Response spectrum (fase 7) — `ResponseSpectrumSolver` combina respuestas máximas modales bajo un espectro $S_d(\omega)$ o $S_a(\omega)$ por SRSS (default) o CQC (Der Kiureghian 1980, para modos cercanos). Devuelve `ResponseSpectrumResult` con respuesta envolvente, contribución modal individual, factores de participación, masas efectivas y método `.cumulative_effective_mass_ratio()` para verificación normativa (≥ 0.9).

Reglas C y D aplicadas durante esta extensión (auditoría arquitectural 2026-05-13):

- **Regla C** — `entry.py::run_yaml` despacha por atributo de clase `PIPELINE_KIND` $\in$ `{"static", "modal", "transient", "harmonic", "spectrum"}` en lugar de cadena de `isinstance`. Solvers no clásicos futuros no requieren tocar `entry.py`.
- **Regla D** — `solidum/math/modal_response.py` agrupa todos los cómputos sobre modos: `free_vibration` (movido desde `results.py`), `participation_factors`, `response_spectrum_srss`, `response_spectrum_cqc`, helpers de espectros (`spectrum_from_sa`, `spectrum_tabulated`). `ModalResult.free_vibration` queda como wrapper delgado preservando la API histórica. `results.py` vuelve a su propósito declarado: dataclasses inmutables, no algoritmos.

Topología efectiva tras la extensión:

```
[Entrada] - [Inicializacion] - [Interprete: PIPELINE_KIND] - [Dominio] - [Numerica: K,
    M, C, lump_hrz, modal_response] - [Salida]


                              [run, run_modal, run_transient, run_harmonic,
                                  run_response_spectrum] →       [Result type específico]
```

Riesgo de arquitectura

Bajo. Las cinco piezas son extensiones aditivas: nuevo módulo `mass_lumping.py`, nuevo módulo `modal_response.py`, cinco solvers nuevos en `solidum/math/solvers/`, dos `Result` nuevos (`HarmonicResult`, `ResponseSpectrumResult`). El despacho declarativo `PIPELINE_KIND` reemplaza la cadena de `isinstance` sin romper contratos — los solvers existentes heredan el nuevo atributo. La regla D centraliza algoritmos sin modificar la API pública de `ModalResult`.

9.6 Fase 5 — Problema térmico estacionario y transitorio

Estado: implementada en su núcleo de conducción pura (Etapa 8, agosto de 2026). Quedan fuera la convección, la radiación y toda forma de no linealidad térmica.

Cambios efectivamente introducidos:

- Un campo escalar nuevo, la temperatura `T`, con elementos que declaran `DOF_NAMES = ['T']`.
- Las matrices de conductividad y de capacidad calorífica, análogos térmicos de la rigidez y de la masa.
- Una familia de elementos térmicos paralela a la mecánica (`Quad4Thermal`, `Hex8Thermal`) sobre base común `_ThermalSolid`, y una familia de materiales térmicos con registro e interfaz propios.
- Un solver de integración temporal de primer orden, `ThetaMethodSolver`, con tipo de resultado específico.

Contraste entre lo previsto y lo ocurrido. Esta fase se había estimado de riesgo *medio* por ser «la primera que rompe la suposición implícita de que todos los DOF son mecánicos, presente en numerosos puntos del código». Se anticipaban tres consecuencias: un atributo `DOF_KIND` para clasificar cada grado de libertad, una posible escisión del contrato `Element` en ramas mecánica y térmica, y la adaptación del exportador de resultados.

Ninguna de las dos primeras resultó necesaria. La suposición que se temía no estaba en el código: el ensamblador, la imposición de restricciones por eliminación y el despacho algebraico operan sobre índices de grado de libertad y sobre propiedades de la matriz, sin consultar en ningún punto el significado físico del campo. Las condiciones de frontera y las cargas nodales se aplican por *nombre* de grado de libertad, de modo que `T` funcionó por el mismo mecanismo genérico que `ux`. El régimen estacionario, en consecuencia, no requirió solver alguno: el solver lineal existente resolvió el problema de conducción sin una sola línea de modificación.

La tercera predicción sí se confirmó al cerrar la etapa: **el exportador VTK escribía únicamente campos mecánicos** —desplazamientos y rotaciones— y no exportaba el campo de temperatura. Era la limitación práctica más visible del subsistema térmico; desde el 22 de septiembre de 2026 el exportador escribe el campo nodal `Temperature` cuando algún nodo tiene el grado de libertad `T`.

La lección de arquitectura que deja la fase es que el coste real de la extensión no se concentró donde la previsión lo situaba. Lo caro no fue generalizar la infraestructura —que ya era agnóstica al campo, aunque nadie lo hubiera comprobado— sino la física propia del problema: la elección de los esquemas por defecto contra el principio del máximo de la ecuación de difusión, y la constatación de que la ecuación de conducción, al ser de primer orden en el tiempo, no admite reutilizar la familia

de integradores de Newmark. La interfaz del material tampoco fue la anticipada `compute_flux_and_tangent(grad_T, state)`, sino `compute_flux(grad_T)` sin argumento de estado, porque la conducción de Fourier carece de variables internas con historia.

Topología resultante:

```
[Entrada] - [Inicializacion] - [Interprete] - [Dominio] - [Numerica: K_m, M_m, K_t, C_t
    , solvers] - [Salida]
                                                    |
                                  [DOF mecanicos] + [DOF termicos]
                                  [Materiales mec.] + [Materiales term.]
                                  [Elementos mec.]  + [Elementos term.]
```

Riesgo de arquitectura

Resultó **bajo**, frente al riesgo medio estimado. La infraestructura existente absorbió el campo escalar sin generalización previa, lo que constituye una verificación empírica de las decisiones de despacho algebraico e imposición de restricciones frente a un campo físico que no existía cuando se tomaron. La deuda que quedó era de salida, no de arquitectura (la exportación del campo de temperatura a VTK), y se saldó el 22 de septiembre de 2026.

9.7 Fase 6 — Acoplamiento termo-mecánico

Estado: estrategia decidida en el ADR 0016 (22 de septiembre de 2026), sin implementar.

Lo que ya existe. Antes de decidir se verificó sobre el código que la infraestructura admite nodos con desplazamiento y temperatura a la vez: la numeración de ecuaciones itera los grados de libertad por nombre, el ensamblador reúne ambos campos en una sola matriz, la reducción acepta condiciones mixtas sobre un mismo nodo, masa y capacidad calorífica se ensamblan con el mismo operador y un análisis estático de dos campos se resuelve de extremo a extremo. La previsión que figuraba aquí —una nueva infraestructura de ensamblaje multifísico— resultó innecesaria.

Lo que falta es la física de los bloques fuera de la diagonal: la dilatación térmica (el campo de temperatura induce fuerzas mecánicas, efecto de primer orden) y el calor de deformación (efecto de segundo orden salvo en régimen adiabático).

Decisiones del ADR 0016:

- *Acoplamiento débil unidireccional primero*: se resuelve el problema térmico y su campo entra en el mecánico, sin retroalimentación. El monolítico queda como extensión aditiva cuando exista el caso real, sobre la misma infraestructura.
- *Deformación propia genérica* ε_0, con σ `= C:(`ε `-` ε_0`)`: la térmica es el caso particular ε_0 `=` $\alpha \cdot \Delta$`T`, y la higroscópica —dominante en madera y bambú— entra por el mismo canal sin volver a tocar el contrato de material.
- ε_0 vive **por punto de Gauss**, no en el material (compartido por todos los elementos de una familia de lote) ni en el elemento (varía dentro de él).
- *Sin familia de elementos acoplados*: el elemento mecánico interpola el campo de temperatura con sus propias funciones de forma, con mallas conformes.

Riesgo de arquitectura

Bajo, tras la verificación del ADR 0016. El cambio de contrato se limita a un canal aditivo para ε_0 con valor por defecto cero, que deja los materiales existentes intactos.

9.8 Fase 7 — Rendimiento y robustez de la capa numérica

Estado: implementado (ADR 0014, 0015, 0017, 0018 y 0019, 22 y 23 de septiembre de 2026; capa 4 de la red de seguridad, 27 de octubre de 2026).

Ninguna de estas piezas añade un tipo de análisis: cambian cómo se ensambla, cómo se resuelve y qué se comprueba, sin cambiar un resultado.

- **Ensamblaje por lotes** (ADR 0014): la familia de elementos se deriva de los contratos declarativos y se evalúa en un kernel compilado, serie o paralelo, con el camino por elemento como referencia obligatoria; el estado interno se declara (`STATE_SCHEMA`) y se guarda en arreglos.
- **Corrector de Newton compartido** (ADR 0015): un solo bucle de corrección para todos los solvers iterativos; cada solver aporta su problema por paso y su control de paso.
- **Backends algebraicos** (ADR 0017 y 0018): Pardiso multihilo opcional, y un iterativo (CG/MINRES con multimalla algebraica sembrada con los modos de cuerpo rígido) sólo a petición.
- **Red de seguridad del análisis estático** (ADR 0019): mecanismos rígidos detectados antes de resolver y descritos en términos del modelo; mecanismos internos, solución del sistema y pérdida de cifras por mal condicionamiento comprobados después.

```
[Numerica] = [Ensamblaje por familias (Numba)] + [NewtonCorrector] + [Despacho:
    Cholesky | Pardiso | LU | iterativo] + [Red de seguridad]
```

Riesgo de arquitectura

Bajo. Todo es interno a la capa numérica y se verificó contra el camino anterior a precisión de máquina; el contrato que gana un componente nuevo es opcional (`BATCH_KINEMATICS`, `BATCH_KERNEL`) salvo `STATE_SCHEMA`, una línea.

9.9 Fase 8 — Módulos de usuario y frontera del programa principal

Estado: implementado (ADR 0020, 24 de septiembre de 2026; revisión del 29 de septiembre con el criterio de admisión de `Reglas.md` §1).

Las formulaciones no estándar salen del programa principal a módulos de usuario (`solidum/user/<nombre>/`), con carga explícita (`user_modules` en el YAML o `solidum.load_user_module`). El programa principal sólo ofrece piezas genéricas: registros con metadatos de familia, un lector YAML con un solo bucle sobre las familias, referencias declaradas por el elemento (`REFERENCE_KWARGS`), el gancho `prepare_step` y la cinemática pública. Dos módulos: `discontinuities` (discontinuidades interiores embebidas, ADR 0010) y `dissipation_control` (arco por disipación). Un test de pureza vigila que el programa principal no los nombre.

```
[Programa principal: solidum/]  <-- piezas genéricas --  [Modulos de usuario: solidum/
    user/<nombre>/]
```

Riesgo de arquitectura

Bajo. La frontera es aditiva y la vigilan tres controles automáticos (estructural, léxico y en ejecución). Una pieza estándar que hoy sólo usa un módulo sigue en él hasta que la necesite el programa principal; así volvió la familia cohesiva el 10 de noviembre de 2026 (fase 12).

9.10 Fase 9 — Secciones de marco

Estado: implementado (ADR 0021, 29 de septiembre de 2026).

Una cuarta familia de objetos con sección YAML propia, `SectionRegistry` (`sections`), sin estado: el estado de cada fibra vive en el elemento. `FiberSection` da a los marcos Euler-Bernoulli plasticidad distribuida con cualquier material 1D; los marcos aceptan un material elástico con propiedades geométricas o una sección con `n_ip` puntos de Gauss-Legendre.

Riesgo de arquitectura

Bajo para la arquitectura, alto para los modelos existentes: un marco con material inelástico sin sección deja de construirse, porque antes daba flectores incorrectos en silencio.

9.11 Fase 10 — Cargas variables y verificación por soluciones manufacturadas

Estado: implementado (ADR 0022 y 0023, 27 de octubre de 2026).

Las cargas distribuidas aceptan campos de la posición integrados sobre la geometría real, también desde el YAML con expresiones compiladas por un intérprete con lista blanca. El contrato declarativo de la celda de referencia (`REFERENCE_CELL`, `REFERENCE_NODES`) habilita un generador de mallas estructuradas (`solidum.mesh`) y la verificación por soluciones manufacturadas (`solidum.verification`), que recorre el registro: un elemento nuevo queda verificado sin escribir pruebas.

Riesgo de arquitectura

Bajo. Extensiones aditivas de las cargas y de la entrada; la verificación es una herramienta, no una capa.

9.12 Fase 11 — Análisis por etapas y continuidad

Estado: implementado (ADR 0024 a 0031, 9 de noviembre de 2026).

Hasta aquí cada análisis empezaba en `U = 0` y terminaba en sí mismo. Esta fase encadena análisis y hace legible cómo terminó cada uno:

- **Masas de nudo y grados de libertad sin masa** (ADR 0024): la masa de nudo es un dato del dominio; el modal condensa de forma exacta los grados de libertad sin masa y los

transitorios implícitos los admiten con aceleración inicial consistente.

- **Terminación y registro** (ADR 0025): `termination` legible en todos los solvers incrementales y en el tiempo, excepciones tipadas, parada en un valor objetivo de la magnitud controlada (`control_target`) y una política de niveles del registro.
- **Etapas de carga** (ADR 0026): `solve(F, u0=…, constant_load=…)`, `solidum.run(..., previous=…)` y el bloque YAML `stages`.
- **Cargas con sus cargas equivalentes** (ADR 0027): `LoadVector` combina linealmente las cargas equivalentes por elemento con la carga.
- **Longitud característica del punto** (ADR 0028): viaja en el estado del punto de integración para regularizar el ablandamiento (`ManderConcrete1D`).
- **Continuidad en el tiempo** (ADR 0029), **reacciones dinámicas** (ADR 0030) y **análisis lineales sobre el estado de una etapa** (ADR 0031): un transitorio continúa un estático u otro transitorio; el modal, el armónico, el espectral y el lineal linealizan alrededor del estado de la etapa sin cambiarlo.

```
[Etapa 1: estatico] --previous--> [Etapa 2: estatico | transitorio | modal | armonico |
    espectral | lineal] --previous--> ...
                        (estado de los elementos + carga constante + cargas
                            equivalentes)
```

Riesgo de arquitectura

Medio, ya cerrado. Una etapa hereda el estado que guardan los elementos, así que cada solver debía arrancar de un estado ajeno; se resolvió con mezclas comunes (`TransientTerminationMixin`, `LinearizedStateMixin`) y con comprobación de equilibrio al empezar, sin tocar el contrato de elemento ni de material.

9.13 Fase 12 — Interfaces, refuerzo embebido y restricciones generadas

Estado: implementado (ADR 0032 y 0033, 10 de noviembre de 2026).

- **Restricciones generadas** (ADR 0032): una familia de generadores (`ConstraintRegistry`, bloque YAML `constraints`) crea sobre el dominio mallado los nodos, elementos y MPC que su geometría pide; la localización de puntos invierte el mapeo isoparamétrico con lo que todo sólido declara. Primer generador: el refuerzo embebido, con barras de dos o tres nodos (`Truss2D3`, `Truss3D3`).
- **Materiales de interfaz** (ADR 0033): una quinta familia, `InterfaceMaterialRegistry` (`interface_materials`), de leyes tracción–desplazamiento relativo para superficies (grietas) y líneas (barras embebidas). Con `bond`, el refuerzo genera interfaces de adherencia-deslizamiento no registradas; la familia cohesiva del módulo `discontinuities` volvió al programa principal y se amplió con la ley de modo mixto.

- En el módulo `discontinuities`, el seguimiento de grieta (ADR 0010, fase E).

```
[Dominio mallado] -- generador de restricciones --> [nodos + elementos de barra + MPC |
    interfaces de adherencia]
[Materiales de interfaz] --> [interfaces de adherencia (linea)] + [discontinuidades
   embebidas (superficie)]
```

Riesgo de arquitectura

Bajo. Los generadores producen objetos que el resto del programa ya sabía tratar (nodos, elementos, MPC); las interfaces generadas no se registran ni se exportan a VTK, y sólo exigieron dos piezas genéricas (`VTK_EXPORT` en el exportador y `YAML_KEYS` en el lector).

9.14 Mantenimiento de este capítulo

Cada vez que se cierre la implementación de una fase y se acepte su ADR de apertura, esta sección se actualiza:

1. La fase pasa de "*[Pendiente] ...*" a "implementado".
2. La hipótesis inicial sobre la arquitectura se sustituye por la descripción real, ajustada a las decisiones que se hayan tomado durante la implementación.
3. El diagrama de capas se redibuja conforme a la topología efectiva.
4. Las consecuencias no previstas (ajustes en capas anteriores, contratos modificados) se registran en una subsección "Lecciones de la fase N", de cara a fases posteriores.

De este modo, el capítulo se convierte en una crónica ordenada del crecimiento del programa: en cualquier momento futuro, abrir el manual permite reconstruir no solo el estado actual de Solidum FEM sino la trayectoria completa que lo ha llevado hasta él.

Capítulo 10

Compromisos de diseño

Los Architecture Decision Records (ADR) del capítulo 8 registran las decisiones de arquitectura que han sido **tomadas e implementadas**. Este capítulo registra el complemento simétrico: las decisiones que han sido **deliberadamente postergadas o rechazadas**, junto con la justificación de su no adopción. La función es preservar memoria sobre los caminos no recorridos, de modo que cuestiones ya resueltas no se reabran innecesariamente en sesiones futuras y que, si en algún momento las circunstancias del proyecto cambian, la revisión parta de un punto informado y no desde cero.

A diferencia de los ADR, las entradas de este capítulo no se materializan en archivos individuales en `docs/adr/`: viven exclusivamente aquí. Cada entrada se actualiza cuando las circunstancias que motivaron la decisión cambien sustancialmente.

10.1 Paralelización con MPI

Decisión: postergada sin fecha.

Contexto. La paralelización mediante el estándar Message Passing Interface (MPI) sería el paso natural para llevar Solidum FEM a problemas de gran escala (mallas de millones de elementos, descomposición de dominio, ejecución en clúster).

Razones de la postergación. Un solo desarrollador trabaja en el proyecto y los problemas que motivan su uso (investigación en mecánica de sólidos, validación de modelos constitutivos, prototipado de algoritmos) se ejecutan satisfactoriamente en máquinas de un solo nodo. La introducción de MPI multiplicaría el coste de cualquier extensión por la necesidad de mantener consistencia de estado entre procesos, complicaría sustancialmente las pruebas (ejecución determinística reproducible bajo distintas particiones de malla) y exigiría rediseñar el ensamblaje y el subsistema algebraico. El compromiso entre coste de mantenimiento y beneficio inmediato no es favorable mientras el caso de uso del proyecto se mantenga en su perfil actual.

Reapertura de la decisión

Procedería revisar este compromiso si Solidum FEM se aplicara de forma habitual a mallas mayores que 10^6 elementos, o si se incorporasen colaboradores adicionales con experiencia en programación paralela.

10.2 Adopción de PETSc o Trilinos como backend numérico

Decisión: rechazada para el alcance actual.

Contexto. PETSc y Trilinos son las dos bibliotecas estándar para álgebra lineal dispersa, solvers iterativos, descomposición de dominio y gestión de matrices distribuidas en programas de elementos finitos de gran escala. Su adopción daría acceso inmediato a multimalla algebraica, precondicionadores avanzados y soporte MPI maduro.

Razones del rechazo. Ambas son dependencias binarias pesadas con un proceso de instalación no trivial en Windows, plataforma de trabajo del desarrollador. Su modelo de programación es de matriz distribuida y orientación procedural, lo que choca con la arquitectura actual basada en `scipy.sparse` y orientación a objetos. La capacidad de Solidum FEM hoy se ajusta a lo que SuperLU y CHOLMOD ofrecen mediante `scipy.sparse` y `scikit-sparse`; añadir PETSc o Trilinos sin un caso de uso concreto que lo exija supondría una dependencia infrautilizada y un coste de mantenimiento desproporcionado.

Reapertura de la decisión

Procedería si se adopta MPI (compromiso anterior) y, simultáneamente, los problemas tratados requirieran solvers iterativos con precondicionamiento avanzado. En ese escenario, PETSc se convertiría en la opción natural por encima de implementar dichos algoritmos en código propio.

10.3 Arquitectura cliente-servidor o servicio web

Decisión: rechazada.

Contexto. Una arquitectura cliente-servidor expondría Solidum FEM como un servicio remoto al que la GUI externa (SolidumBAR, antes FenixBAR) y otros consumidores se conectarían vía protocolo HTTP, gRPC o similar. Es el modelo dominante en simulación industrial moderna como servicio.

Razones del rechazo. El consumidor actual (SolidumBAR) es local, está bajo control del mismo desarrollador y no tiene requisitos de concurrencia ni de despliegue distribuido. La interfaz pública se ha estabilizado mediante el ADR 0002 con `SolveResult`, `solidum.run` y `solidum.run_yaml` en proceso, lo que cubre el caso de uso de consumidor externo sin la sobrecarga de un protocolo de red. La introducción de un servidor exigiría serialización de mallas y resultados, gestión de sesiones, autenticación si el servicio fuera multiusuario, y un ciclo de despliegue paralelo al del programa. Ninguna de estas necesidades existe hoy.

Reapertura de la decisión

Procedería si Solidum FEM se ofreciera como servicio público a múltiples consumidores no relacionados con el desarrollo, o si la GUI se desacoplara hasta el punto de ejecutarse en máquinas distintas.

10.4 Soporte de múltiples lenguajes en la interfaz pública

Decisión: rechazada para el alcance actual.

Contexto. La interfaz pública del programa (`solidum.run`, `solidum.run_yaml`, `SolveResult`) es accesible exclusivamente desde Python. Una alternativa sería exponer la misma interfaz en C, C++ o como biblioteca compartida llamable desde Fortran y MATLAB, lo que ampliaría el espectro de consumidores potenciales.

Razones del rechazo. El consumidor identificado (SolidumBAR) está escrito en Python; el ecosistema de programas docentes y de investigación al que se dirige Solidum FEM utiliza Python como lenguaje dominante. Ofrecer un binding multilingüe duplicaría el esfuerzo de mantenimiento de la interfaz pública sin un caso de uso concreto que lo justifique. La integración con MATLAB, si fuera necesaria en algún momento, se realizaría mediante el mecanismo nativo de MATLAB para llamar a Python (`py.solidum.run_yaml(path)`), sin necesidad de modificar Solidum FEM.

Reapertura de la decisión

Procedería si surgiera un consumidor estable escrito en un lenguaje distinto de Python y la solución *vía* el puente nativo de cada lenguaje no fuera viable.

10.5 Sistema de plugins externo al repositorio

Decisión: rechazada.

Contexto. Algunos programas grandes de elementos finitos permiten que terceros publiquen materiales, elementos o solvers como paquetes externos que se descubren en tiempo de ejecución sin formar parte del repositorio principal. La alternativa es la actual: todo componente vive dentro del repositorio de Solidum FEM y se valida con sus pruebas.

Razones del rechazo. La filosofía del proyecto (ver capítulo 1) considera la transparencia y la auditabilidad del código como valor primordial. Un sistema de plugins externos abriría la puerta a componentes no auditados que podrían silenciosamente romper la convención de signos del proyecto, los contratos declarativos del catálogo o las hipótesis de validación contra soluciones analíticas. Mientras el desarrollo lo lleve un único equipo, la incorporación de cualquier componente nuevo pasa por el repositorio y por las pruebas correspondientes, lo que mantiene la trazabilidad completa.

Reapertura de la decisión

Procedería si el proyecto creciera hasta tener una comunidad amplia de contribuyentes externos con necesidades especializadas no absorbibles en el catálogo principal. Hoy no es el caso ni se prevé que lo sea.

10.6 Refactor a tipado estático estricto con `mypy --strict`

Decisión: postergada sin fecha.

Contexto. El código de Solidum FEM utiliza anotaciones de tipo en sus interfaces públicas pero no se valida con `mypy` en modo estricto. Adoptarlo proporcionaría garantías estáticas adicionales y detectaría algunas familias de errores antes de la ejecución.

Razones de la postergación. La librería `numpy` y, en menor medida, `scipy.sparse` tienen un soporte de tipado estático que ha mejorado mucho pero sigue dejando puntos donde `mypy --strict` produce ruido sin valor real. El esfuerzo de pulir todas esas fricciones no se compensa, en el alcance actual del proyecto, con el beneficio en detección de defectos. La política presente — anotaciones

de tipo en interfaces públicas y validación de comportamiento mediante pruebas de física — se considera suficiente.

Reapertura de la decisión

Procedería cuando el ecosistema NumPy/SciPy cierre las lagunas de tipado restantes y la activación del modo estricto deje de generar fricción artificial.

10.7 Mantenimiento de este capítulo

Cada vez que se reabra una decisión y se modifique su estado (de "postergada" a "rechazada", o viceversa), o cuando una decisión postergada se materialice en un ADR formal, la entrada correspondiente se actualiza. Si la decisión pasa a aceptarse, su entrada se retira de este capítulo y se sustituye por el ADR correspondiente en el capítulo 8.

Capítulo 11

Glosario

Términos técnicos centrales de Solidum FEM, ordenados alfabéticamente. Cada entrada da una definición breve (de una a tres frases) y, cuando procede, remite al capítulo donde el concepto se desarrolla.

Adherencia-deslizamiento. Transmisión de fuerza entre una barra de refuerzo y el sólido que la rodea a través de una ley entre el esfuerzo de adherencia y el deslizamiento relativo de la barra, en lugar de la unión perfecta. En Solidum la genera el refuerzo embebido con `bond`: una interfaz `BondSlipInterface` por tramo de barra, con una ley de interfaz de línea (`BondSlipModelCode`, `BondSlipMultilinear`). ADR 0032 y 0033.

Ajuste de paso (incremento de carga). División de la carga total en fracciones que se aplican sucesivamente en problemas no lineales. Permite que el método de Newton-Raphson converja a partir de una aproximación cercana a la solución del paso anterior.

Corrotacional (formulación). Descomposición del movimiento de un elemento en una rotación de cuerpo rígido más una deformación local pequeña. La ley constitutiva trabaja en la configuración local; la cinemática del elemento absorbe las grandes rotaciones. Aplicable a armaduras, cables y marcos en régimen geométricamente no lineal con deformación pequeña.

Architecture Decision Record (ADR). Documento breve que registra una decisión de arquitectura junto con su contexto, alternativas consideradas y consecuencias. Reside en `docs/adr/000N-titulo.md`. Ver capítulo 8.

`ArcProblem`. Problema de Newton por paso del arc-length (iterado `(U, λ, ΔU)`, dos resoluciones por iteración y la restricción del paso como método `constraint`), exportado con nombre público en `solidum.math.solvers`. Una variante de arco que sólo cambia la restricción lo hereda y redefine `constraint`: así lo hacen `IndirectDisplacementSolver` en el programa principal y `DissipationArcLengthSolver` en el módulo de usuario `dissipation_control`. Ver capítulo 5, ADR 0015 (protocolo `NewtonProblem`) y ADR 0020.

Autodiscover (descubrimiento automático). Procedimiento que recorre las carpetas canónicas del proyecto e importa todos sus módulos al inicio del programa, lo que dispara la ejecución de los decoradores `@register` y, con ello, la inscripción de cada componente en su registro. No recorre `solidum/user/`: los módulos de usuario sólo se cargan cuando un caso los pide. Ver capítulo 5.

Capa algebraica. Subsistema en `solidum/math/linalg/` responsable de resolver el sistema lineal `K · x = b` que aparece dentro de cada iteración del solver no lineal. Está separada del solver no lineal y selecciona automáticamente el algoritmo de factorización adecuado mediante un despachador. Ver capítulo 4 y ADR 0003.

Corrector de Newton. Bucle de corrección compartido por los solvers iterativos (`NewtonCorrector`, ADR 0015): ensambla una vez en el iterado, evalúa el criterio dual con el residuo de ese ensamblaje y el incremento previo, resuelve con el backend algebraico (degradación a LU, factorización congelada opcional), aplica el line search opcional y comitea al converger. Cada solver le entrega un `NewtonProblem` con su residuo, su sistema tangente y su actualización, y conserva su propio control de paso. Ver capítulos 3 y 4.

Catálogo. Documento navegable que enumera todos los componentes implementados de una categoría (elementos, materiales, solvers, restricciones generadas) con sus parámetros, validez y referencias. Reside en `docs/catalogo_<categoría>.md`. Es índice y resumen; el detalle vive en las especificaciones.

Contrato declarativo. Mecanismo por el cual una clase declara, mediante atributos de clase como `STRAIN_DIM`, `DOF_NAMES`, `N_INTEGRATION_POINTS`, `PRIMARY_STATE_VAR` o `IS_LINEAR_ELASTIC`, las propiedades fijas que el sistema necesita conocer de ella. La clase base lee estos atributos y se autoconfigura, sustituyendo métodos de inicialización repetitivos en cada subclase. Ver capítulo 5.

Despachador algebraico. Función `select_solver(props, override)` que recibe las propiedades de la matriz de rigidez (simetría, definición positiva) y devuelve el algoritmo adecuado: Cholesky cuando la matriz es simétrica definida positiva y CHOLMOD está instalado, Pardiso si está instalado (ADR 0017) y LU en otro caso; el iterativo sólo se usa a petición (ADR 0018). Ver ADR 0003.

`DOF_NAMES`. Atributo de clase de cada elemento que enumera los nombres de los grados de libertad activos en cada uno de sus nodos. La clase base lo lee para registrar los grados de libertad globales en el `Domain`.

`Domain`. Objeto central de la capa de dominio que mantiene la lista de nodos, la conectividad de los elementos, la numeración global de los grados de libertad y, tras la solución, el agregado `SolveResult`.

Etapa (de análisis). Análisis que parte del estado que dejó otro: los desplazamientos, lo que guardan los elementos y la carga que ya actúa, que pasa como carga constante. Así se aplican la gravedad y después el empuje lateral, un transitorio tras un estático o un modal sobre una estructura cargada. Desde Python, `solidum.run(..., previous=resultado)`; desde el YAML, el bloque `stages`. ADR 0026, 0029 y 0031.

Eliminación directa (de condiciones de Dirichlet). Estrategia de imposición de condiciones esenciales que retira los grados de libertad prescritos del sistema antes de resolverlo, en oposición a la penalización (que añade un valor grande a la diagonal) y a los multiplicadores de Lagrange (que amplían el sistema). Toda restricción se expresa en forma afín `u_s = g_s + Σ α_si · u_mi` y se acumula en un `ConstraintSet` (`solidum/bc/`). El ensamblador construye un operador disperso `T` y un vector `g` tales que `u = T · u_libre + g`, y entrega al solver el sistema reducido `K_red = T⊤KT`, `F_red = T⊤(F − K · g)`. La imposición es exacta a redondeo y preserva la simetría y la positividad definida de la matriz original. Ver ADR 0004.

Elemento. Entidad geométrica que construye su matriz de gradientes `B`, su matriz de rigidez tangente y su vector de fuerzas internas a partir de los desplazamientos nodales y del material asignado (o, en un marco, de su sección). Sus contratos declarativos están en `solidum/elements/`. `Element.materials()` enumera todos los materiales que usa —el suyo, o los de las fibras de su sección— para las comprobaciones agregadas de masa, peso propio y simetría (ADR 0021). Ver capítulo 4.

`ElementState`. Objeto que encapsula las dos copias de las variables internas de cada elemento: una copia *trial* (la que se explora durante las iteraciones del solver) y otra comprometida (la del

último paso convergido). Ver capítulo 5.

Ensamblaje con topología cacheada. Estrategia de ensamblaje en la que la primera invocación calcula los pares de índices `(i, j)` de cada contribución elemental en formato Coordinate (COO), deriva la estructura CSR de la matriz y un mapa COO → CSR; las invocaciones siguientes reescriben únicamente el vector de datos y lo acumulan sobre la misma estructura, sin recalcular índices ni reordenar. Ver capítulo 5 y ADR 0014.

Ensamblaje por lotes. Evaluación de todos los puntos de Gauss de una familia de lote dentro de un único kernel compilado (`solid_family_kernel`, con variante paralela `solid_family_kernel_parallel` bit a bit equivalente), que recibe la cinemática del elemento y la constitutiva del material como funciones tipadas por su firma. El post-proceso por familia (`Family.gauss_state`) usa las mismas funciones. Coexiste con el camino por elemento, que sigue siendo el contrato obligatorio y la referencia física. Ver capítulo 5 y ADR 0014.

Esfuerzo plano / deformación plana. Hipótesis de modelado para problemas bidimensionales. El esfuerzo plano asume que el esfuerzo perpendicular al plano es nulo (placas delgadas); la deformación plana asume que la deformación perpendicular es nula (cuerpos extensos en una dimensión, secciones de tuberías largas).

Especificación (spec). Documento que describe un componente físico nuevo antes de su implementación: especificación física, formulación numérica, contrato YAML y criterios de aceptación. Reside en `docs/specs/<Nombre>.md`. Es a la vez orden de trabajo y referencia detallada del componente. Ver capítulo 7.

Estado *trial* y estado comprometido. Las dos copias de las variables internas de cada elemento. El estado *trial* se explora durante las iteraciones del solver no lineal; al converger un paso, el solver invoca `commit_state()` y promueve el *trial* a estado comprometido. Esta semántica es indispensable en problemas con plasticidad o daño para no contaminar el historial con tentativas posteriormente descartadas.

Familia de elementos. Conjunto de elementos que comparten la cinemática esencial: armaduras, cables, marcos (vigas), sólidos 2D, sólidos 3D, cáscaras. Ver capítulo 4.

Familia de lote. Conjunto de elementos de un dominio que comparten clase de elemento, instancia de material, regla de cuadratura y número de nodos, y que el ensamblador evalúa juntos en un kernel compilado. Su estado interno vive en un `FamilyState` (arreglos por punto de Gauss) del que `elem.state` es una vista (`BatchedElementState`). Ver capítulo 5 y ADR 0014.

Familia de material. Registro que declara la sección de primer nivel del YAML con sus objetos (`YAML_SECTION`): `materials` para los mecánicos, `thermal_materials` para los térmicos e `interface_materials` para las leyes tracción–desplazamiento relativo de una interfaz (ADR 0033). `Registry.families()` las enumera, en su orden de definición, junto con las familias que no son de materiales (hoy, la de secciones de marco, `sections`), y el intérprete del YAML las recorre sin conocerlas por nombre; un elemento declara a qué familia pertenecen sus materiales o su sección con `REFERENCE_KWARGS`. Ver capítulo 5, ADR 0020 y ADR 0021.

`from_yaml` (gancho). Método de clase opcional `from_yaml(data, resolve)` de una clase de familia con referencias anidadas a otras familias (una sección por fibras referencia materiales dentro de sus parches). El intérprete del YAML lo llama en lugar de `cls(**datos)` y le pasa `resolve(registro, id, contexto)`, que devuelve el objeto de una familia ya construida o lanza un error con el contexto de la referencia. Las familias se construyen en orden de definición. Ver capítulo 5 y ADR 0021.

Gradiente conjugado (CG) y MINRES. Métodos iterativos de Krylov para sistemas lineales

simétricos: CG para matrices definidas positivas, MINRES para indefinidas. El solver iterativo de Solidum pasa de uno a otro de forma automática al detectar curvatura negativa. Capítulo 4, ADR 0018.

Generador de restricciones (restricción generada). Objeto que, sobre un dominio ya mallado, crea los nodos, los elementos y las restricciones multipunto que pide una geometría, antes de numerar las ecuaciones (`ConstraintRegistry`, bloque YAML `constraints`, `solidum/bc/`). El primero es el refuerzo embebido. ADR 0032.

Grado de libertad (DOF). Cada incógnita escalar del problema discreto. En estática mecánica, los grados de libertad son los desplazamientos nodales y, en su caso, las rotaciones nodales. La numeración global se establece en el `Domain`.

Grado de libertad sin masa. Grado de libertad en el que la matriz de masa es nula (masas sólo en algunos nudos, elementos sin densidad, giros sin inercia rotacional). El análisis modal los condensa estáticamente de forma exacta y los transitorios implícitos los admiten con una aceleración inicial consistente. ADR 0024.

`IS_LINEAR_ELASTIC`. Atributo de clase de cada material (`False` por defecto) que declara que su respuesta es lineal y sin historia: `True` en `Elastic1D`, `Elastic2D`, `Elastic3D` y `Orthotropic2D`. Los marcos lo exigen en su camino de forma cerrada (`material` + propiedades geométricas) y rechazan al construir cualquier otro material; la no linealidad material de un marco entra por su sección. Ver capítulo 5 y ADR 0021.

`LoadVector`. Vector de cargas que lleva consigo las cargas equivalentes por elemento de su parte distribuida y las combina linealmente con la carga (`F_a` $\pm$ `F_b`, `a`·`F`). Así las fuerzas internas de los elementos cargados son correctas con varias cargas, casos o etapas. ADR 0027.

Localización de puntos. Búsqueda del elemento sólido que contiene un punto y de sus coordenadas naturales, por inversión del mapeo isoparamétrico con Newton a partir de lo que el elemento declara (`REFERENCE_CELL`, funciones de forma), sin código por tipo de elemento (`solidum/mesh/point_location.py`). La usan los generadores de restricciones. ADR 0032.

Longitud característica (del punto de integración). Longitud `h` asociada a cada punto de integración con la que un material con ablandamiento regularizado por energía de fractura ajusta su rama descendente para disipar `G_f/h` por unidad de volumen. La siembra el elemento al construirse y viaja en el estado del punto; el material declara si la usa (`uses_characteristic_length`). ADR 0028.

Longitud de arco (método de Crisfield). Estrategia de solución no lineal que introduce el factor de carga como incógnita adicional y añade una restricción geométrica sobre la trayectoria en el espacio (U, λ). Permite recorrer ramas con derivada infinita o negativa, imprescindible para problemas con *snap-back* o *snap-through*.

Masa de nudo. Masa concentrada declarada como dato del dominio en grados de libertad de nudos, traslacionales o rotacionales (`Domain.add_nodal_mass`, bloques YAML `nodal_masses*`). Es sólo inercia: no genera peso con `gravity`. ADR 0024.

Material. Ley constitutiva que, dada una deformación y un estado interno, devuelve el esfuerzo y el módulo tangente consistente. Su contrato declarativo está en `solidum/materials/`. Ver capítulo 4.

Material de interfaz. Ley entre la tracción sobre una interfaz y el desplazamiento relativo entre sus dos lados, en el sistema local de la interfaz (normales primero): de superficie para grietas y juntas, de línea para barras embebidas. Familia propia (`InterfaceMaterialRegistry`, sección YAML `interface_materials`, base `InterfaceMaterial`). ADR 0033.

Mecanismo. Modelo cuyos apoyos o cuya conectividad permiten un movimiento sin deformación: la matriz de rigidez es singular y el problema estático no tiene solución única. Si el movimiento es de sólido rígido del conjunto, Solidum lo detecta antes de resolver y lo describe; si es interno (una rótula de más), lo detecta tras resolver. Capítulo 5, ADR 0019.

Método de Elementos Finitos (MEF). Técnica de discretización para resolver ecuaciones diferenciales parciales mediante la subdivisión del dominio en elementos sobre los que se interpolan los campos incógnita. Solidum FEM trabaja en aproximación de desplazamientos.

Modos de cuerpo rígido. Movimientos de un modelo sin apoyos que no producen deformación: traslaciones y giros infinitesimales, y el valor constante en un campo escalar. En Solidum se derivan del nombre de los grados de libertad y los usan el precondicionador multimalla y la detección de mecanismos. Capítulo 5.

Módulo de usuario. Formulación no estándar que vive fuera del programa principal, en su propia carpeta `solidum/user/<nombre>/`, a la manera de los elementos y materiales de usuario de FEAP. El programa principal no la importa ni la nombra: un caso la carga con `user_modules: [<nombre>]` en el YAML o con `solidum.load_user_module` en Python. El primero es `discontinuities` (discontinuidad interior embebida); el segundo, `dissipation_control` (arco por disipación de energía, un solver). Ver capítulo 5 y ADR 0020.

Multimalla algebraico (AMG). Precondicionador que resuelve el sistema en una jerarquía de problemas cada vez más pequeños construidos a partir de la propia matriz. Con los modos de cuerpo rígido como casi-núcleo, el número de iteraciones apenas depende del tamaño del modelo. ADR 0018.

Multipoint constraint (MPC). Restricción afín lineal que liga el desplazamiento de un grado de libertad esclavo a una combinación lineal de los desplazamientos de uno o varios grados de libertad maestros, posiblemente con un término independiente. Modelan apoyos en plano oblicuo, periodicidad de celda unitaria, uniones rígidas entre nodos y simetrías no alineadas con los ejes globales. Se declaran mediante `Domain.add_linear_constraint` o desde el bloque `linear_constraints` del archivo YAML, y se imponen por eliminación directa con la misma maquinaria que las condiciones de Dirichlet. Ver capítulo 5 y ADR 0004.

Notación de Voigt. Representación de un tensor simétrico de segundo orden como un vector. En 2D, los tres componentes son las dos componentes normales y la componente de cortante (Voigt-3); en 3D, los seis componentes son las tres normales y las tres de cortante (Voigt-6).

Numba (compilación Just-In-Time, JIT). Decorador `@njit` que compila a código nativo, en la primera invocación, las funciones a las que se aplica. La primera ejecución asume el coste de compilación; las siguientes corren a velocidad cercana a Fortran. Restringido a tipos primitivos y arreglos NumPy.

Pivote pequeño. Valor de la diagonal de la factorización pequeño frente a la diagonal de la matriz ($|U_{ii}|/\sqrt{(|K_{rr}|\cdot|K_{cc}|)} < 10^{-8}$): puede delatar una matriz singular, pero su tamaño también depende del contraste de rigideces y del tamaño del modelo. Desde el 20 de octubre de 2026 Solidum lo usa sólo como filtro: el veredicto sobre un mecanismo interno lo da la energía por elemento del modo de menor energía. ADR 0019.

Perturbación lineal. Análisis lineal estático (`LinearSolver`) sobre el estado de una etapa, con la rigidez tangente de los elementos y sin cambiar ese estado: da casos superponibles sobre la estructura cargada (`SolveResult.perturbation`). ADR 0031.

`PRIMARY_STATE_VAR`. Atributo de clase de cada material que declara cuál de sus variables internas es la principal a efectos de visualización. El exportador VTK la lee genéricamente sin conocimiento del material que la origina.

Programa principal. Solidum estándar: los elementos, materiales y solvers de elementos finitos clásicos, es decir, todo `solidum/` salvo `solidum/user/`. Admite un componente según el criterio de admisión de Reglas.md §1 (textos de referencia y código general establecido). No conoce los módulos de usuario; el test de pureza (`tests/test_main_program_purity.py`) lo vigila con tres controles: estructural (ningún import de un módulo de usuario), léxico (ninguna mención de lo que un módulo registra, con los nombres derivados automáticamente) y en ejecución (`import solidum` no carga ningún módulo). Ver capítulo 5 y ADR 0020.

Registro (registry). Diccionario global por categoría (`MaterialRegistry`, `ThermalMaterialRegistry`, `InterfaceMaterialRegistry`, `SectionRegistry`, `ElementRegistry`, `SolverRegistry`, `ConstraintRegist` que mapea el nombre de cada componente con su clase. Todos son subclases de la clase genérica `Registry` y declaran sus metadatos con atributos públicos de clase: `YAML_LABEL`, la etiqueta de un objeto del registro en los mensajes de error, que tiene todo registro (`"elemento"`, `"solver"`, `"material térmico"`, `"material de interfaz"`, `"restricción"`) y sustituye al antiguo atributo privado `_kind`, ya retirado; `SPEC_KIND`, el tipo de spec de sus componentes; y, sólo en las familias (de material o de secciones), `YAML_SECTION`. Cada clase se inscribe automáticamente en su registro mediante un decorador. Un módulo de usuario sólo puede añadir entradas: reasignar una existente hace fallar y deshacer su carga. Ver capítulo 5.

Refuerzo embebido. Barra de refuerzo con su propia malla de elementos de línea que atraviesa una malla de sólidos sin coincidir con sus nodos y queda atada al campo interpolado del sólido (adherencia perfecta) o unida a él por interfaces de adherencia-deslizamiento (`EmbeddedReinforcement`). ADR 0032.

Regla de la mano derecha (RHR). Convención de orientación tridimensional para ejes y vectores momento, adoptada universalmente en Solidum FEM para convenciones de signos de magnitudes vectoriales en 3D. Ver capítulo 6.

Relleno (fill-in). Entradas no nulas que aparecen en los factores de una factorización directa donde la matriz tenía ceros. Crece con el tamaño del modelo y domina el tiempo y la memoria de un solver directo: de $\times 10$ a $\times 42$ la matriz original entre 4 000 y 53 000 grados de libertad. ADR 0017.

Retorno radial (return mapping). Algoritmo predictor-corrector para integrar la ecuación constitutiva de plasticidad J2: se asume un paso elástico (predictor), se evalúa el criterio de fluencia y, si el esfuerzo predictor lo viola, se proyecta de vuelta a la superficie de fluencia (corrector). El módulo tangente consistente se obtiene linealizando el algoritmo discreto.

Seguimiento de grieta. Procedimiento del módulo de usuario `discontinuities` que decide, entre pasos, por dónde continúa una grieta embebida de un elemento al siguiente, para que las particiones de elementos vecinos sean coherentes: global por región conexa (`GlobalCrackTracking`) o con una trayectoria prescrita (`PrescribedCrackPath`). ADR 0010, fase E.

Sección de marco. Ley constitutiva de un marco a nivel de sección transversal (ADR 0021): recibe las deformaciones generalizadas del eje en un punto de Gauss, `[`ε_0`, `κ`_z]` o `[`ε_0`, `κ`_y, `κ`_z]`, y devuelve las fuerzas internas `[N, M_z]` o `[N, M_y, M_z]` y su tangente. Familia propia (`SectionRegistry`, sección YAML `sections`, `kind: section`), con la base abstracta `Section` en `solidum/core/section.py`. No guarda estado: la historia de sus puntos materiales en cada punto de Gauss vive en el elemento. La consumen los marcos Euler-Bernoulli con su parámetro `section`. Ver capítulo 4.

Sección por fibras (`FiberSection`). Sección de marco discretizada en fibras `(y, z, A, material)`, referidas al eje del elemento, cada una con su material 1D y su historia; las fuerzas internas son las sumas de ``σ`_i A_i` ponderadas por `1`, `z_i` y `-y_i`. Da plasticidad y daño

distribuidos en la sección. Las fibras se generan con parches rectangulares y circulares, capas de barras y fibras sueltas. Ver capítulo 4, ADR 0021 y la spec `FiberSection`.

Solver directo / solver iterativo. El directo factoriza la matriz y resuelve por sustitución: robusto ante cualquier condicionamiento, pero su memoria crece más deprisa que el modelo. El iterativo aproxima la solución hasta una tolerancia sin factorizar: memoria proporcional al modelo, pero sensible al condicionamiento. Capítulo 4, ADR 0017 y 0018.

`SolveResult`. Agregado inmutable que el `Domain` construye al final de la solución. Contiene los desplazamientos globales, las cargas aplicadas, las reacciones y las fuerzas internas (N, V, M, T) por elemento. Es la interfaz pública para consumidores externos. Ver ADR 0002.

Solver no lineal. Componente que orquesta la solución de un problema no lineal: subdivisión en pasos, iteraciones internas, criterio de convergencia. En el programa principal de Solidum FEM, los estáticos no lineales son `NonlinearSolver`, `ArcLengthSolver` e `IndirectDisplacementSolver`, y los transitorios no lineales `NewtonNewmarkSolver` y `NewtonHHTSolver`; el módulo de usuario `dissipation_control` añade `DissipationArcLengthSolver`. Es nivel estratégico, distinto del subsistema algebraico. Ver capítulo 4.

`STRAIN_DIM`. Atributo de clase de cada material y elemento que declara la dimensión del vector de deformación que manejan: 1 (escalar axial), 3 (Voigt-3 bidimensional), 6 (Voigt-6 tridimensional). El sistema lo usa para validar la compatibilidad material↔elemento en el momento de construcción.

Terminación. Cómo terminó un análisis, legible en atributos y no sólo en el registro: `termination` (`max_lambda`, `control_target`, `max_steps`, `t_end` o `diverged`), `completed`, el número de pasos y el último estado convergido, también tras una excepción. ADR 0025.

Validación temprana en construcción. Comprobación que la clase base de elemento ejecuta cuando se le asigna un material: verifica que la dimensión de deformación del material coincida con la del elemento. Errores físicos de combinación se detectan en la construcción del caso, no doscientas iteraciones después.

Visualization Toolkit (VTK). Conjunto de formatos de archivo y bibliotecas para representación de mallas y campos discretos en visualizadores de simulación numérica como ParaView. El `VtkExporter` de Solidum FEM produce los archivos correspondientes.

YAML (YAML Ain't Markup Language). Formato de serialización legible por humanos en el que se describen los casos de Solidum FEM (nodos, materiales, elementos, condiciones de contorno, cargas, solver). Es contrato público del programa.

Apéndice A

Pendientes de desarrollo

Este apéndice consolida, en una sola vista, todos los elementos marcados a lo largo del manual como pendientes de desarrollo. Cada entrada indica el capítulo de procedencia y el texto del pendiente. La lista se genera de forma automática a partir de los marcadores `[PENDIENTE: ...]` insertados en las fuentes; cualquier modificación se realiza sobre el archivo fuente correspondiente, no sobre este apéndice.

Bloques funcionales

- *[Pendiente] Modelo de Mohr-Coulomb 2D con esquinas (multi-superficie) para mayor fidelidad geotécnica respecto a Drucker-Prager.*
- *[Pendiente] Endurecimiento cinemático (Bauschinger) en J2 para plasticidad cíclica y fatiga.*
- *[Pendiente] Hiperelasticidad isótropa (Neo-Hooke, Mooney-Rivlin, Ogden).*
- *[Pendiente] Viscoplasticidad de Perzyna y Duvaut-Lions.*
- *[Pendiente] Hiperelasticidad incompresible con tratamiento mixto desplazamiento-presión.*
- *[Pendiente] Drucker-Prager en esfuerzo plano (la proyección con `σ_zz=0` acoplada a flujo dilatante queda fuera del alcance de la versión actual).*
- *[Pendiente] Armadura 2D de segundo orden corrotacional.*
- *[Pendiente] Armadura 2D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Armadura 3D de segundo orden corrotacional.*
- *[Pendiente] Armadura 3D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Cable 2D corrotacional con ley constitutiva no lineal a tracción (relajación, fluencia, daño a tracción).*
- *[Pendiente] Cable 3D corrotacional con ley constitutiva no lineal a tracción.*
- *[Pendiente] Cable 2D de segundo orden corrotacional.*

- *[Pendiente] Cable 2D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Cable 3D de segundo orden corrotacional.*
- *[Pendiente] Cable 3D de segundo orden corrotacional con material no lineal.*
- *[Pendiente] Marco 3D Euler-Bernoulli corrotacional con grandes rotaciones.*
- *[Pendiente] Marco 3D Euler-Bernoulli corrotacional con material no lineal.*
- *[Pendiente] Sección por fibras de `Frame2DTimoshenko`, etapa 2 del ADR 0021; hoy un material inelástico se rechaza al construir.*
- *[Pendiente] Marco 3D Timoshenko con deformación por cortante.*
- *[Pendiente] Marco 3D Timoshenko con material no lineal.*
- *[Pendiente] Marco 2D Timoshenko corrotacional.*
- *[Pendiente] Marco 2D Timoshenko corrotacional con material no lineal.*
- *[Pendiente] Marco 3D Timoshenko corrotacional.*
- *[Pendiente] Marco 3D Timoshenko corrotacional con material no lineal.*
- *[Pendiente] Marco de segundo orden (interpolación cuadrática) en cualquiera de las formulaciones anteriores.*
- *[Pendiente] Elemento en fuerzas con sección por fibras (Spacone, Filippou y Taucer 1996), un elemento por miembro; descartado en la etapa 1 del ADR 0021 a favor del elemento en desplazamientos.*
- *[Pendiente] Regularización del ablandamiento en las fibras (deuda #29 de STATUS).*
- *[Pendiente] Marco con sección variable a lo largo del eje (sección no prismática).*
- *[Pendiente] Consumo por `Frame2DTimoshenko` (etapa 2 del ADR 0021, con su cinemática de cortante).*
- *[Pendiente] Evaluación vectorizada de las fibras por material y marcos por el camino por lotes del ADR 0014, cuando la medición lo justifique.*
- *[Pendiente] Sólido 2D con no linealidad geométrica (formulación lagrangiana actualizada o total).*
- *[Pendiente] Elementos de cáscara y lámina (formulación de Mindlin-Reissner para cáscaras gruesas; formulación de Kirchhoff-Love para cáscaras delgadas).*
- *[Pendiente] Elementos mixtos desplazamiento-presión (Q1-P0, Taylor-Hood) para problemas casi incompresibles.*

- *[Pendiente] Elementos de interfaz de espesor nulo entre superficies, para juntas y grietas cohesivas de trayectoria conocida y para contacto con fricción. La familia de leyes ya existe (materiales de interfaz, ADR 0033); hoy la consumen la interfaz de adherencia barra–sólido, que genera el refuerzo embebido, y la discontinuidad embebida del módulo de usuario.*
- *[Pendiente] Newton-Raphson con búsqueda lineal (line-search) para mejorar la robustez ante no convergencia.* ***Parcialmente entregado por ADR 0011*** *— line search disponible como opt-in (`line_search=True`); default `False` por la justificación bibliográfica de Borst & Sluys 1999.*
- *[Pendiente] Solver cuasi-Newton tipo BFGS para reducir el coste de actualización de la matriz tangente.*
- *[Pendiente] Generalized-α, Bossak-α — esquemas adicionales de la familia con disipación numérica.*
- *[Pendiente] Solver de relajación dinámica para búsqueda de configuraciones de equilibrio en estructuras flexibles.*
- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, regla 100/30/30) y multi-support seismic excitation.*
- *[Pendiente] Control del Δt por estimación del error local en los transitorios. Hoy el paso sólo se subdivide cuando el Newton de Newton-Newmark o Newton-HHT no converge.*
- *[Pendiente]* `LDLTSolver` *— factorización $LDL^\top$ con conteo de inercia para el caso simétrico indefinido (deuda técnica #7). Pardiso puede aportarla, pero exige el triángulo superior de la matriz.*
- *[Pendiente] solver iterativo para matrices no simétricas (GMRES o BiCGSTAB); hoy esas matrices van siempre a un solver directo.*
- *[Pendiente] Conducción no lineal — conductividad dependiente de la temperatura, radiación y cambio de fase; los tres exigen iteraciones de Newton dentro de cada paso, que el esquema actual evita por hipótesis de linealidad.*
- *[Pendiente] Condición de frontera de convección (Robin), q = h (T - T_inf); añade un término a la matriz de conductividad y otro al vector de cargas, sin alterar la estructura del solver.*
- *[Pendiente] Acoplamiento termo-mecánico desacoplado (staggered) para casos en que la influencia mecánica sobre el campo térmico sea despreciable.*
- *[Pendiente] Acoplamiento termo-mecánico monolítico para problemas con fuerte interacción bidireccional.*
- *[Pendiente] Excitación sísmica multi-direccional simultánea (CQC3, 100/30/30) y multi-support para diferencias de movimiento entre apoyos.*

Evolución de la arquitectura

- *[Pendiente] ...*

Solidum FEM — Manual de Arquitectura

Sigla del manual: `FF-MA`

Compilado el 11 de noviembre de 2026, 10:12

Commit Git: `bfff90e`

Documento generado automáticamente desde los archivos fuente en `manuals/sources/architecture/` mediante `python manuals/build_architecture_manual.py`. No editar directamente el PDF: cualquier corrección debe realizarse sobre el archivo fuente correspondiente y regenerar el manual.